DēGlitch Software Requirements Specification

Context-Aware Adaptive Connectivity | Version 1.4

Elijah Fullah

29 October 2026

Table of Contents

- Document Control
- 1. Introduction
- 2. Product description
- 3. System context and model
- 4. Requirements baseline
  - 4.1 External interfaces
  - 4.2 Functional requirements
  - 4.3 Quality, security, reliability, availability and observability requirements
  - 4.4 Compliance, installation, build, maintenance, portability, release validation and change requirements
  - 4.5 Conditional AI/ML requirement
  - 4.6 Identity, onboarding and account lifecycle requirements
  - 4.7 Network profiles and declared preferences
  - 4.8 AI assistant and personalization requirements
  - 4.9 Endpoint discovery and network optimization requirements
  - 4.10 Notifications, localization, accessibility and privacy requirements
  - 4.11 Control-plane, operations and model-governance requirements
  - 4.12 Allocation, priority, applicability and status register
- 5. Use-case specifications and flow diagrams
  - UC-01 — Connect or disconnect using a valid local profile
  - UC-02 — Import, validate and manage a local profile
  - UC-03 — Browse and select an endpoint
  - UC-04 — Generate and review a context-aware recommendation
  - UC-05 — Run on-demand network diagnostics
  - UC-06 — Enable or disable Lite Mode
  - UC-07 — View and clear local connection history
  - UC-08 — Recover visible state after network/tunnel interruption
  - UC-09 — Continue safely when an optional remote service is unavailable
  - UC-10 — Build, install and validate a product release
  - UC-11 — Onboard and configure permissions
  - UC-12 — Register, authenticate and manage a session
  - UC-13 — Understand the current network profile
  - UC-14 — Configure activity and connection preferences
  - UC-15 — Receive and manage predictive preference suggestions
  - UC-16 — Use AI chat for product and network assistance
  - UC-17 — Search, filter and favorite endpoints
  - UC-18 — Configure language, accessibility and notifications
  - UC-19 — Manage consent, export and deletion requests
  - UC-20 — Measure Network Boost recommendations
  - UC-21 — Maintain the authorized endpoint catalogue
  - UC-22 — Monitor service health, capacity and cost
  - UC-23 — Evaluate, promote and roll back an AI model
  - UC-24 — Request support using redacted diagnostics
- 6. Quality attributes and service objectives
- 7. System architecture and external interfaces
- 8. Data architecture, ERD and AI contracts
- 9. Data governance, privacy and security
- 10. Verification and acceptance evidence
- 11. Production delivery, service operations and release management
- 12. Infrastructure and cost baseline
- 13. Risks, dependencies and decision register
- 14. References
- Appendix A. Change record
- Appendix B. Review finding record
- Appendix C. Status definitions

Document Control

Document ID: DG-SRS
Title: DēGlitch — Context-Aware Adaptive Connectivity
Version: 1.4
Owner: Elijah Fullah
Date: 29 October 2026
Lifecycle: Maintained product specification
Supersedes: Version 1.3; retains and expands the v1.2 requirements, models and planning baseline.

Revision history

Version	Date	Change
0.1	16 Oct 2026	Initial product requirements.
0.2	17 Oct 2026	Reorganized requirements and document structure.
1.0	21 Oct 2026	Established the initial requirements baseline and UI inventory.
1.1	26 Oct 2026	Added system models, first use case, architecture and API rationale, logical data model, data controls and project planning.
1.2	26 Oct 2026	Expanded product features, Firebase feasibility and data model, plus infrastructure and cost scenarios.
1.3	28 Oct 2026	Expanded core use cases, flow diagrams, traceability and verification scenarios.
1.4	29 Oct 2026	Consolidated architecture, core and personalization ERDs, 62 requirements, 24 use cases, AI/chat/predictive preference contracts, operations and infrastructure cost model into one product specification.

Baseline and change control

Version 1.4 is the maintained requirements baseline. Requirement identifiers are stable across versions. Any future change shall record its rationale, affected requirements, use cases, architecture/data interfaces, tests, privacy/security impact, cost and migration implications. IDs shall not be reused or silently renumbered. Requirements added in this version extend the original 31 identifiers to 62; the original identifiers are retained unchanged.

Product assumptions and lifecycle

DēGlitch is specified as a production-intended software product. Production readiness is a target that must be demonstrated through implementation, security and privacy review, operational readiness, platform-specific verification and release evidence. A target architecture or requirement does not by itself establish that a service is deployed or a capability is verified. The product shall keep these states distinct in release records: **planned**, **implemented**, **tested**, and **verified for release**.

The product design assumes that users may have constrained mobile networks, fluctuating connectivity, different device capabilities and different language preferences. The first supported platform, authorized tunnel provider, endpoint regions, production identity provider, remote AI provider and hosting topology must be explicitly selected and validated before they are treated as deployed configuration.

1. Introduction

1.1 Purpose

This SRS defines DēGlitch's scope, actors, functional and non-functional requirements, external interfaces, constraints, use cases, acceptance criteria and verification approach. It is the shared baseline for implementation, peer review and change

control. A screen mock-up is not evidence that behavior meets a requirement; verification requires recorded results tied to an ID and build.

1.2 Scope

DēGlitch is a context-aware adaptive connectivity application intended to help users understand connection state, choose an authorized endpoint/profile, obtain explainable connection suggestions, run supported diagnostics and manage selected local preferences. It includes a Flutter UI and platform-specific integration with an authorized tunnel implementation where feasible.

In scope for the product

Truthful connection-state display and user-requested connect/disconnect behavior where the platform/tunnel adapter supports it.

Local profile reference/validation and authorized endpoint selection.

Reading only permitted connectivity indicators; explainable recommendations with insufficient-data handling.

On-demand diagnostics, user-controlled Lite Mode, optional local session history and privacy-aware evidence.

Handling adapter failure, network/tunnel loss and service outage; reproducible production builds, deployment and release verification.

Conditional capabilities and product constraints

A production service footprint including an authorized endpoint fleet, secure control plane, identity/session management, support and operations is a target product capability; its regional rollout and service objectives require capacity, security, provider and operational evidence.

AI chat and predictive preferences are defined product capabilities, subject to the consent, grounding, privacy, model-governance and failure controls in this specification.

Features such as automatic failover, split tunnelling, always-on VPN/killswitch and billing/subscriptions require platform-specific design and acceptance criteria before the relevant release is enabled.

Claims such as “10% faster”, lower battery use or lower data use require controlled measurements that substantiate the exact claim and conditions.

1.3 Intended readers

Product owner, engineering, security/privacy, operations, support, quality assurance and other authorized stakeholders. Readers should be able to identify each requirement, understand its user value, see the expected behavior in normal and failure scenarios, and determine how compliance will be tested.

1.4 Definitions and abbreviations

Term	Definition
Adapter	Platform-specific wrapper around supported operating-system network/VPN/tunnel APIs.
Endpoint	Authorized VPN/tunnel destination represented by configured metadata.
Lite Mode	User-controlled mode that reduces only documented non-essential background activities.
SRS	Software Requirements Specification.
TBD	To be determined and resolved before the affected requirement is formally verified.
Fixture/sample data	Test data that must be labelled and must never be presented as live telemetry.
Verified	Acceptance criteria met with recorded, reviewable evidence.
Implemented	Code or configuration exists; this does not by itself demonstrate that acceptance criteria have passed.
Verified for release	Defined acceptance criteria passed for a named build/platform/configuration, with reviewable evidence.

2. Product description

2.1 Product perspective

The recommended architecture uses a local client core and a separately secured control plane. Flutter presents the UI; application services orchestrate features; domain services handle connection state, network context, recommendations, preference prediction and assistant behavior; platform adapters use the OS network/tunnel interfaces. Local profiles and default connection history stay on-device. An authenticated, versioned HTTPS control plane provides endpoint metadata, identity/consent, notifications, support and operations where enabled. VPN user payload traffic travels through the authorized tunnel endpoint and never through the application metadata API.

2.2 User classes

Actor	Responsibilities/needs
End user	Request connect/disconnect, choose a profile/endpoint, review recommendations, run diagnostics, control Lite Mode and optionally view/delete local history.
Developer/tester	Configure an authorized test path, run repeatable scenarios, capture evidence and declare platform limitations.
Operating system / tunnel adapter	Provide permission flows and the authoritative tunnel state/measurement fields exposed on the platform.
Authorized endpoint/configuration source	Supply endpoint/profile configuration allowed for the test; not assumed to be a DēGlitch-operated server fleet.
Optional remote service	Only if selected: return defined endpoint metadata or other approved service responses; optional and not assumed deployed.
Product owner	Approve product scope, priorities, data use, risk appetite and release criteria.
Operations administrator	Maintain endpoint metadata, health, access controls, service alerts and budget limits using least privilege.
Support agent	Diagnose user-reported issues using only user-authorized, redacted information.
AI/model operator	Evaluate, promote and roll back versioned models under documented approval controls.

2.3 Operating environment and platform support

Flutter is used for the application UI. Android, iOS, macOS, Windows and Linux are intended targets, but a UI launch on a platform does not establish VPN/tunnel support. The release/test record must identify exact device/OS/build combinations and the VPN capabilities actually verified for each. Production claims shall be limited to named device/OS/build combinations that have passed acceptance tests. A phased rollout may be used, but every phase must state supported features and limitations.

2.4 Design principles that constrain requirements

Truthful state: report adapter-confirmed state; absent metrics remain unavailable, not guessed.

User control: recommendations are suggestions. No action switches endpoint or changes OS network configuration without a separately approved and enabled requirement.

Least data: collect only fields required by a documented feature or test. Do not collect browsing content or packet payloads for diagnostics.

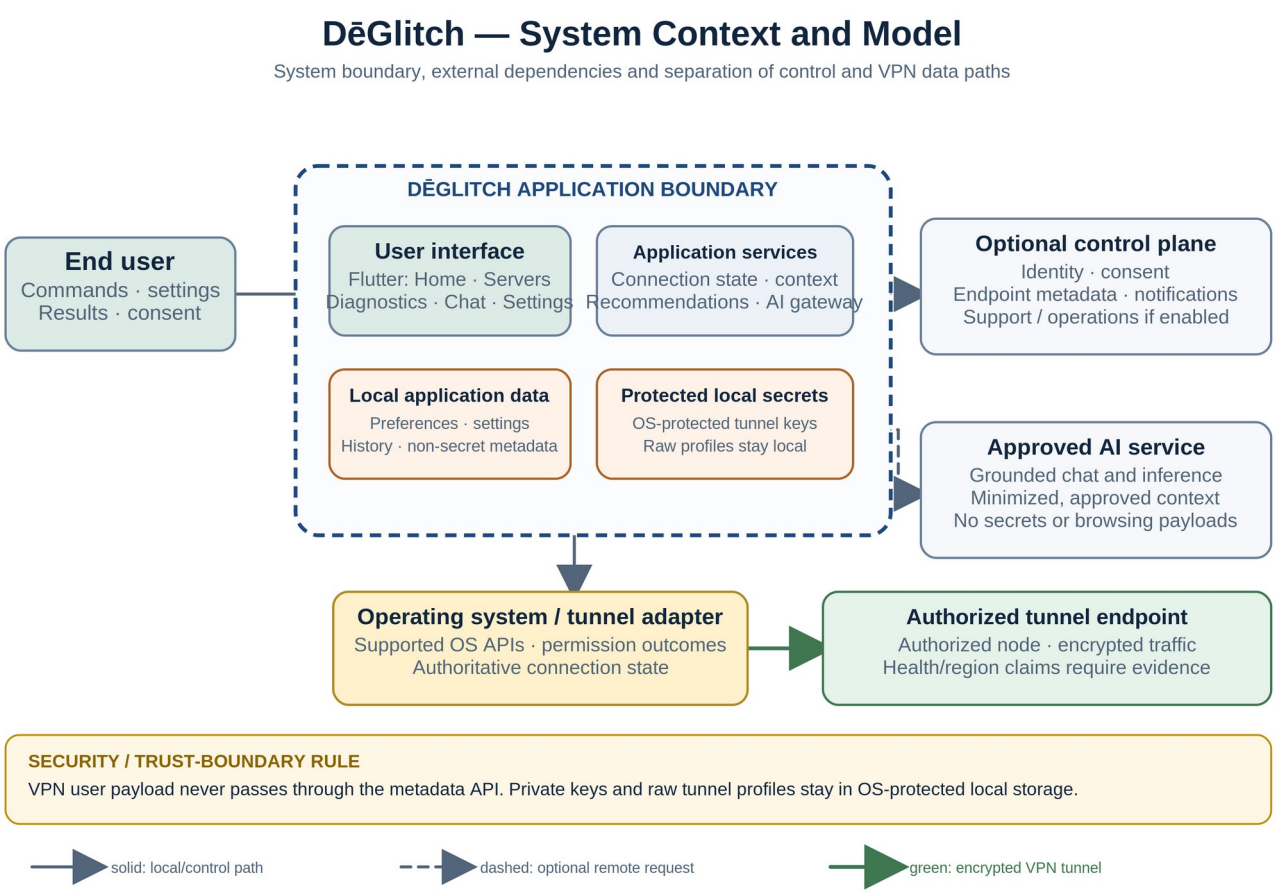
Local secret boundary: private keys, raw tunnel configuration, access tokens and credentials must not be sent to APIs/Firebase/chatbot services or included in repository/logs/screenshots.

Scope honesty: label live measurements, cached catalogue data, sample fixtures and unavailable data distinctly.

3. System context and model

3.1 System boundary

Actors and boundaries shown below describe the requirements baseline, not proof of a deployed backend or live server fleet.



System context showing the DēGlitch application boundary, user, optional control-plane and AI services, OS tunnel adapter, authorized endpoint, and trust-boundary rules.

3.2 Requirements model and traceability rule

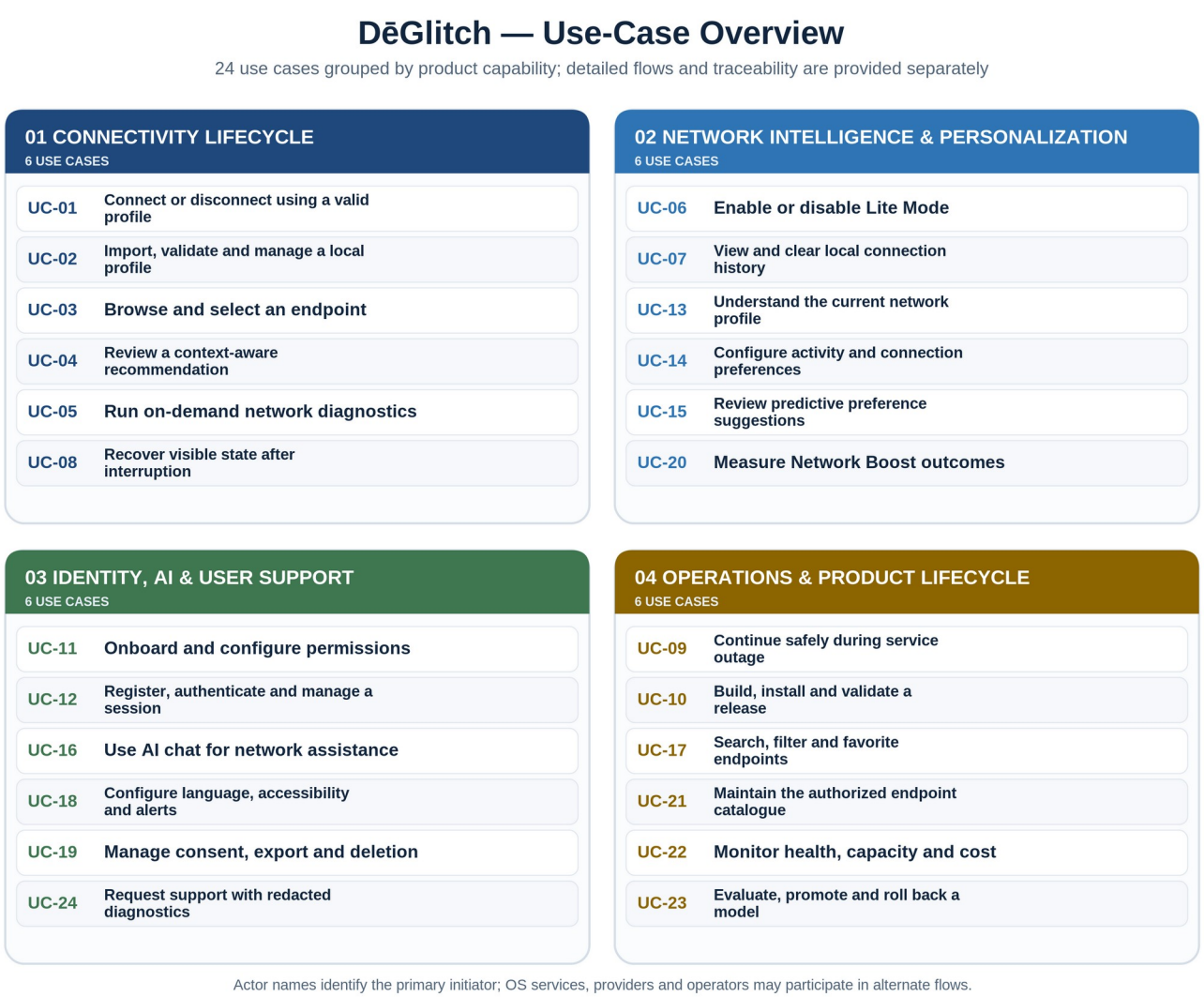
Every requirement must have a stable ID, shall-statement, rationale, source/need, priority, dependencies, component, acceptance criterion, verification method, current implementation state and evidence reference. Traceability operates in both directions: every requirement maps to one or more tests/evidence records; every test maps to one or more requirements or is explicitly labelled exploratory.

3.3 Use-case overview

DēGlitch defines 24 use cases. Each use case has a primary actor, preconditions, trigger, main success flow, alternate/failure flows, postconditions, requirement traceability and a diagram. Use cases express intended product behavior; release status is verified separately.

Use case	Title	Primary actor	Main requirement areas
UC-01	Connect or disconnect using a valid local profile	End user	Connection state, tunnel adapter, secret protection
UC-02	Import, validate and manage a local profile	End user	Local persistence, profile validation, secure storage
UC-03	Browse and select an endpoint	End user	Catalogue, eligibility, source/freshness
UC-04	Generate and review a context-aware recommendation	End user	Context, explainable recommendation, human control
UC-05	Run on-demand network diagnostics	End user	Measurements, source/freshness, privacy-aware logs
UC-06	Enable or disable Lite Mode	End user	Background activity control and resource measurement
UC-07	View and clear local connection history	End user	Local history, retention, deletion
UC-08	Recover visible state after network/tunnel interruption	OS / end user	State synchronization and recovery
UC-09	Continue safely when an optional service is unavailable	End user	Graceful degradation and local functionality

Use case	Title	Primary actor	Main requirement areas
UC-10	Install, update and validate a product build	Release engineer / tester	Build, installation, portability and dependency compliance
UC-11	Onboard and configure permissions	New user	Onboarding, locale, permission explanation
UC-12	Register, authenticate and manage a session	Account holder	Authentication, recovery, session and roles
UC-13	Understand the current network profile	End user	Network profile, source/freshness and explanation
UC-14	Configure activity and connection preferences	End user	Declared preferences, user control
UC-15	Receive and manage predictive preference suggestions	Consenting user	Activity signals, inference, consent and opt-out
UC-16	Use AI chat for product and network assistance	End user	Grounded chat, safe actions, privacy
UC-17	Search, filter and favorite endpoints	End user	Endpoint discovery and persisted favorites
UC-18	Configure language, accessibility and notifications	End user	Localization, accessibility and notifications
UC-19	Manage consent, export and deletion requests	Account holder	Consent, data rights, lifecycle controls
UC-20	Measure Network Boost recommendations	End user	Baseline/post-measurement, approval, rollback
UC-21	Maintain the authorized endpoint catalogue	Authorized administrator	Role-based administration and audit
UC-22	Monitor service health, capacity and cost	Operations administrator	Observability, alerts, capacity and spend control
UC-23	Evaluate, promote and roll back an AI model	AI/model operator	Evaluation gates, versioning and rollback
UC-24	Request support using redacted diagnostics	End user / support agent	Consent, data minimization and support workflow



Overview of all 24 DēGlitch use cases grouped into connectivity lifecycle, network intelligence and personalization, identity AI and support, and operations and product lifecycle.

3.4 Actors and relationships

End user: controls connections, profiles, endpoints, diagnostics, settings, recommendations and assistant interactions.

Operating system/tunnel adapter: exposes authoritative permission and connection lifecycle signals. DēGlitch shall not infer an active tunnel from UI state alone.

Identity provider: authenticates accounts; the application backend remains responsible for authorization decisions and role enforcement.

Endpoint catalogue/control plane: supplies approved endpoint metadata, catalogue freshness and authorized operational state; it does not carry user tunnel payload traffic.

AI provider/model runtime: receives only the minimum approved context needed for the specific request. It must not receive profile secrets, raw tunnel configurations or browsing payloads.

Operations administrator and support agent: use privileged functionality under least-privilege authorization, with audit events that omit secrets and sensitive payloads.

Notification service: delivers only user-enabled categories; critical connection state remains visible in the app even when push notifications are unavailable.

3.5 Requirement traceability model

Each requirement maps to a user need, one or more use cases or operational scenarios, an owning component, an acceptance criterion and verification evidence. The same identifier is used in the SRS, issues, design views, test cases, release checklist and defect records. The supplemental Requirement_Traceability_Matrix.md provides the two-way mapping.

4. Requirements baseline

The 31 original requirement IDs are retained verbatim and extended by 31 additional requirements in this version. Applicability, dependencies, acceptance criteria and release conditions are explicit. Numerical thresholds marked TBD shall be resolved before formal verification.

4.1 External interfaces

ID / title	Shall statement	Acceptance criterion	Verification
REQ-INT-001 — Read permitted OS network indicators	The system shall obtain only OS-exposed network-state indicators required by approved features.	Permission denied or field unavailable is represented as unavailable; no value is fabricated.	Test
REQ-INT-002 — Use supported VPN/tunnel interface	The system shall interact with a selected authorized tunnel implementation through supported platform interfaces.	Connection requests and resulting states are recorded by the adapter contract on each claimed platform.	Integration test
REQ-INT-003 — Persist local profile/history safely	When profiles or history are enabled, the system shall use a defined local persistence mechanism with documented fields and protection.	A profile/history survives expected app restart behavior; secrets are not stored in unprotected files or logs.	Test + inspection
REQ-INT-004 — Define optional remote interfaces	Any endpoint directory, diagnostics service, or ML service shall have a documented interface before integration.	No undocumented remote dependency is required for local core workflows; selected interface has schema and error behavior.	Inspection + integration test

4.2 Functional requirements

ID / title	Shall statement	Acceptance criterion	Verification
REQ-FUNC-001 — Display confirmed connection state	The system shall display the connection state reported by the underlying implementation.	Each supplied state is mapped correctly; unavailable remains unknown, and Connected is never guessed.	Test + demonstration
REQ-FUNC-002 — Start and stop connection	The system shall provide user controls to request connect/disconnect where supported by the platform and configured tunnel.	Valid actions reach the adapter; UI shows state only when confirmed by adapter feedback.	Integration test + demonstration
REQ-FUNC-003 — Report connection failures	The system shall display an understandable error when a connection request or tunnel fails.	Permission denial, missing endpoint and tunnel failure produce safe, actionable feedback without false Connected state.	Test
REQ-FUNC-004 — Select configured endpoint	The system shall allow users to select from endpoints supplied by the configured endpoint source.	Only eligible supplied entries can be selected; empty/unavailable source has an explicit empty state.	Test
REQ-FUNC-005 — Read available network context	The system shall read supported network indicators needed for status or recommendations, subject to OS permissions.	Denied permission and unsupported values are marked unavailable; source and time are retained where useful.	Test
REQ-FUNC-006 — Produce a recommendation when evidence is sufficient	The system shall produce a connection recommendation only when configured inputs are available; otherwise, it shall report insufficient information.	Reviewed inputs produce a reproducible recommendation or an explicit insufficient-data/no-candidate result and supported reasons.	Scenario test
REQ-FUNC-007 — Keep recommendations under user control	The system shall present recommendations as suggestions unless a separately specified automatic action is enabled.	With automatic action disabled, recommendation display alone never initiates a connection or settings change.	Test
REQ-FUNC-008 — Provide user-controlled Lite Mode	The system shall provide Lite Mode that reduces only non-essential background activities explicitly listed in the implementation specification.	Toggle state is visible and documented activities change as specified; no battery/data saving claim without measurement.	Inspection + controlled measurement

ID / title	Shall statement	Acceptance criterion	Verification
REQ-FUNC-009 — Run diagnostics on demand	The system shall allow a user to start on-demand diagnostics using measurements supported by the implementation.	Each result includes units where applicable and source/time; unsupported metrics are explicitly unavailable.	Test + demonstration
REQ-FUNC-010 — Manage local history and profiles	When enabled, the system shall let users view/delete locally stored history and manage saved profile settings.	View, delete, edit and profile-removal paths pass tests; persistence and retention rules are documented.	Test

4.3 Quality, security, reliability, availability and observability requirements

ID / title	Shall statement	Acceptance criterion	Verification
REQ-PERF-001 — Measure recommendation response time	Under a defined device, workload and network profile, the project shall measure recommendation response time against an approved threshold.	Report platform, device, inputs, sample count, percentile/maximum and measured latency; threshold is approved before pass/fail.	Measurement
REQ-PERF-002 — Compare resource use with Lite Mode	The project shall compare agreed background resource use with Lite Mode enabled and disabled under matched conditions.	Repeatable report compares CPU, memory, network traffic and battery where measurable; unsupported values are marked unavailable.	Controlled measurement
REQ-SEC-001 — Protect secrets	The system shall not embed credentials, private keys, tokens or other secrets in source, public documentation, logs or screenshots.	Repository and logging inspection finds no committed secrets; any suspected exposure is rotated and documented.	Inspection
REQ-SEC-002 — Use authorized tunnel configuration	The system shall use only endpoints and tunnel configurations supplied by an authorized source or explicitly configured for testing.	Configuration provenance is recorded; malformed or unauthorized configurations are rejected or flagged before use.	Inspection + test
REQ-SEC-003 — Minimize diagnostic data	The system shall not collect or store browsing content for network diagnostics unless separately reviewed and approved.	Diagnostic payloads and stored records contain no browsing content, packet payloads or raw profile secrets.	Inspection + test
REQ-REL-001 — Handle network/tunnel loss	When the platform reports network loss or tunnel failure, the system shall update visible state and shall not continue to report Connected.	Injected loss/failure updates state within an approved timing threshold; threshold recorded before formal pass/fail.	Integration test
REQ-AVAIL-001 — Handle optional service outage	If an optional endpoint directory or remote service is unavailable, the system shall explain the limitation and preserve unrelated local functions where feasible.	With optional service disabled, user sees a non-sensitive error and usable local settings/diagnostics remain available.	Test
REQ-OBS-001 — Provide privacy-aware diagnostic evidence	The system shall provide enough diagnostic information to explain connection state and recommendation results without logging secrets or unnecessary personal data.	A documented field list and log inspection show secrets absent/redacted and relevant scenario/build metadata available.	Inspection + test

4.4 Compliance, installation, build, maintenance, portability, release validation and change requirements

ID / title	Shall statement	Acceptance criterion	Verification
REQ-COMP-001 — Record dependency licenses	Before a third-party library or protocol implementation is included in a release build, the project shall record its name, version, source and license.	Dependency inventory covers direct/transitive release dependencies as required by selected tooling.	Inspection
REQ-COMP-002 — Follow platform permission rules	The application shall request and use network/VPN permissions according to each selected platform's documented mechanisms.	Permission flows are documented and tested per platform claimed as supported.	Inspection + test

ID / title	Shall statement	Acceptance criterion	Verification
REQ-INST-001 — Document test installation	The project shall document supported platform(s), prerequisites, setup and installation steps before formal testing.	A reviewer can follow the steps in a clean supported environment using a test device/emulator.	Demonstration + inspection
REQ-BUILD-001 — Document reproducible builds	The project shall document build procedures and required non-secret configuration for each submitted test build.	Instructions specify toolchain and setup without committing secrets; an independent repeat build is recorded.	Inspection + repeat build
REQ-MAINT-001 — Maintain versioned documentation and evidence	The project shall maintain requirements, implementation documentation and verification evidence under version control.	Stable IDs, status/evidence entries and revision history are present; changes are traceable.	Inspection
REQ-PORT-001 — Declare verified platforms accurately	The project shall identify platforms supported by each submitted build and shall not claim cross-platform VPN support without platform-specific verification.	Release/test report names verified device/OS combinations and clearly separates untested targets.	Inspection + test
REQ-POC-001 — Validate end-to-end connection lifecycle	The product shall validate the authorized connection lifecycle and failure handling on every platform claimed for release.	Repeatable tests verify connect, adapter-confirmed state, disconnect, permission denial and tunnel/endpoint failure for each platform in the release matrix.	Demonstration + test
REQ-CM-001 — Control baseline changes	Changes to the SRS baseline shall record affected requirement IDs, reason, impacts on dependent requirements/tests and review status.	Change appears in revision history, traceability and verification tables; IDs are not silently reused.	Inspection

4.5 Conditional AI/ML requirement

ID / title	Shall statement	Acceptance criterion	Verification
REQ-ML-001 — Specify ML before including it	If ML is included in an implementation increment, the project shall document task, features, outputs, baseline comparator, dataset, metrics and acceptance threshold before performance claims.	Reviewed model specification exists before evaluation; dataset governance and threshold are explicit.	Inspection + analysis

4.6 Identity, onboarding and account lifecycle requirements

ID / title	Shall statement	Acceptance criterion	Verification
REQ-AUTH-001 — Register and authenticate users	The system shall support the selected production identity provider for account registration and sign-in where account-backed features are enabled.	Valid identity-provider responses establish a session; invalid or expired responses are handled without disclosing whether a particular account exists.	Integration test
REQ-AUTH-002 — Manage session and recovery	The system shall support sign-out, session expiry, reauthentication and the configured account-recovery process.	Expired/revoked sessions cannot access protected data; recovery follows identity-provider policy and produces a safe outcome.	Security and integration test
REQ-AUTH-003 — Enforce server-side authorization	The control plane shall enforce authorization on every protected administrative or account-data operation; client-side route hiding shall not be the sole control.	Role/ownership tests deny unauthorized reads and writes, including direct API calls.	Security test
REQ-ONB-001 — Explain onboarding and permissions	The system shall explain core capabilities and the purpose of each sensitive permission before requesting it when practical.	Permission prompts are purpose-linked; decline paths remain safe and affected features show a clear limitation.	Inspection and usability test

4.7 Network profiles and declared preferences

ID / title	Shall statement	Acceptance criterion	Verification
REQ-NPROF-001 — Build a network profile from permitted inputs	The system shall assemble a current network profile only from OS-exposed, user-provided or authorized telemetry fields defined in the data contract.	Profile contains source, observation time and availability per field; unauthorized or unsupported values are omitted or unknown.	Test and inspection
REQ-NPROF-002 — Explain profile factors	The system shall explain the meaning and limitation of network-profile factors in understandable language.	Each displayed factor distinguishes measured, cached, inferred and unavailable data; explanatory text does not claim unsupported cause or certainty.	Scenario and usability test
REQ-PREF-001 — Store declared preferences	The system shall let users select and revise explicitly declared activity and connection priorities.	Changes persist according to the account/local-storage policy and take effect in subsequent recommendations; reset restores documented defaults.	Test
REQ-PREF-002 — Infer activity-based preferences with consent	The system shall generate predictive preference suggestions only from approved, minimized activity signals after explicit opt-in.	Without valid consent or adequate data no inference is produced; each candidate is tied to a method version, reason codes and expiry.	Privacy and scenario test
REQ-PREF-003 — Review and control inferred preferences	The system shall let users inspect, accept, edit, reject and reset inferred preference suggestions and withdraw related consent.	Rejected suggestions do not continue to influence recommendations; withdrawal blocks future collection/inference for the purpose and triggers the documented deletion policy.	Test and inspection

4.8 AI assistant and personalization requirements

ID / title	Shall statement	Acceptance criterion	Verification
REQ-AI-001 — Provide in-app AI chat	Where the AI assistant is enabled, the system shall answer product-help and network-context questions using approved help content and only the context allowed for the request.	Supported questions produce a response or a clear safe fallback; a disconnected/unconfigured AI backend is not shown as operational.	Integration and scenario test
REQ-AI-002 — Ground assistant responses	The assistant shall distinguish source-backed facts, current measurements, cached data, inference and unavailable information, and shall not claim live access unless that data was supplied.	Test prompts involving absent metrics, stale endpoints and unsupported claims produce an uncertainty statement or ask for data rather than fabricating.	Safety evaluation
REQ-AI-003 — Require confirmation for consequential actions	The assistant shall not initiate connection, endpoint, profile, permission, billing or network-setting changes without a separate explicit user confirmation.	Chat outputs alone have no side effect; a confirmed action is routed through the same validated application service as the native UI.	Security and integration test
REQ-AI-004 — Localize assistant interactions	The assistant shall follow the selected supported locale and provide a safe fallback when high-quality localization is unavailable.	Locale-specific test set verifies language selection, fallback labeling and no silent language mismatch for critical errors.	Localization test
REQ-AI-005 — Protect conversation data	The system shall apply documented access, retention, deletion and export rules to chat history and shall not send secrets or unnecessary personal data to a model provider.	Payload inspection and lifecycle tests show minimization, access isolation, configured retention and deletion behavior.	Privacy/security test

4.9 Endpoint discovery and network optimization requirements

ID / title	Shall statement	Acceptance criterion	Verification
REQ-ENDPT-001 — Search, filter and favorite endpoints	The system shall allow users to search/filter eligible endpoint entries and manage favorites where endpoint discovery is enabled.	Search/filter returns only matching eligible catalogue entries; favorites persist and can be removed; unavailable catalogue has an explicit state.	Functional test

ID / title	Shall statement	Acceptance criterion	Verification
REQ-ENDPT-002 — Show endpoint provenance and freshness	The system shall identify the source, observation time or cache age, and availability state of endpoint-health data.	Test fixtures, stale cache, live probes and unknown states have distinct labels; stale values are not presented as live.	Test and inspection
REQ-NETOPT-001 — Measure Network Boost outcomes	The system shall compare a supported network-optimization action using documented baseline and post-action measurements under comparable conditions when the measurements are available.	Report lists action, approval, device/network conditions, metrics, sample count and limitations; no fixed percentage improvement is promised without evidence.	Controlled measurement

4.10 Notifications, localization, accessibility and privacy requirements

ID / title	Shall statement	Acceptance criterion	Verification
REQ-NOTIF-001 — Respect notification preferences	The system shall let users enable/disable supported notification categories and shall respect operating-system notification permission.	Disabled categories do not send notifications; denied OS permission results in a non-blocking explanation and in-app state remains available.	Test
REQ-NOTIF-002 — Handle duplicate and delayed notifications	When push notifications are enabled, the system shall deduplicate repeated events and prevent delayed or expired connection events from being presented as current state.	Repeated event IDs produce at most one notification within the configured idempotency window; stale events are suppressed or labelled with event time, and the in-app adapter state remains authoritative.	Resilience and integration test
REQ-I18N-001 — Localize supported interfaces	The system shall support the selected release locales from English, Chinese, Spanish, French, Russian and Krio according to the release locale matrix.	Each claimed locale has reviewed critical flows, text overflow checks and a defined fallback for untranslated content.	Localization test
REQ-UX-001 — Support accessibility and reduced motion	The system shall support scalable text, accessible labels and contrast, keyboard/screen-reader use where applicable, and a reduced-motion preference.	Accessibility review finds no blocker in critical flows; animations respect reduced-motion settings on supported platforms.	Inspection and accessibility test
REQ-PRIV-001 — Fulfil data export and deletion requests	The system shall provide an authenticated path to export or delete account-linked data where applicable and a local path to clear local data.	A request is tracked to completion or a clear failure/pending state; deletion covers documented stores, caches and retention exceptions.	Integration and inspection
REQ-PRIV-002 — Record granular consent and withdrawal	The system shall record user consent by purpose and policy version for optional activity inference, remote AI context, chat storage and diagnostic sharing.	Disabled or withdrawn purpose prevents subsequent processing for that purpose, subject only to documented legal/security retention exceptions.	Privacy test and inspection

4.11 Control-plane, operations and model-governance requirements

ID / title	Shall statement	Acceptance criterion	Verification
REQ-API-001 — Provide a secure versioned API	Any production control-plane API shall use versioned HTTPS interfaces with authenticated service/user context, defined schemas, error codes and compatibility policy.	Contract tests cover valid/invalid requests, authentication, version negotiation and safe error payloads.	API contract and integration test
REQ-API-002 — Validate and protect API requests	The control plane shall validate input, enforce object-level authorization, rate-limit abuse-prone operations and apply centralized secret management.	Tests cover malformed/oversized payloads, unauthorized object IDs, throttling and secret absence from logs.	Security test
REQ-OPS-001 — Maintain authorized endpoint metadata	Authorized administrators shall be able to create, update, disable and inspect endpoint catalogue records with provenance and configuration version references.	Every mutation is validated, authorized, auditable and reflected in the catalogue only after the configured approval/publish step.	Integration test

ID / title	Shall statement	Acceptance criterion	Verification
REQ-OPS-002 — Monitor operational health	The production service shall collect approved service-health and resource metrics and generate alerts for agreed service thresholds.	Alert tests exercise healthy, unhealthy and unknown states; telemetry omits tunnel secrets and browsing content.	Operational test
REQ-OPS-003 — Enforce cost and traffic safeguards	The system operations process shall monitor provider billing/transfer quotas and issue threshold alerts and approved protective actions.	Configured 50/75/90 percent or explicitly approved thresholds trigger test alerts; safeguards do not silently affect active user sessions without policy.	Operational test and inspection
REQ-ML-002 — Govern model versions and rollback	If a trained model is enabled, the system shall record model version, dataset/evaluation reference, approval, monitored metrics and rollback target.	A candidate cannot be promoted without passing the declared gate; an injected regression triggers rollback or disables the model per policy.	Analysis, integration and operational test
REQ-PERF-003 — Measure AI assistant latency	Under a documented platform, prompt class, provider/model and workload, the system shall measure assistant response latency against an approved service objective.	Report includes sample count, percentile, timeout/error rates and model/provider; objective is approved before pass/fail.	Measurement
REQ-AVAIL-002 — Bound retries and isolate failing dependencies	Remote integrations shall use bounded retries, timeouts and circuit breaking or an equivalent isolation mechanism appropriate to the dependency.	Fault-injection shows no retry storm, prompt user feedback, bounded resource use and recovery after the dependency returns.	Resilience test
REQ-DATA-001 — Apply data lifecycle rules across stores	Every retained data category shall have an owner, purpose, location, access policy, retention period and deletion/expiry rule.	A data inventory covers local storage, cloud stores, logs, AI provider requests, backups and exports; deletion exceptions are explicit.	Inspection and privacy test

4.12 Allocation, priority, applicability and status register

ID	Responsible component	Use case / verification coverage	Priority and applicability	Evidence reference
REQ-INT-001	Network/context adapter	UC-04, UC-05, UC-08	Should; Baseline / TBD	Release evidence register.
REQ-INT-002	Platform tunnel adapter	UC-01, UC-08, UC-10	Must; Baseline / TBD	Release evidence register.
REQ-INT-003	Local persistence / secure storage	UC-01, UC-02, UC-06, UC-07	Should; Baseline; evidence recorded per release	Release evidence register.
REQ-INT-004	Optional service adapter	UC-03, UC-09	Should; Conditional	Release evidence register.
REQ-FUNC-001	Connection coordinator / UI	UC-01, UC-08	Must; Baseline; evidence recorded per release	Release evidence register.
REQ-FUNC-002	UI / connection coordinator	UC-01	Must; Baseline	Release evidence register.
REQ-FUNC-003	Connection coordinator / UI	UC-01, UC-08	Must; Baseline; evidence recorded per release	Release evidence register.
REQ-FUNC-004	Endpoint catalogue / server UI	UC-03	Must; Baseline; evidence recorded per release	Release evidence register.
REQ-FUNC-005	Context collector	UC-04	Should; Baseline	Release evidence register.
REQ-FUNC-006	Recommendation service	UC-04	Must; Baseline; rule-based first	Release evidence register.
REQ-FUNC-007	Recommendation UI / coordinator	UC-04	Must; Baseline	Release evidence register.
REQ-FUNC-008	Settings / scheduler	UC-06	Should; Baseline; exact activity list TBD	Release evidence register.
REQ-FUNC-009	Diagnostics service / UI	UC-05	Should; Baseline	Release evidence register.
REQ-FUNC-010	Local storage / profile manager	UC-02, UC-07	Should; Conditional on feature release	Release evidence register.

ID	Responsible component	Use case / verification coverage	Priority and applicability	Evidence reference
REQ-PERF-001	Recommendation service	UC-04	Should; Baseline / threshold TBD	Release evidence register.
REQ-PERF-002	Scheduler / diagnostics	UC-06	Should; Baseline / target TBD	Release evidence register.
REQ-SEC-001	All components / repository	UC-01, UC-02, UC-07, UC-09, UC-10	Must; Baseline	Release evidence register.
REQ-SEC-002	Profile manager / tunnel adapter	UC-01, UC-02, UC-03, UC-10	Must; Baseline	Release evidence register.
REQ-SEC-003	Diagnostics / logging	UC-04, UC-05, UC-07	Must; Baseline	Release evidence register.
REQ-REL-001	Connection coordinator / adapter	UC-01, UC-08	Must; Baseline / threshold TBD	Release evidence register.
REQ-AVAIL-001	Optional service adapter / UI	UC-03, UC-05, UC-09	Should; Conditional service integration	Release evidence register.
REQ-OBS-001	Logging / diagnostics	UC-02, UC-05, UC-06, UC-07, UC-08, UC-09	Should; Baseline	Release evidence register.
REQ-COMP-001	Build/release engineering	UC-10	Should; Baseline	Release evidence register.
REQ-COMP-002	Platform adapters	UC-10	Should; Baseline	Release evidence register.
REQ-INST-001	Build/release documentation	UC-10	Should; Baseline	Release evidence register.
REQ-BUILD-001	Build/release documentation	UC-10	Should; Baseline	Release evidence register.
REQ-MAINT-001	Repository / documentation	UC-10	Should; Baseline	Release evidence register.
REQ-PORT-001	Release documentation / QA	UC-10	Should; Baseline	Release evidence register.
REQ-POC-001	Platform tunnel adapter / QA	UC-10	Must; Baseline	Release evidence register.
REQ-CM-001	Configuration management	UC-10	Should; Baseline; evidence recorded per release	Release evidence register.
REQ-ML-001	Conditional recommendation/model component	UC-04	Should; Conditional; custom ML not in current baseline	Release evidence register.
REQ-AUTH-001	Identity provider / account service	UC-12, TC-33	Must; account-backed release	Release evidence register.
REQ-AUTH-002	Session manager / identity provider	UC-12, UC-19, TC-34	Must; account-backed release	Release evidence register.
REQ-AUTH-003	API authorization middleware	UC-12, UC-21, TC-35	Must; any privileged backend operation	Release evidence register.
REQ-ONB-001	Onboarding / permission UX	UC-11, UC-18, TC-36	Must; baseline onboarding	Release evidence register.
REQ-NPROF-001	Network context service	UC-13, UC-04, TC-37	Must; profile view	Release evidence register.
REQ-NPROF-002	Network profile explainer	UC-13, UC-16, TC-38	Must; baseline profile	Release evidence register.
REQ-PREF-001	Preference service / local store	UC-14, UC-04, TC-39	Must; declared preferences	Release evidence register.
REQ-PREF-002	Inference service / consent policy	UC-15, UC-19, TC-40	Conditional; only after opt-in	Release evidence register.
REQ-PREF-003	Preference controls / data lifecycle	UC-15, UC-19, TC-41	Must; when inference is offered	Release evidence register.
REQ-AI-001	Assistant orchestration / provider adapter	UC-16, UC-24, TC-42	Conditional on approved AI service	Release evidence register.
REQ-AI-002	Grounding/safety layer	UC-13, UC-16, TC-43	Must; any AI response	Release evidence register.

ID	Responsible component	Use case / verification coverage	Priority and applicability	Evidence reference
REQ-AI-003	Application action coordinator	UC-01, UC-03, UC-16, TC-44	Must; AI action boundary	Release evidence register.
REQ-AI-004	Localization / assistant layer	UC-16, UC-18, TC-45	When AI is enabled	Release evidence register.
REQ-AI-005	Conversation data service	UC-16, UC-19, TC-46	Must; if chat storage enabled	Release evidence register.
REQ-ENDPT-001	Endpoint directory / client store	UC-03, UC-17, TC-47	Must; endpoint discovery	Release evidence register.
REQ-ENDPT-002	Endpoint metadata API / cache	UC-03, UC-17, UC-22, TC-48	Must; health metadata	Release evidence register.
REQ-NETOPT-001	Optimization coordinator / diagnostics	UC-20, TC-49	Should; optimization is supported	Release evidence register.
REQ-NOTIF-001	Notification preferences / messaging	UC-18, TC-50	Should; notification enabled	Release evidence register.
REQ-NOTIF-002	Notification delivery / idempotency	UC-18, TC-64	Should; push delivery enabled	Release evidence register.
REQ-I18N-001	Localization framework	UC-11, UC-18, TC-51	Must; release locales declared	Release evidence register.
REQ-UX-001	Presentation/accessibility layer	UC-11, UC-18, TC-52	Must; critical flows	Release evidence register.
REQ-PRIV-001	Privacy centre / data lifecycle service	UC-19, UC-07, TC-53	Must; where account data exists	Release evidence register.
REQ-PRIV-002	Consent policy and telemetry gates	UC-15, UC-16, UC-19, UC-24, TC-54	Must; optional data processing	Release evidence register.
REQ-API-001	Control-plane API gateway	UC-03, UC-12, UC-21, TC-55	Must; when remote control plane enabled	Release evidence register.
REQ-API-002	API validation / security controls	UC-12, UC-21, TC-56	Must; control plane	Release evidence register.
REQ-OPS-001	Endpoint administration service	UC-21, TC-57	Must; production endpoint catalogue	Release evidence register.
REQ-OPS-002	Monitoring / alerting	UC-22, TC-58	Must; production operation	Release evidence register.
REQ-OPS-003	Billing / traffic monitors	UC-22, TC-59	Must; production operation	Release evidence register.
REQ-ML-002	Model registry / release pipeline	UC-23, TC-60	Conditional; trained model	Release evidence register.
REQ-PERF-003	Assistant provider adapter / telemetry	UC-16, TC-61	Conditional; AI service enabled	Release evidence register.
REQ-AVAIL-002	Remote dependency adapters	UC-09, UC-16, UC-22, TC-62	Must; remote dependency enabled	Release evidence register.
REQ-DATA-001	Data governance / retention services	UC-07, UC-15, UC-16, UC-19, UC-24, TC-63	Must; all retained data	Release evidence register.

Identifier integrity: all 31 original requirement IDs are retained; 31 new IDs extend the baseline to 62. Priority is an implementation-planning signal and does not waive security, privacy, or acceptance criteria.

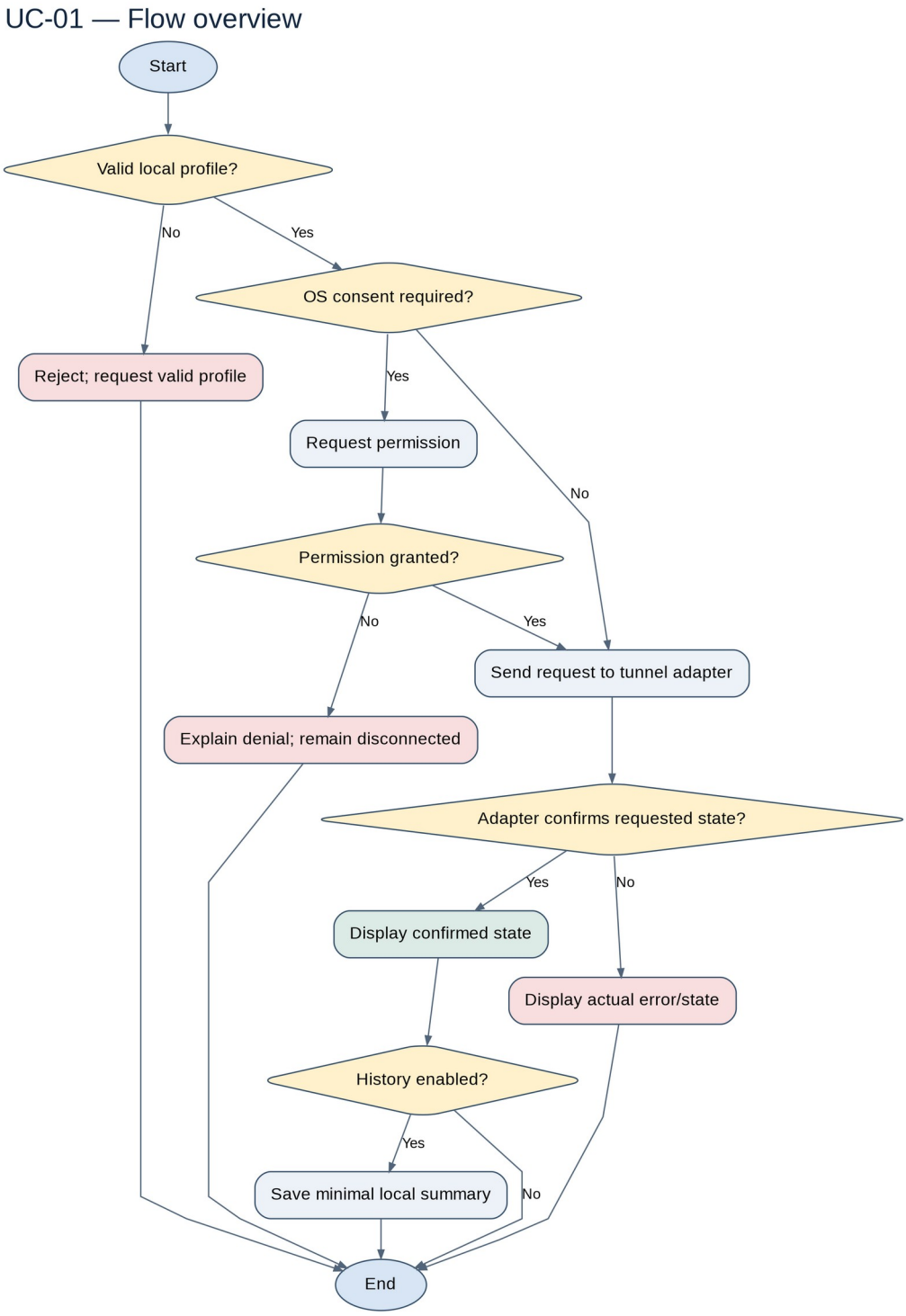
5. Use-case specifications and flow diagrams

Each specification uses the same template: goal/scope, primary actor, preconditions, trigger, related IDs, main success flow, alternate/failure flow and postcondition. The flow image is an overview, not a substitute for the numbered steps and exception rules.

UC-01 — Connect or disconnect using a valid local profile

Primary actor: End user
Goal: Request a connection or disconnection and display only the state confirmed by the platform/tunnel adapter.
Preconditions: Supported test platform and adapter are configured; an authorized, validated local profile is available for connecting.
Trigger: User taps Connect or Disconnect.

Related requirements: REQ-FUNC-001, REQ-FUNC-002, REQ-FUNC-003, REQ-INT-002, REQ-INT-003, REQ-SEC-001, REQ-SEC-002, REQ-REL-001



UC-01 flow diagram

Main success scenario

The UI reads the current state from the adapter and displays the reported value.

The user chooses Connect or Disconnect.

For Connect, the application confirms that a valid local profile reference exists.

If required, the app invokes the documented OS permission/consent flow.

The coordinator sends the requested action to the platform adapter.

The UI shows a transitional state only when reported by the adapter.

The adapter emits state changes; the coordinator maps them to the UI.

The UI shows Connected only after explicit adapter confirmation; optional counters are shown only if exposed.

If history is enabled, a minimal session summary is stored locally without secrets or browsing content.

Alternative and failure flows

A1 — Permission denied: Do not report Connected; show explanation and allow retry after the user changes permission.

A2 — Missing/invalid profile: Reject before invoking the adapter and guide the user to import/select a valid local profile.

A3 — Adapter, endpoint or tunnel failure: Display Error or the actual adapter state with safe guidance; never present a false success.

A4 — Measurement unavailable: Show the connection state if known, but label unsupported metrics unavailable rather than use fixtures as live values.

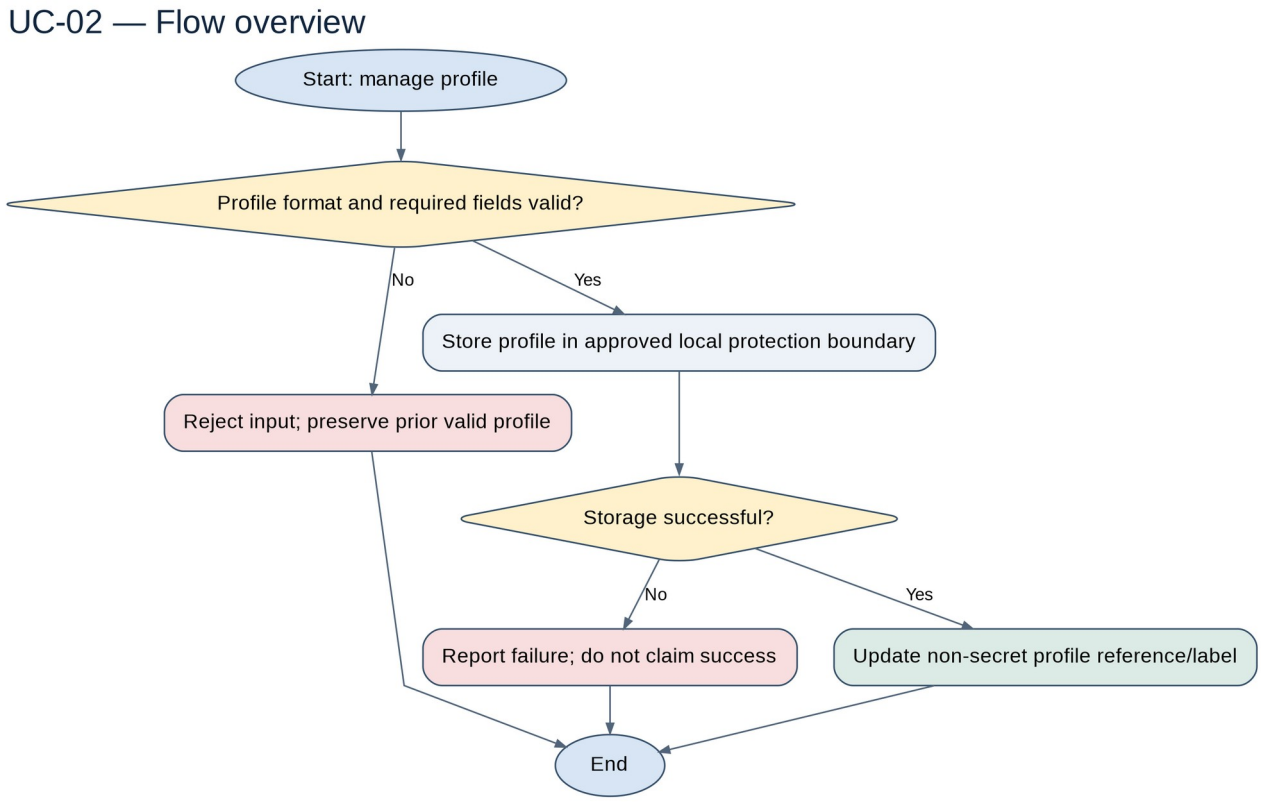
A5 — Network loss/tunnel stops: Update state when the adapter reports loss; do not leave stale Connected visible.

A6 — History disabled: Complete connection workflow without writing a session-history record.

Postconditions: Connection state matches adapter-confirmed state; failed actions do not create false success; any history is minimal and local.

UC-02 — Import, validate and manage a local profile

Primary actor: End user
Goal: Add or change a local profile safely before it is used for connection.
Preconditions: User has a profile file or an already configured local profile; a supported local storage path is available.
Trigger: User chooses Import Profile, Select Profile, Replace or Remove.
Related requirements: REQ-INT-003, REQ-FUNC-002, REQ-FUNC-010, REQ-SEC-001, REQ-SEC-002, REQ-OBS-001



UC-02 flow diagram

Main success scenario

User opens profile management and selects an operation.

For import/replace, the app validates file type, structure and required fields without exposing secret contents.

The app reports validation findings in safe, actionable language.

If valid, the profile is stored only in the approved local/protected storage boundary.

The app assigns or preserves a non-secret display label and updates the active profile reference.

The UI confirms the operation and records no profile secret in logs.

Alternative and failure flows

A1 — Unsupported/malformed file: Reject the file, preserve the previously valid profile, and explain the issue.

A2 — Storage/permission failure: Do not claim success; preserve existing data and report that the profile could not be stored.

A3 — Remove active profile: Warn if connected; require disconnection or confirmed safe handling before removing the profile reference.

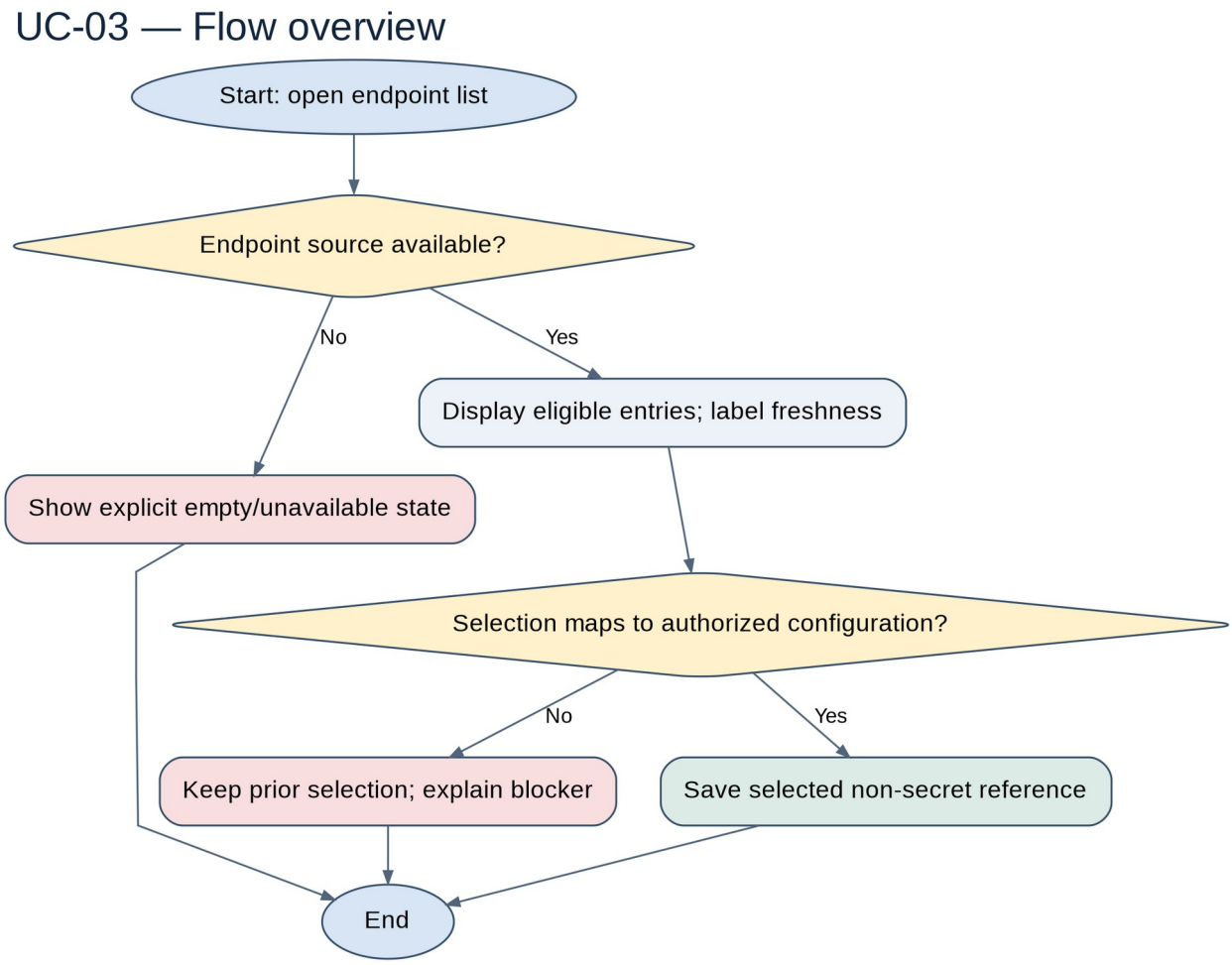
A4 — Duplicate import: Avoid duplicate/stale entries or explain the chosen replacement behavior.

A5 — Secret-bearing diagnostics: Redact content; do not export raw profile or private key.

Postconditions: Valid selected profile remains local and usable; invalid input is not activated; secrets are not exposed.

UC-03 — Browse and select an endpoint

Primary actor: End user
Goal: Review configured endpoint entries and select one for a later connection or recommendation.
Preconditions: An authorized endpoint source or local profile catalogue is configured; unavailable data can be represented honestly.
Trigger: User opens Servers/Server Recommendation.
Related requirements: REQ-FUNC-004, REQ-FUNC-007, REQ-INT-004, REQ-AVAIL-001, REQ-SEC-002



UC-03 flow diagram

Main success scenario

The app requests endpoint metadata from the configured source (or reads approved local entries).

The app distinguishes configured metadata from live health/performance telemetry.

The UI displays eligible endpoints and labels missing/stale metrics.

The user searches, filters or opens an endpoint if those controls are implemented.

The user selects an endpoint.

The app validates that the selection maps to an authorized profile/configuration.

The app saves the selected non-secret reference and confirms the selection.

Alternative and failure flows

A1 — Empty endpoint list: Show an explicit empty state and do not create a selectable placeholder endpoint.

A2 — Source unavailable: Show cached/local entries only if their source/freshness is identified; otherwise explain unavailability.

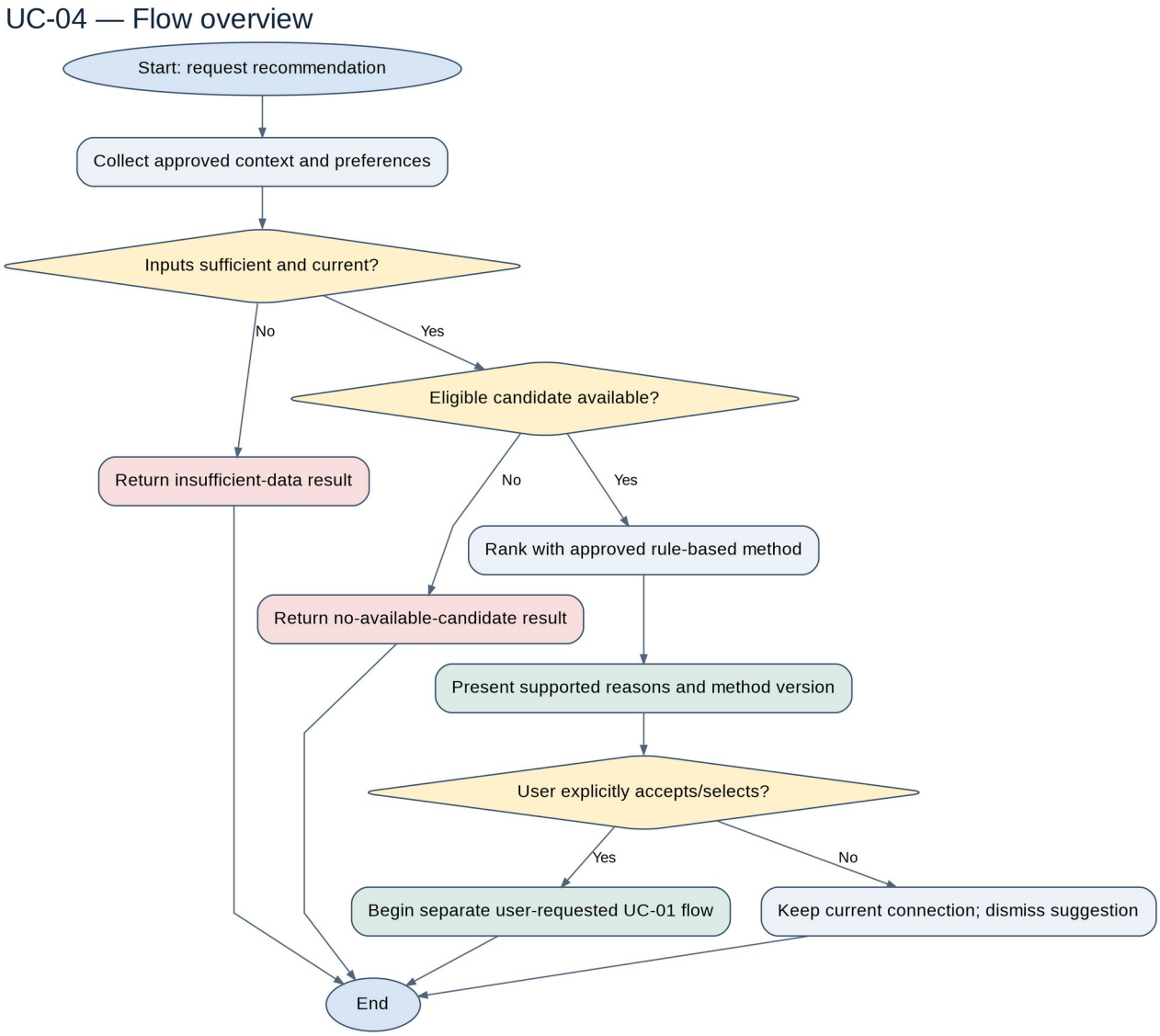
A3 — Endpoint not authorized or profile mapping missing: Block selection for connection use and explain how to configure a valid profile.

A4 — Metrics missing: Show unavailable, never fabricate health, latency or load.

Postconditions: A valid non-secret endpoint/profile reference is selected, or selection remains unchanged with a clear error.

UC-04 — Generate and review a context-aware recommendation

Primary actor: End user
Goal: Obtain an explainable endpoint suggestion based on available approved measurements and user priorities.
Preconditions: Recommendation baseline is enabled; endpoint candidates and permitted input measurements may be available. Custom ML is not assumed.
Trigger: User opens recommendations or requests a new recommendation.
Related requirements: REQ-FUNC-005, REQ-FUNC-006, REQ-FUNC-007, REQ-PERF-001, REQ-ML-001 (conditional), REQ-SEC-003



UC-04 flow diagram

Main success scenario

The context collector requests only approved OS/endpoint indicators.

The collector labels each field with source, timestamp and availability.

The recommendation service validates input completeness and candidate eligibility.

A versioned rule-based baseline ranks eligible endpoints when enough valid evidence exists.

The service returns ranked IDs, supported reason codes and method version, or an explicit insufficient-data/no-candidate status.

The UI explains the main supported factors and presents the suggestion.

The user accepts, ignores or declines the suggestion; no connection change occurs solely because it was displayed.

Alternative and failure flows

A1 — Permission denied or inputs missing: Return insufficient-data and explain which inputs are unavailable.

A2 — No eligible candidates: Return no_available_candidate and do not fabricate a recommendation.

A3 — Conflicting/stale measurements: Exclude or flag them using documented freshness rules; do not treat them as current live metrics.

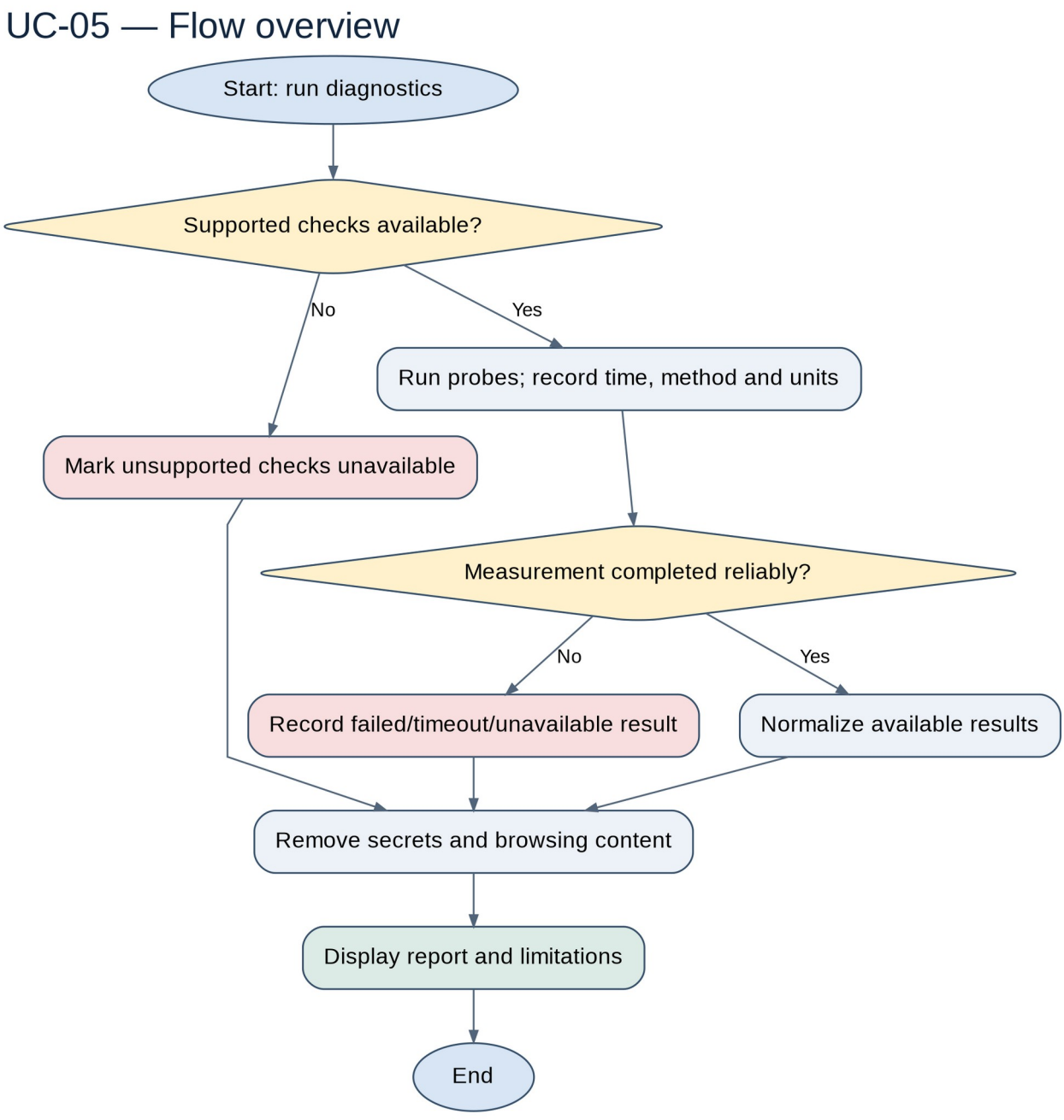
A4 — ML not approved/configured: Use the approved rule-based baseline or report unavailable; do not imply a trained model is running.

A5 — User declines: Record no selection/accepted action; do not switch endpoints.

Postconditions: The user receives a supported suggestion or a truthful insufficiency result; no unapproved automatic network action occurs.

UC-05 — Run on-demand network diagnostics

Primary actor: End user / tester
Goal: Measure available connectivity indicators on demand and explain results and limitations.
Preconditions: Diagnostics screen is available; implemented probes or platform metrics are defined.
Trigger: User taps Run Diagnostics.
Related requirements: REQ-FUNC-009, REQ-INT-001, REQ-SEC-003, REQ-OBS-001, REQ-AVAIL-001



UC-05 flow diagram

Main success scenario

The user starts a diagnostic run.

The app identifies the checks supported by this build/platform.

Each supported check runs and records timestamp, method, duration and result.

The service normalizes results and units; unsupported or failed checks remain unavailable/failed rather than fabricated.

The UI displays completed results and explains limitations.

Logs/evidence omit browsing content, private keys and raw tunnel profile material.

Alternative and failure flows

A1 — Permission unavailable: Skip affected checks and mark them unavailable with reason.

A2 — Network unavailable: Show failed/unreachable checks and preserve the result as a failed diagnostic, not a successful run.

A3 — Probe timeout: Record timeout with duration and allow a retry.

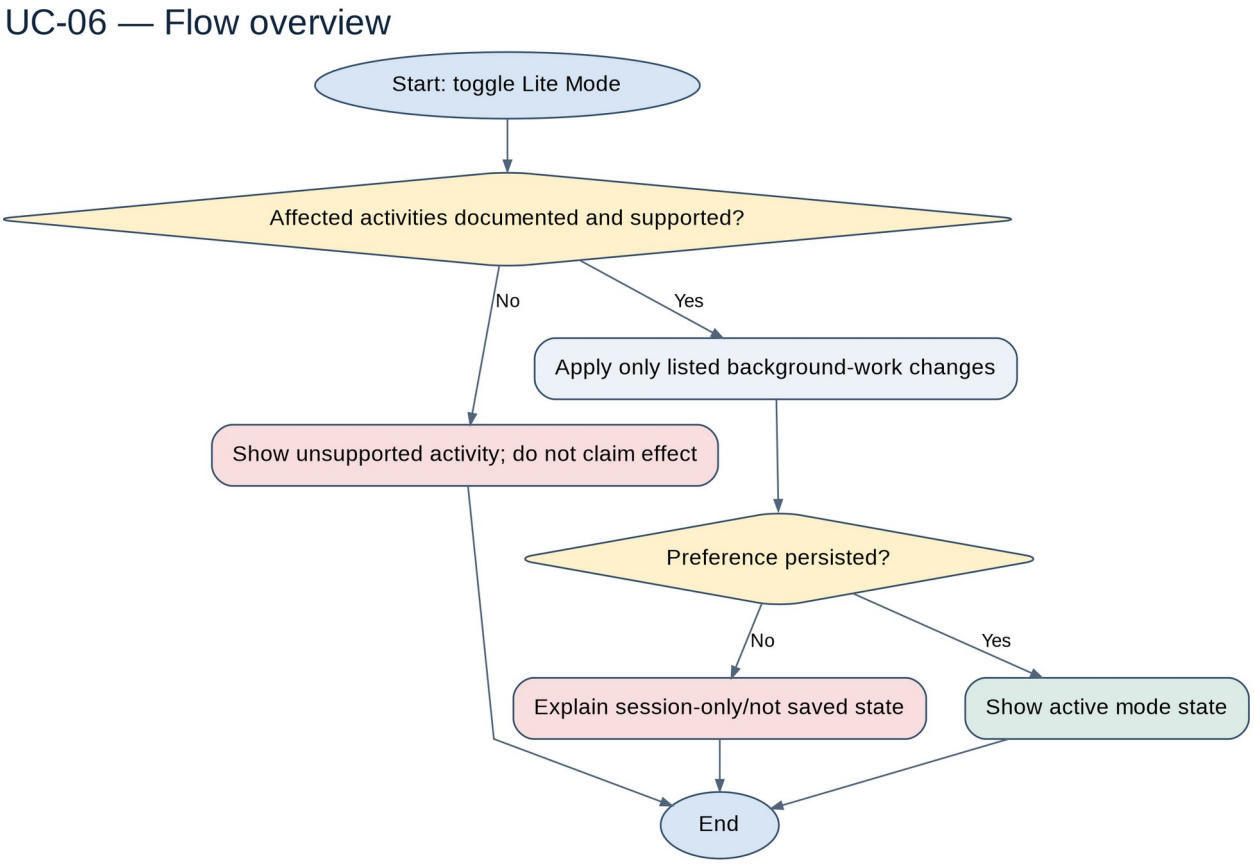
A4 — Remote diagnostics service unavailable: Continue local checks where possible; report optional-service failure.

A5 — No reliable metric: Do not show sample numbers as real measurements.

Postconditions: A timestamped diagnostic result is displayed; only approved minimal evidence is retained.

UC-06 — Enable or disable Lite Mode

Primary actor: End user
Goal: Control explicitly defined non-essential background work on constrained devices or networks.
Preconditions: Lite Mode behavior is documented for the current build; specific activities and settings are identified.
Trigger: User toggles Lite Mode in Settings or a dedicated UI.
Related requirements: REQ-FUNC-008, REQ-PERF-002, REQ-INT-003, REQ-OBS-001



UC-06 flow diagram

Main success scenario

The user opens Lite Mode controls.

The UI explains the exact behavior changed by the current implementation.

The user enables or disables Lite Mode.

The app persists the preference locally when persistence is enabled.

The scheduler/components apply the documented changes.

The UI reflects active mode; any claimed resource improvement is deferred until measurement evidence exists.

Alternative and failure flows

A1 — Persistence failure: Apply only if safely possible for the current session and show that the preference was not saved.

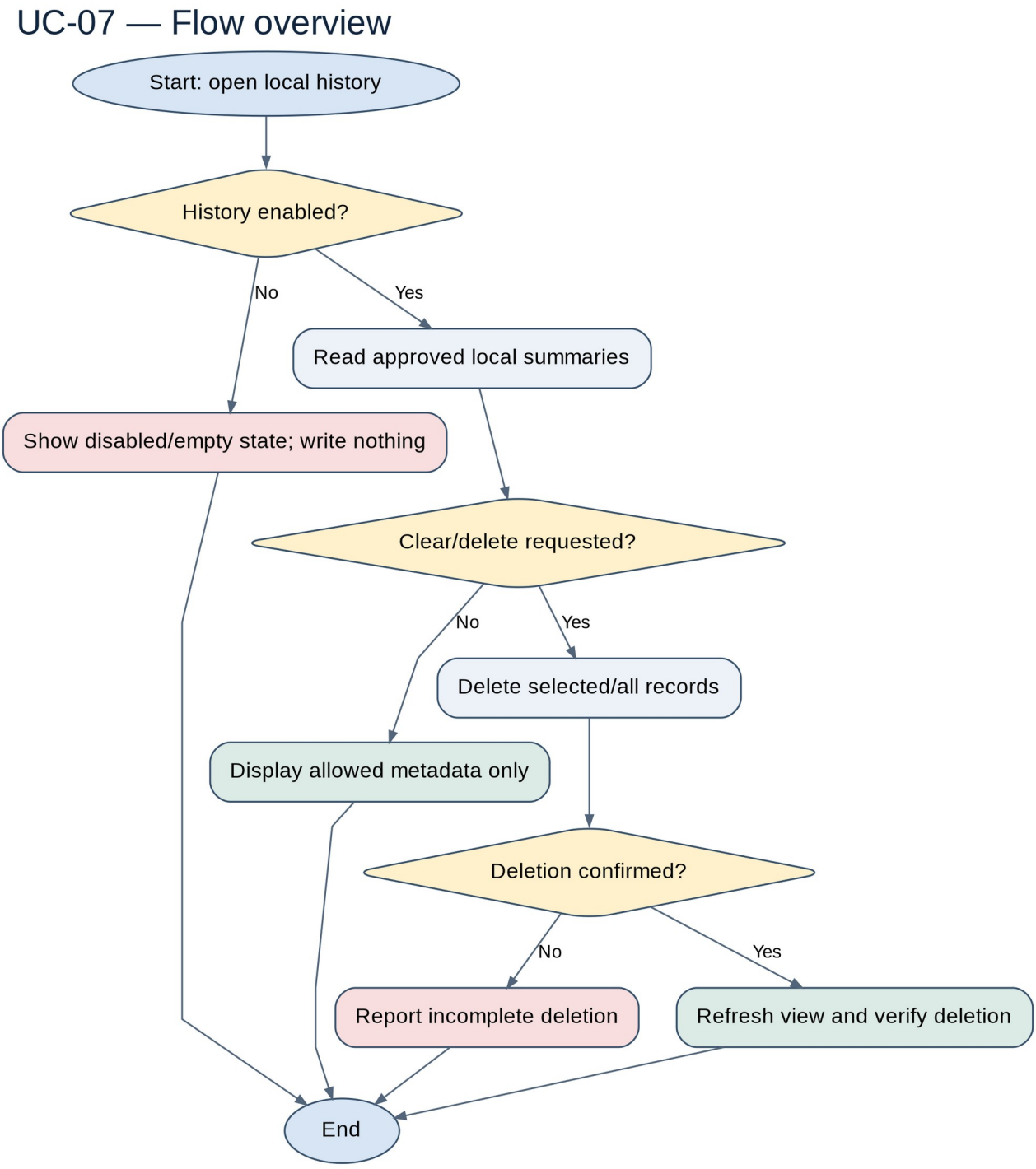
A2 — Activity not supported on platform: Mark that activity unsupported; do not imply universal control.

A3 — User requests savings metric: Show measured result only when controlled measurements exist; otherwise state not yet measured.

Postconditions: Lite Mode state is clear and any active behavior matches the documented activity list.

UC-07 — View and clear local connection history

Primary actor: End user
Goal: Review or delete optional local session summaries while retaining privacy boundaries.
Preconditions: History feature is included and explicitly enabled; fields, local location and retention/deletion behavior are defined.
Trigger: User opens History, selects a record or chooses Clear History.
Related requirements: REQ-FUNC-010, REQ-INT-003, REQ-SEC-001, REQ-SEC-003, REQ-OBS-001



UC-07 flow diagram

Main success scenario

The app checks whether history is enabled.

If enabled, the app reads local session summaries.

The UI displays only approved metadata such as start/end, duration, endpoint label and counters when available.

The user selects an entry or requests clearing the history.

The app confirms destructive bulk deletion when required by the UI design.

The app deletes the chosen records and verifies the view reflects deletion.

Alternative and failure flows

A1 — History disabled: Show a clear disabled/empty state; do not silently create records.

A2 — Storage read failure: Show a safe error and keep data unchanged.

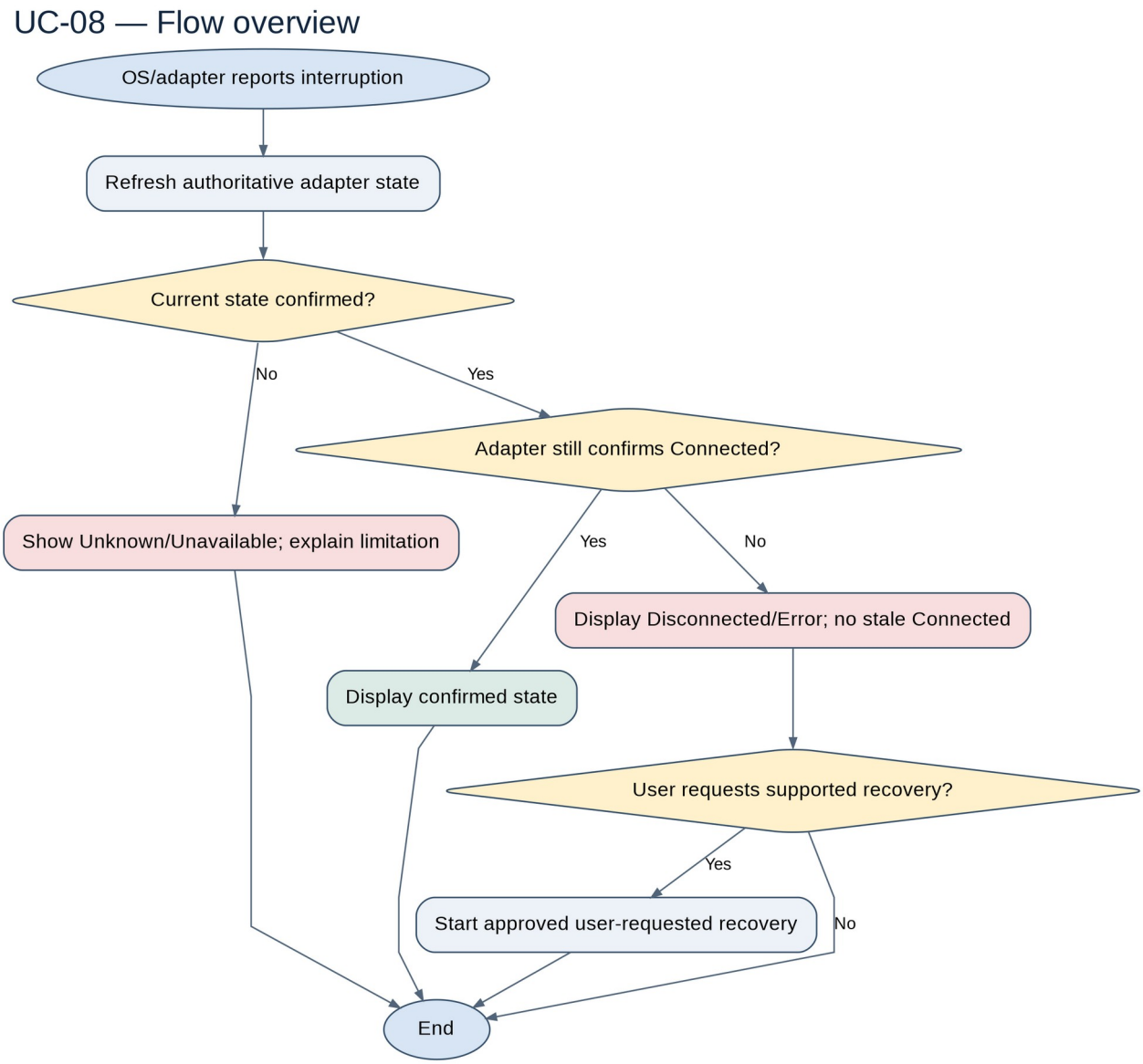
A3 — Deletion failure: Report incomplete deletion, do not claim success, and preserve diagnostic evidence without sensitive values.

A4 — Cloud sync requested: Treat cloud sync as out of baseline; require separate approved requirement, opt-in and retention/deletion design.

Postconditions: Only locally stored, approved history is displayed or deleted; no browsing content or profile secret is included.

UC-08 — Recover visible state after network/tunnel interruption

Primary actor: End user (system event triggered by OS/adapter)
Goal: Make the interface reflect network loss or tunnel failure instead of showing a stale connection.
Preconditions: The adapter can report network/tunnel state changes; app is running or resuming.
Trigger: OS/adapter reports network loss, tunnel stop, error or a state refresh on app resume.
Related requirements: REQ-FUNC-001, REQ-FUNC-003, REQ-REL-001, REQ-INT-001, REQ-INT-002, REQ-OBS-001



UC-08 flow diagram

Main success scenario

The adapter emits a loss/error/state event or the app requests a state refresh.

The coordinator validates the event and timestamp/source.

The UI updates to the adapter-reported Disconnected/Error/Unknown state.

The UI explains the interruption without leaking secrets.

If retry/reconnect is offered, it remains user-confirmed unless a separately approved automatic action exists.

A privacy-aware diagnostic event is captured if required by the verification design.

Alternative and failure flows

A1 — Event missing/out of order: Refresh authoritative adapter state and avoid retaining a stale Connected state without confirmation.

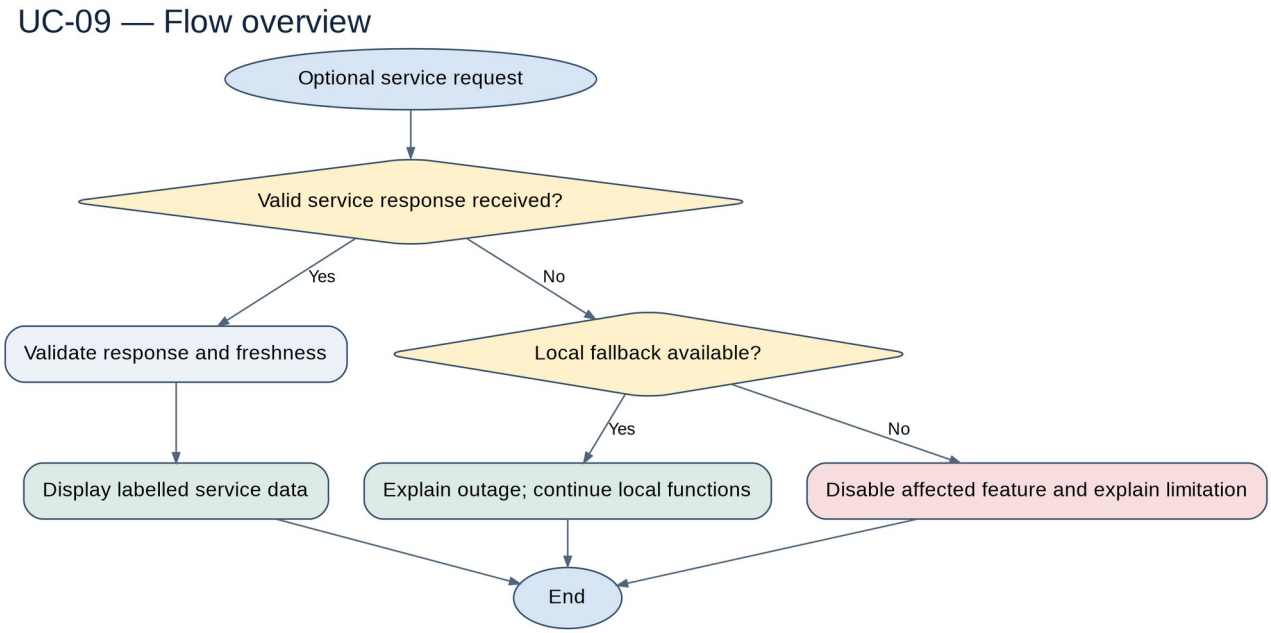
A2 — Adapter unavailable: Show unknown/unavailable state and explain that current tunnel status cannot be confirmed.

A3 — Automatic retry not in scope: Do not silently reconnect; provide manual recovery action where supported.

Postconditions: Visible state does not falsely indicate an active connection; any recovery action remains within approved scope.

UC-09 — Continue safely when an optional remote service is unavailable

Primary actor: End user
Goal: Use unrelated local functions and receive a clear explanation when an optional endpoint/diagnostics service fails.
Preconditions: A build may have an optional remote endpoint directory or service; local features exist independently where supported.
Trigger: Service timeout, unavailable response or offline condition occurs.
Related requirements: REQ-INT-004, REQ-AVAIL-001, REQ-FUNC-009, REQ-SEC-001, REQ-OBS-001



UC-09 flow diagram

Main success scenario

The client calls the optional service only if configured for this build.

The service adapter detects timeout/error/unreachable response.

The adapter returns a normalized unavailable/error result.

The UI explains which remote capability is unavailable and avoids exposing stack traces/secrets.

The app continues local settings and supported local diagnostics where feasible.

The app retries only under a documented policy and avoids uncontrolled retry loops.

Alternative and failure flows

A1 — No local fallback exists: Disable the affected function with a clear explanation, rather than displaying placeholder data as live.

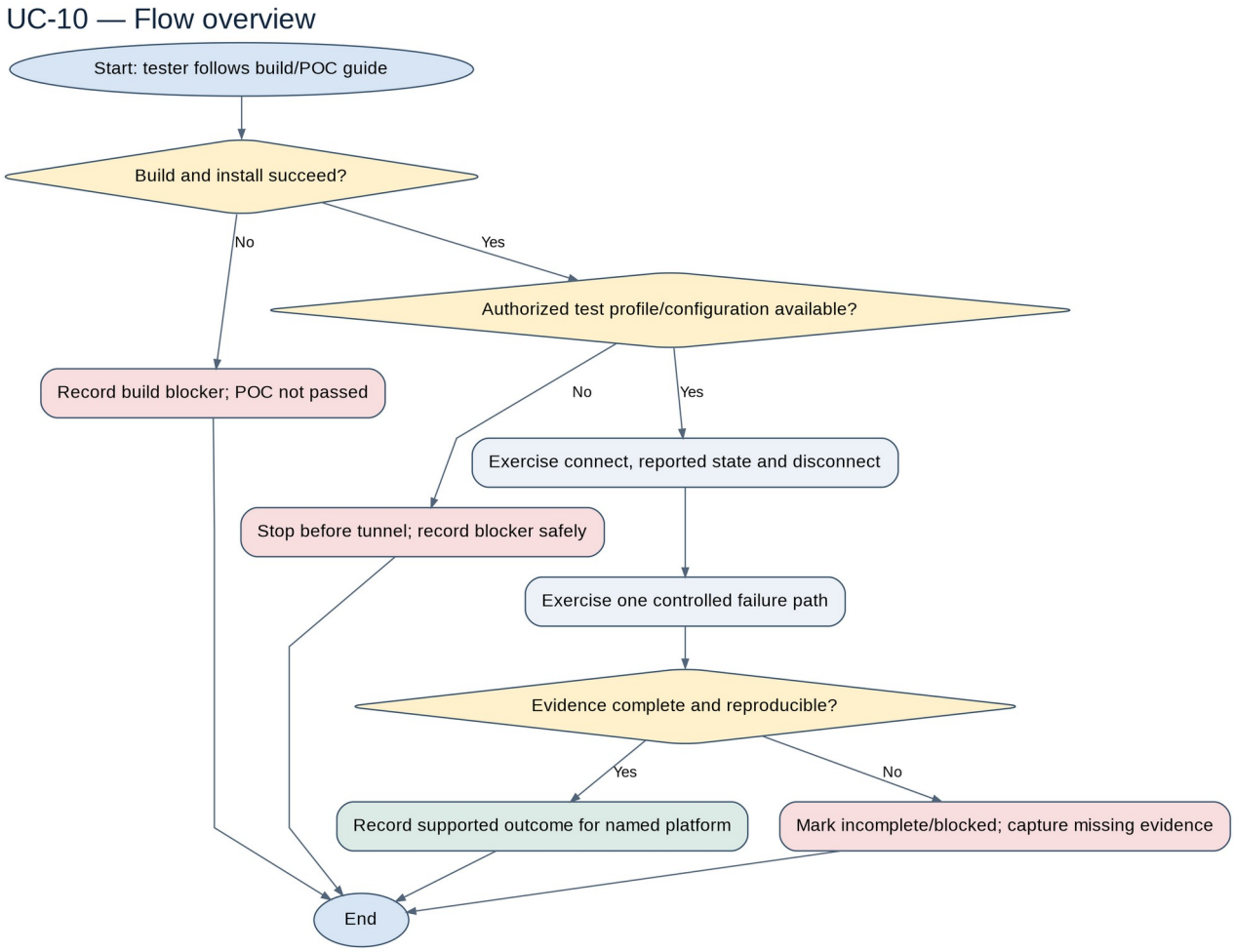
A2 — Cached data exists: Use only if explicitly permitted and label source/freshness/expiry.

A3 — Authentication/API not configured: Treat as a configuration failure, not an endpoint outage; do not invent successful service results.

Postconditions: Unrelated local functions remain usable where feasible; unavailable remote features are labeled truthfully.

UC-10 — Build, install and validate a product release

Primary actor: Developer / tester
Goal: Validate a reproducible product build, platform-specific connection lifecycle, failure handling and release evidence.
Preconditions: Source and dependencies are available; secrets are supplied securely outside version control; authorized test profile/endpoint and a supported device/emulator are available.
Trigger: Release engineer follows the versioned build, install and release-verification procedure.
Related requirements: REQ-POC-001, REQ-INST-001, REQ-BUILD-001, REQ-COMP-001, REQ-COMP-002, REQ-PORT-001, REQ-SEC-001, REQ-SEC-002, REQ-MAINT-001



UC-10 flow diagram

Main success scenario

Tester checks out the documented revision and reads prerequisites.

Tester verifies toolchain and dependency/license inventory.

Tester supplies non-secret configuration and secrets through approved local means.

Tester builds and installs the application on the named platform/device.

Tester exercises connect, confirmed state and disconnect using an authorized profile.

Tester triggers at least one documented failure (such as permission denial or endpoint failure).

Tester records build/version, platform/device, inputs, expected/actual outcome and evidence.

Tester documents limitations and labels untested platforms as unverified.

Alternative and failure flows

A1 — Build fails: Record exact toolchain/error details; do not mark the release verification as passed.

A2 — Unauthorized or missing profile: Stop before initiating a tunnel and record the blocker without sharing secrets.

A3 — Platform interface unsupported: Record as a feasibility limitation and restrict the platform support claim.

A4 — Dependency license unclear: Do not include the dependency in a release baseline until reviewed.

Postconditions: A reproducible evidence record supports pass/fail/blocked status for one named platform only.

UC-11 — Onboard and configure permissions

Goal: The user launches DēGlitch for the first time or resets onboarding.
Primary actor: New user
Preconditions: The application is available; configuration and permissions required for the selected path are satisfied. Purpose-specific consent is checked before optional processing.
Trigger: First launch, reinstall, or user requests onboarding again.
Related requirements: REQ-ONB-001, REQ-I18N-001, REQ-UX-001, REQ-PRIV-002

Main success flow

The app introduces connectivity controls, network profile, recommendations, Lite Mode, AI assistant and privacy choices.

The user selects a language and reviews the relevant privacy and permission explanations.

The app requests each OS permission only when the associated feature is about to be used and the platform requires it.

The user chooses optional features and consent settings; defaults for optional data processing are off.

The app saves onboarding completion and opens the dashboard.

Alternate and failure flows

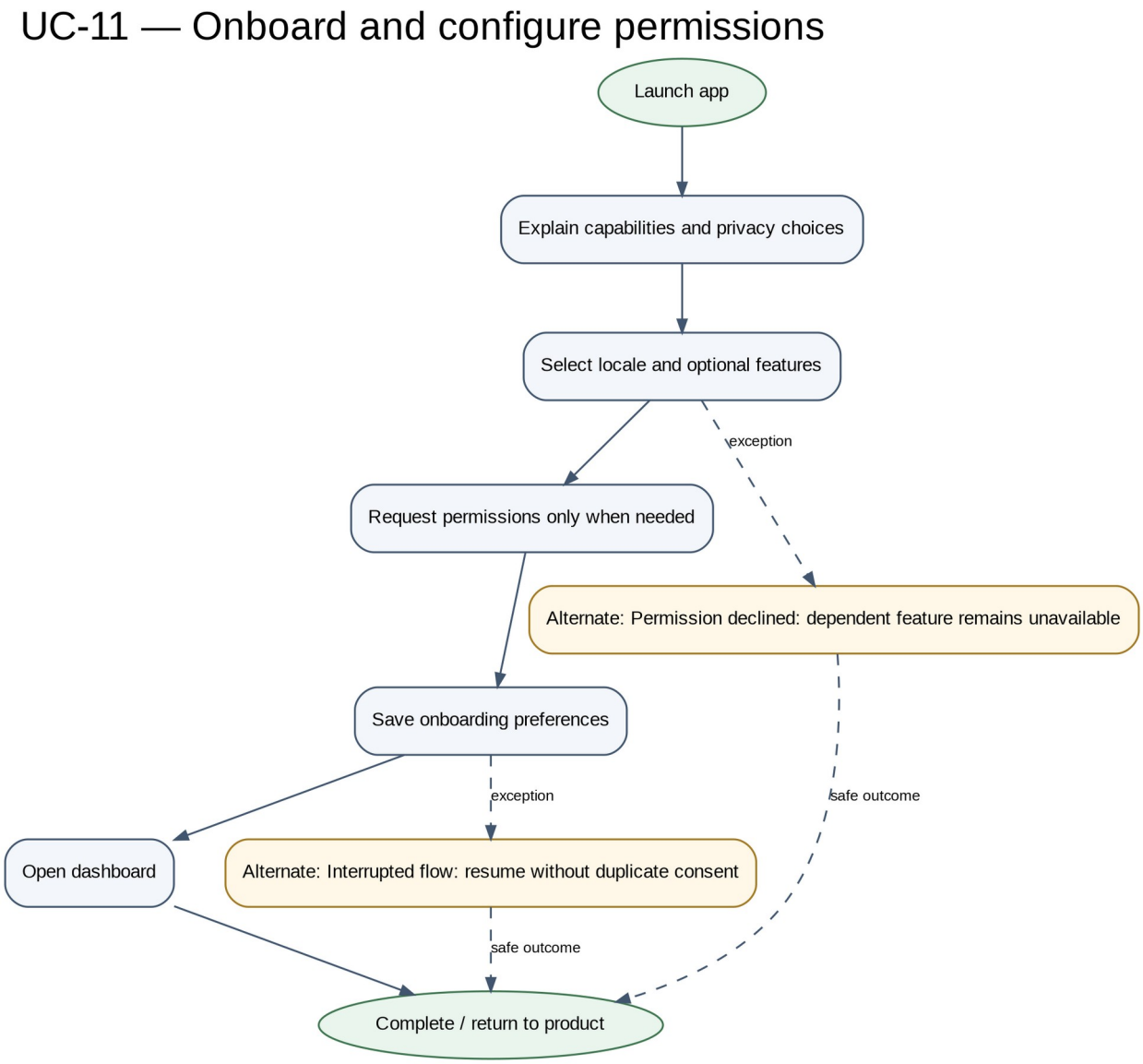
If permission is declined, the app continues and marks dependent values unavailable.

If a locale is not fully available, the app identifies the fallback locale for critical messages.

If onboarding is interrupted, the app resumes without duplicating consent or creating a false success state.

Postconditions

The app stores only the onboarding and preference state required for the stated purposes.



UC-12 — Register, authenticate and manage a session

Goal: The user chooses sign-up, sign-in, recovery or sign-out.
Primary actor: Account holder
Preconditions: The application is available; configuration and permissions required for the selected path are satisfied. Purpose-specific consent is checked before optional processing.
Trigger: User starts an identity action or the session reaches its expiry/revocation condition.
Related requirements: REQ-AUTH-001, REQ-AUTH-002, REQ-AUTH-003, REQ-API-001, REQ-API-002

Main success flow

The client validates required fields and submits an identity request to the configured identity provider over a protected channel.

The provider validates credentials and returns a success or a generic safe failure.

The application establishes a session only after validated provider response and applies session policy.

Protected API requests include the required session context; the server validates identity, ownership and role.

On sign-out, expiry or revocation, the client clears local session material and returns to a public screen.

Alternate and failure flows

Invalid credentials are handled without confirming whether an account exists.

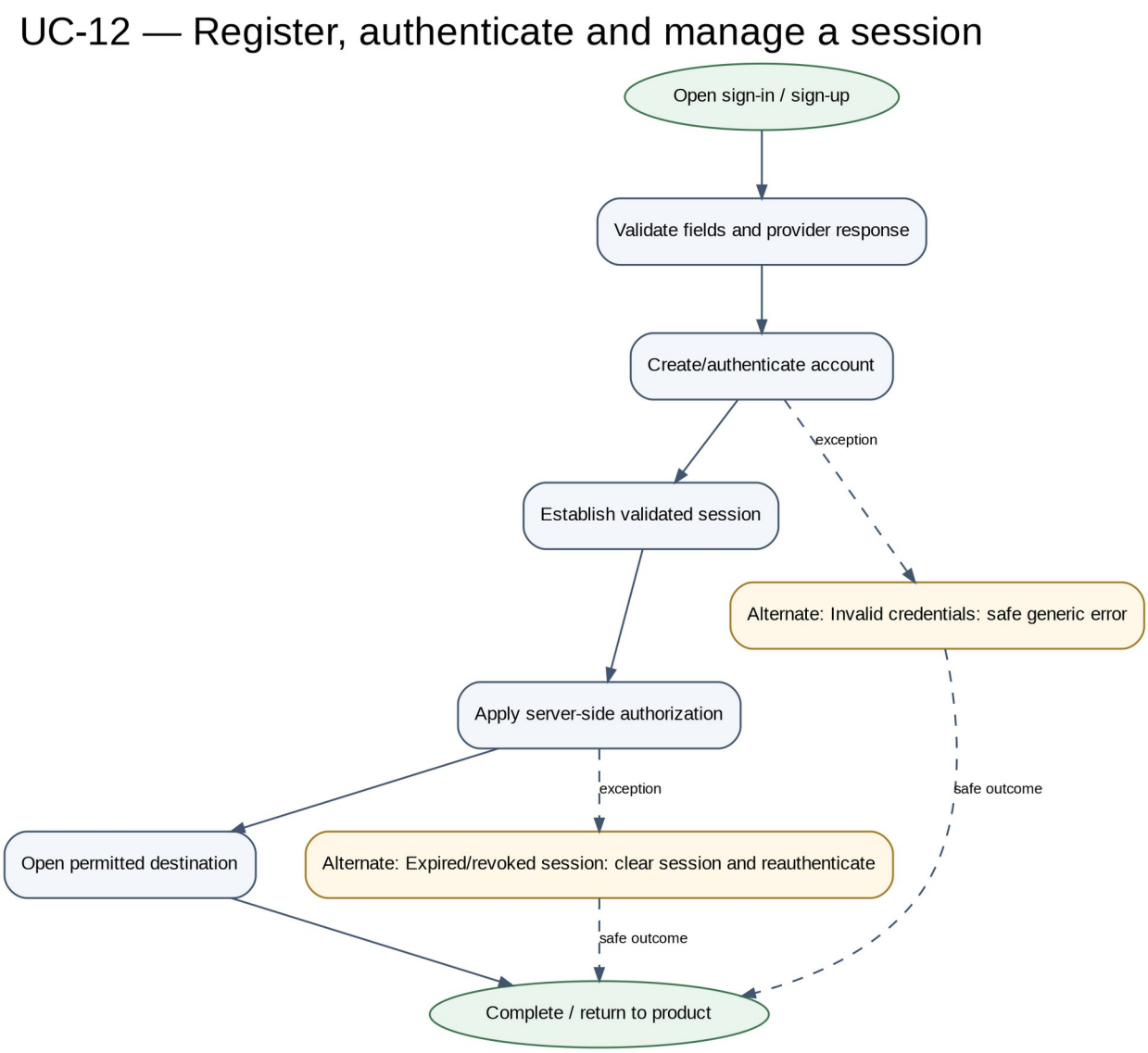
Expired or revoked sessions require reauthentication and cannot access protected records.

Recovery uses the configured provider flow and rate limits.

If the identity provider is unavailable, the app explains the limitation and does not fabricate a logged-in state.

Postconditions

The user is authenticated or returned to an unauthenticated state; privilege checks remain server-side.



UC-13 — Understand the current network profile

Goal: The user opens Network Profile to understand current connection constraints.
Primary actor: End user
Preconditions: The application is available; configuration and permissions required for the selected path are satisfied. Purpose-specific consent is checked before optional processing.
Trigger: User opens the profile view or requests an explanation in the assistant.
Related requirements: REQ-INT-001, REQ-FUNC-005, REQ-NPROF-001, REQ-NPROF-002, REQ-OBS-001

Main success flow

The app obtains permitted OS signals and recent authorized measurements.

Each value is tagged with source, timestamp/freshness and availability state.

The app derives a concise profile summary using documented rules and does not equate radio type with quality.

The UI explains observed constraints and what they may mean for VPN use, streaming, gaming or study.

The user can inspect factor-level explanations and refresh the measurements.

Alternate and failure flows

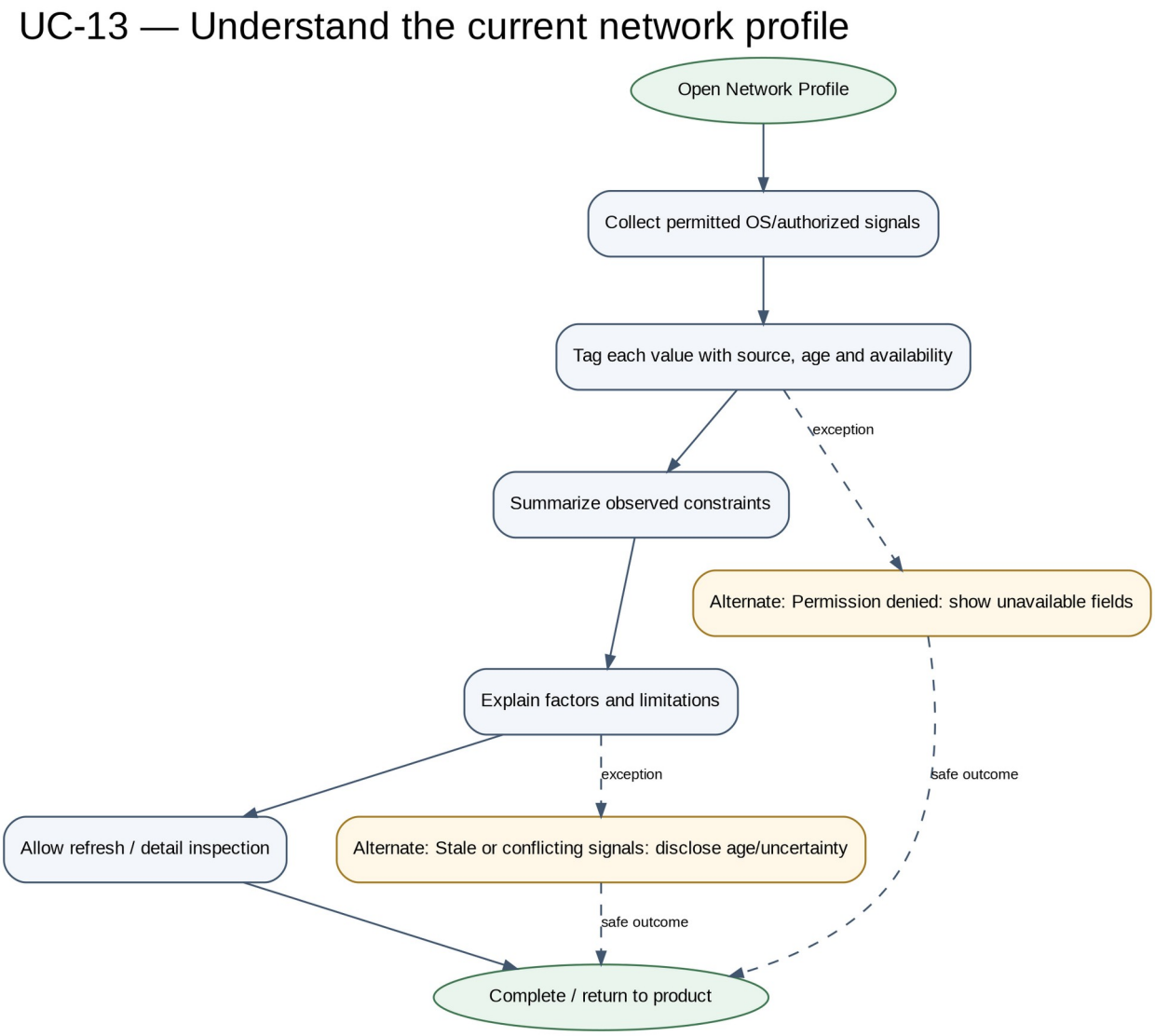
Denied or unsupported fields are shown as unavailable.

Stale cached observations are clearly aged and are not described as live.

Contradictory measurements are shown with caveats rather than collapsed into a certain diagnosis.

Postconditions

A profile summary is shown with provenance and uncertainty; no network configuration changes automatically.



UC-14 — Configure activity and connection preferences

Goal: The user sets desired activities and connectivity priorities.
Primary actor: End user
Preconditions: The application is available; configuration and permissions required for the selected path are satisfied. Purpose-specific consent is checked before optional processing.
Trigger: User opens or edits preferences.
Related requirements: REQ-PREF-001, REQ-FUNC-006, REQ-FUNC-007, REQ-DATA-001

Main success flow

The app presents explicit options such as balanced, low latency, stability, reduced data use, gaming, streaming, studying and work.

The user chooses, edits or clears one or more preferences.

The app validates allowed values and saves them to the configured local/account store.

The recommendation engine receives only the approved preference fields.

The UI displays the saved choices and offers reset.

Alternate and failure flows

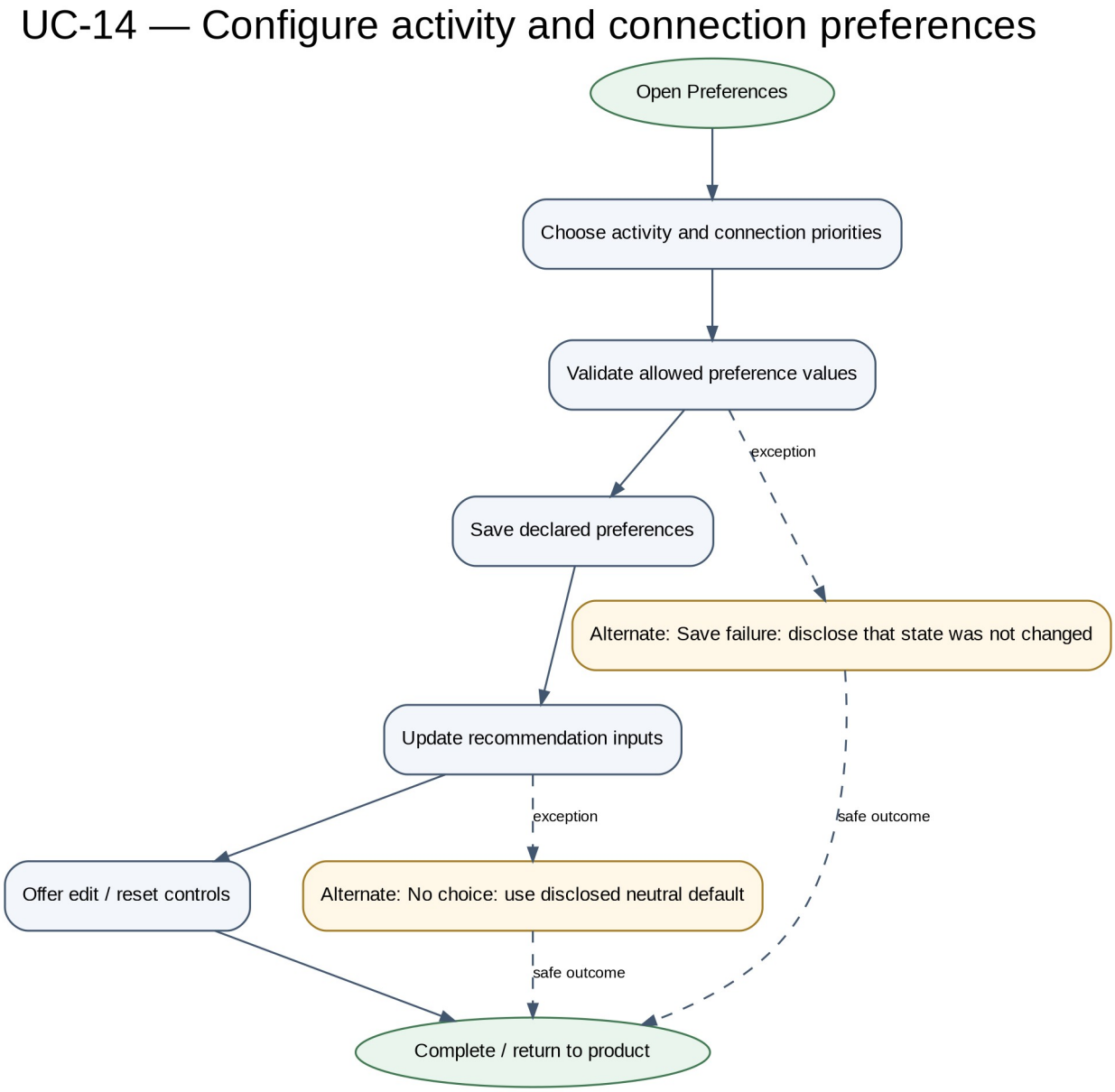
If persistence fails, the UI reports that preferences were not saved.

If no preference is selected, a documented neutral default is applied and labelled.

An inferred suggestion never silently overwrites a declared preference.

Postconditions

Declared preferences reflect the user's explicit choices; no inference is implied.



UC-15 — Receive and manage predictive preference suggestions

Goal: The user opts in to preference prediction and reviews the resulting suggestions.
Primary actor: Consenting end user
Preconditions: The application is available; configuration and permissions required for the selected path are satisfied. Purpose-specific consent is checked before optional processing.
Trigger: User enables the distinct activity-inference purpose or opens the prediction screen after consent.
Related requirements: REQ-PREF-001, REQ-PREF-002, REQ-PREF-003, REQ-PRIV-002, REQ-ML-001, REQ-ML-002, REQ-DATA-001

Main success flow

The app describes which coarse activity signals are used, why they are needed, the retention policy and how to withdraw.

The user gives explicit opt-in consent for the stated purpose.

The inference pipeline checks signal validity, freshness, minimum evidence and consent at processing time.

The system generates a candidate preference with method version, reasons, confidence only if calibrated, and expiry.

The user reviews the explanation and chooses accept, edit, reject or defer.

Only accepted/editable preference state enters subsequent recommendations according to user settings.

Alternate and failure flows

No consent, expired consent or inadequate data means no prediction.

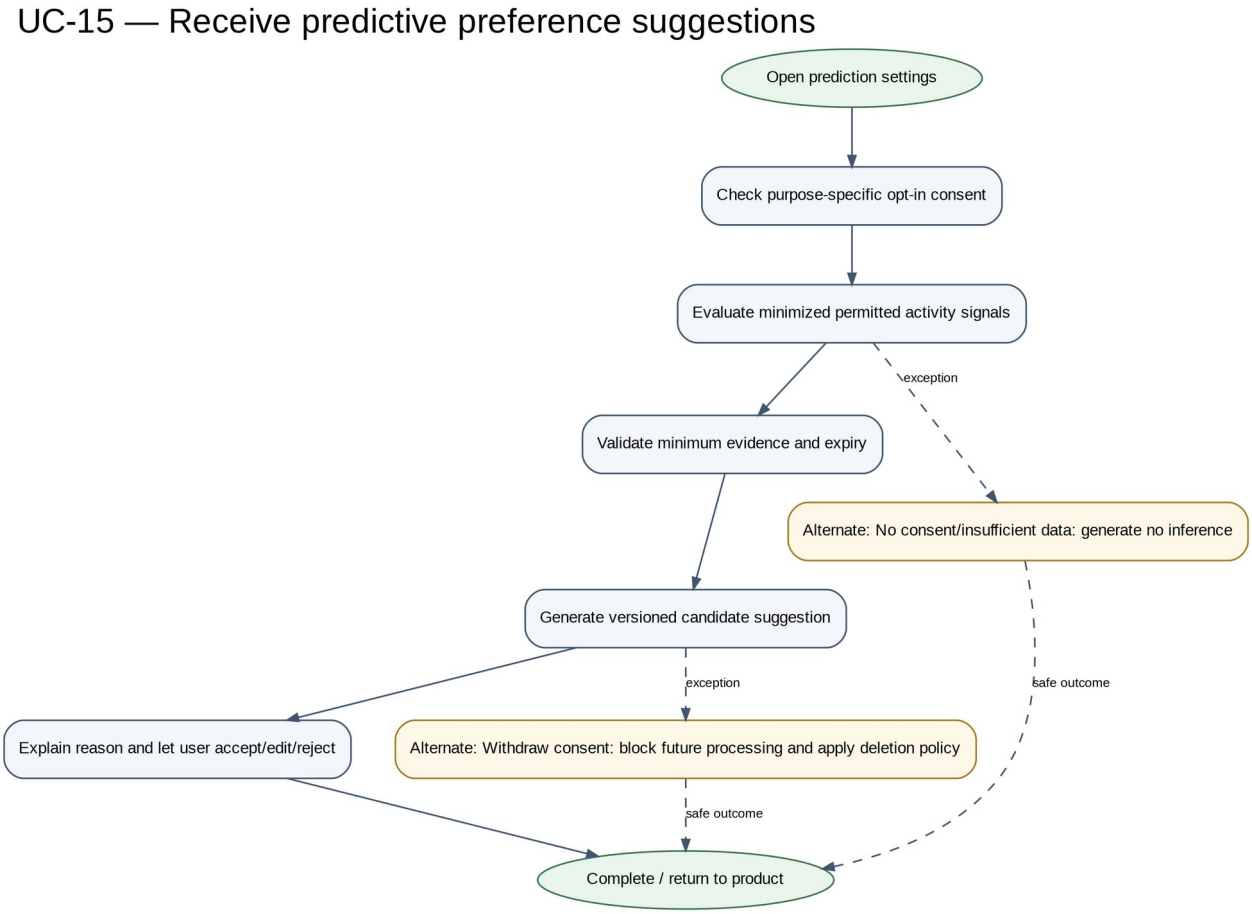
The user rejects a suggestion; the same current inference is not immediately reapplied against that preference.

Consent withdrawal blocks future processing and invokes documented expiry/deletion actions.

A model/service outage returns a safe no-suggestion state.

Postconditions

The chosen preference state and consent status are recorded; prediction is advisory and reversible.



UC-16 — Use AI chat for product and network assistance

Goal: The user asks DēGlitch AI chat about app behavior, the network profile or recommendations.
Primary actor: End user
Preconditions: The application is available; configuration and permissions required for the selected path are satisfied. Purpose-specific consent is checked before optional processing.
Trigger: User sends a question or follows an assistant prompt.
Related requirements: REQ-AI-001, REQ-AI-002, REQ-AI-003, REQ-AI-004, REQ-AI-005, REQ-API-001, REQ-API-002, REQ-PERF-003, REQ-AVAIL-002, REQ-PRIV-002

Main success flow

The application checks whether AI assistance is enabled and identifies the request purpose.

It assembles only approved context such as profile factors, current diagnostics supplied by the app, selected preferences or approved help content.

Secrets, raw tunnel configuration, browsing content, and unrelated account data are excluded.

The assistant retrieves approved knowledge or uses a selected provider/model and generates an answer.

A grounding/safety layer checks source freshness, unsupported claims, action requests and sensitive content.

The UI displays the answer with relevant sources, data age, reason codes or limitations.

For any consequential action the assistant asks for separate confirmation and routes the confirmed request through the normal application service.

Alternate and failure flows

No live telemetry is available; the assistant says it cannot see that metric and may offer to run an approved diagnostic.

No relevant source is available; it returns a bounded fallback rather than inventing facts.

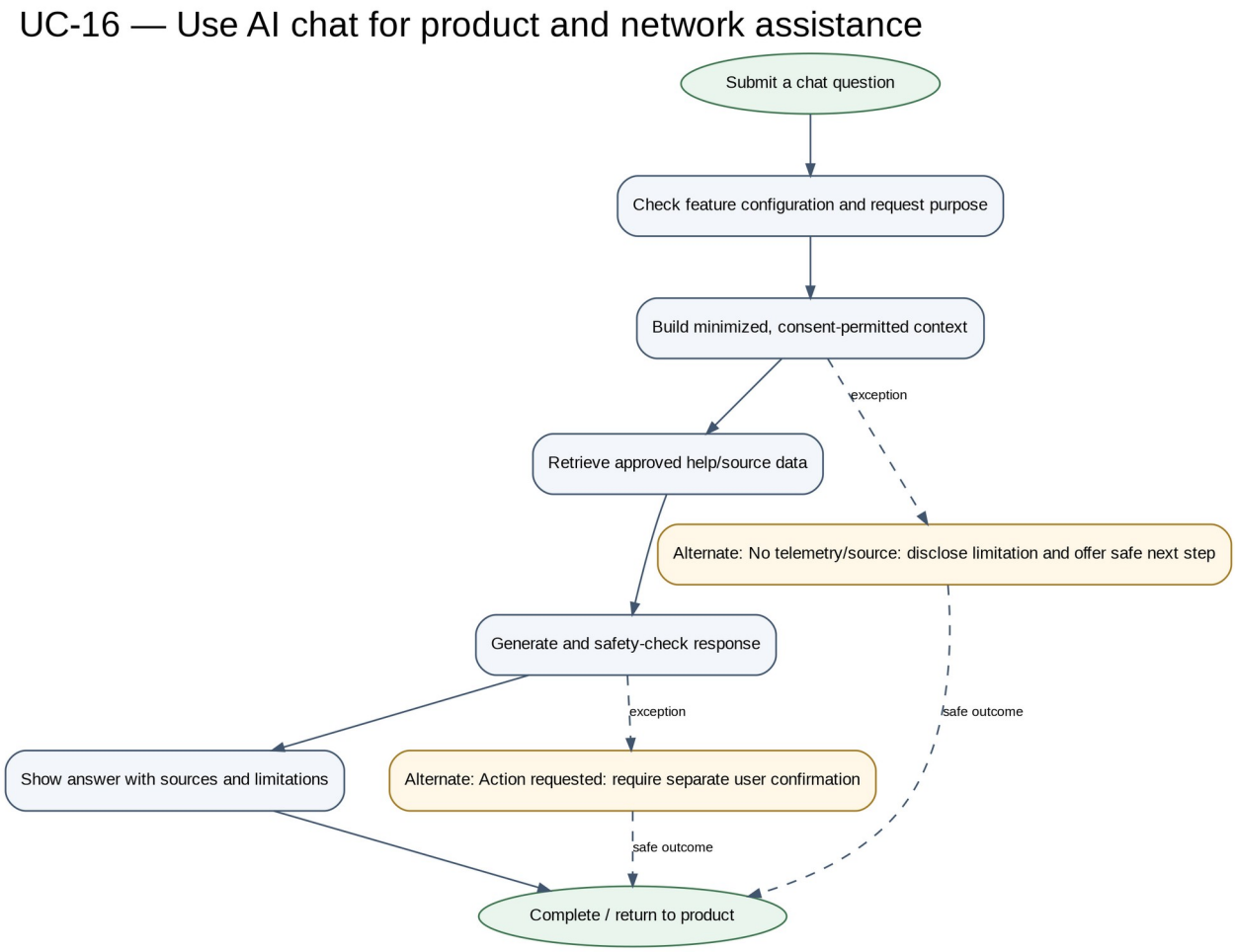
Provider timeout or failure triggers bounded retry/circuit breaker and a clear fallback.

The user asks to connect or change settings; no action occurs until explicit confirmation.

The user deletes chat or withdraws the relevant consent; applicable records are removed/expired under policy.

Postconditions

The answer is displayed with honest boundaries; no unauthorized action or unsupported live-data claim occurs.



UC-17 — Search, filter and favorite endpoints

Goal: The user finds and favorites an endpoint.
Primary actor: End user
Preconditions: The application is available; configuration and permissions required for the selected path are satisfied. Purpose-specific consent is checked before optional processing.
Trigger: User opens the endpoint directory.
Related requirements: REQ-FUNC-004, REQ-INT-004, REQ-ENDPT-001, REQ-ENDPT-002, REQ-AVAIL-001, REQ-SEC-002

Main success flow

The app fetches approved catalogue metadata or uses a cache with a visible freshness marker.

The user searches and filters by available attributes.

The system evaluates whether each result is currently eligible under approved catalogue state and client policy.

The UI shows location/region, supported status and source/freshness for health data if available.

The user adds/removes a favorite or explicitly selects an endpoint.

A selected endpoint is mapped to an authorized profile before a connection can be requested.

Alternate and failure flows

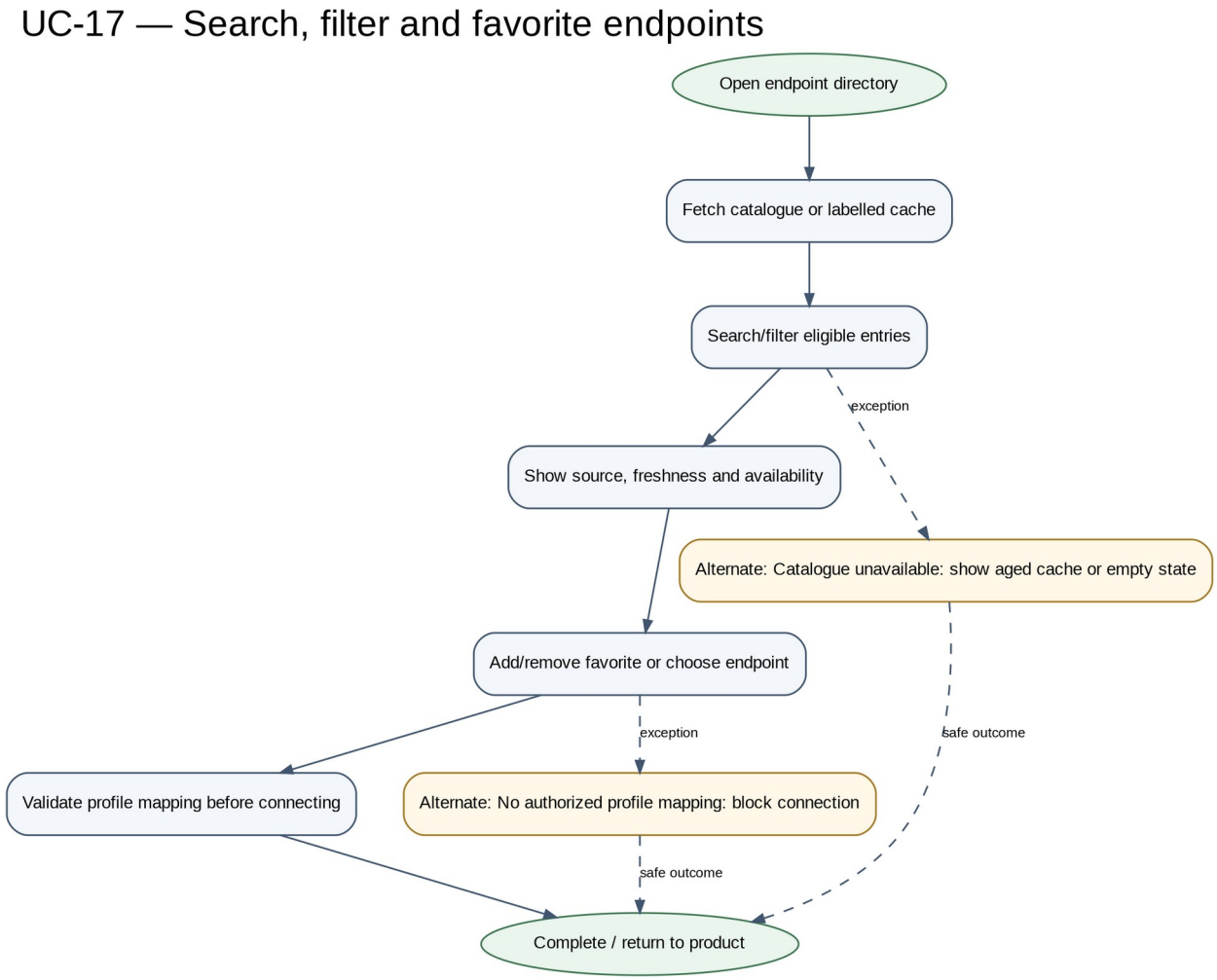
Catalogue is unavailable; cached entries are labelled with age or an explicit empty/unavailable state is shown.

A stale or disabled endpoint cannot be treated as healthy/live.

No authorized profile mapping exists; selection remains informational and connection is blocked.

Postconditions

Selection/favorites are persisted according to privacy rules; endpoint choice does not itself connect.



UC-18 — Configure language, accessibility and notifications

Goal: The user adjusts locale, accessibility or notification settings.
Primary actor: End user
Preconditions: The application is available; configuration and permissions required for the selected path are satisfied. Purpose-specific consent is checked before optional processing.
Trigger: User opens settings or OS permission changes.
Related requirements: REQ-I18N-001, REQ-UX-001, REQ-NOTIF-001, REQ-NOTIF-002, REQ-ONB-001

Main success flow

The app presents supported languages and relevant accessibility settings.

The user selects a locale, text scaling/reduced motion option and notification categories.

The app previews/apply settings and persists them through the chosen preference store.

The app requests notification permission only when needed and only after an explanation.

The UI and supported alerts follow the current settings.

Alternate and failure flows

An untranslated string uses the declared fallback and critical error language is not silently misleading.

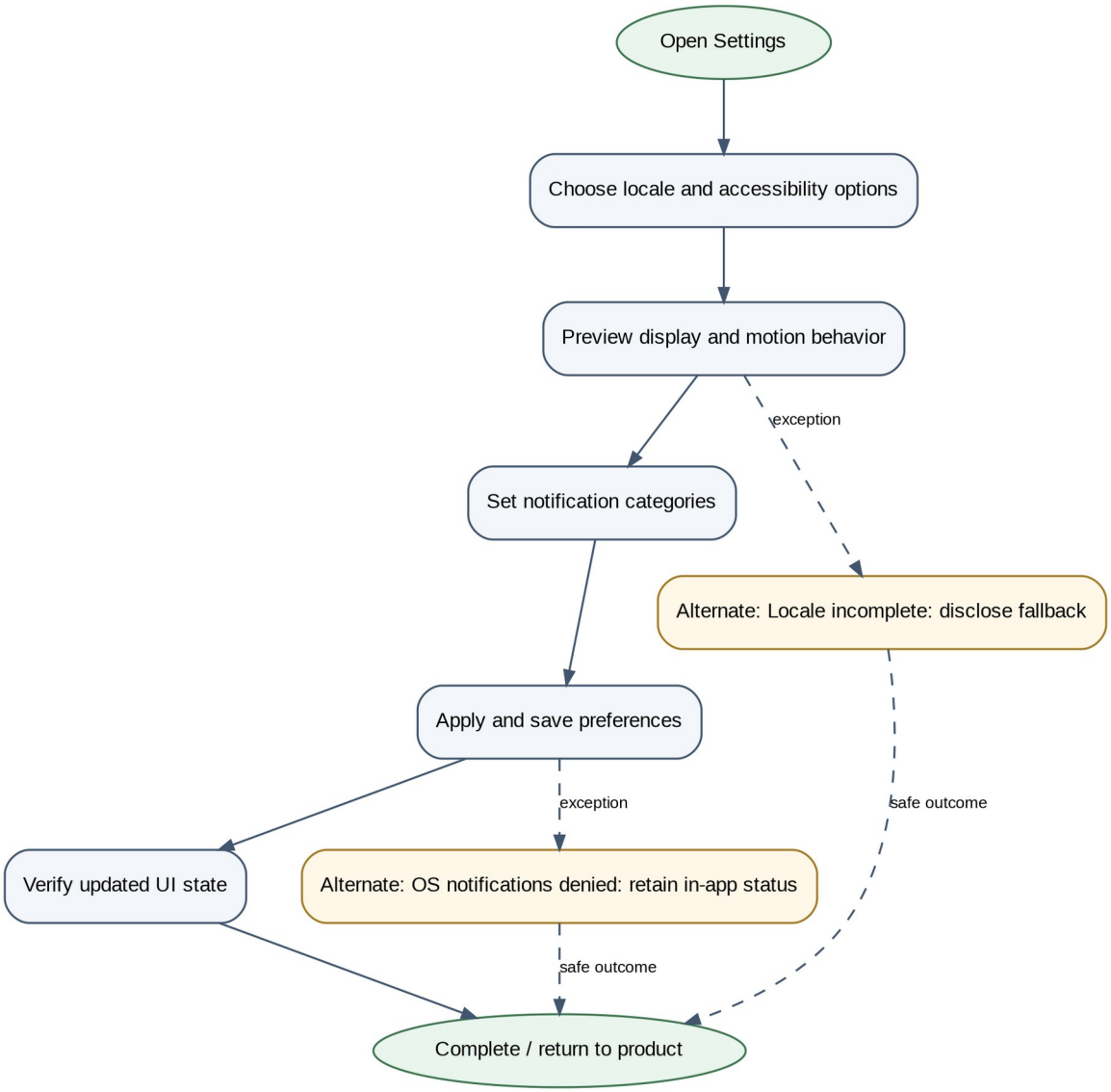
OS notification permission is denied; in-app status remains available.

Reduced motion suppresses nonessential animated transitions where supported.

Postconditions

User configuration is applied and remains editable/resettable.

UC-18 — Configure language, accessibility and notifications



UC-19 — Manage consent, export and deletion requests

Goal: The user reviews consent, exports data or requests deletion.
Primary actor: Authenticated account holder
Preconditions: The application is available; configuration and permissions required for the selected path are satisfied. Purpose-specific consent is checked before optional processing.
Trigger: User opens the privacy centre.
Related requirements: REQ-PRIV-001, REQ-PRIV-002, REQ-DATA-001, REQ-AUTH-002, REQ-AUTH-003, REQ-API-002

Main success flow

The system lists each data purpose, category, location, retention behavior and whether it is optional.

The user grants/withdraws a purpose-specific consent or submits export/deletion.

The system authenticates the requester and verifies object ownership/authorization.

The request is queued or executed across relevant stores, provider integrations, caches and local client records.

The system records safe audit metadata and shows completed, pending or failed status.

The user receives the export via a protected channel or an accurate deletion result.

Alternate and failure flows

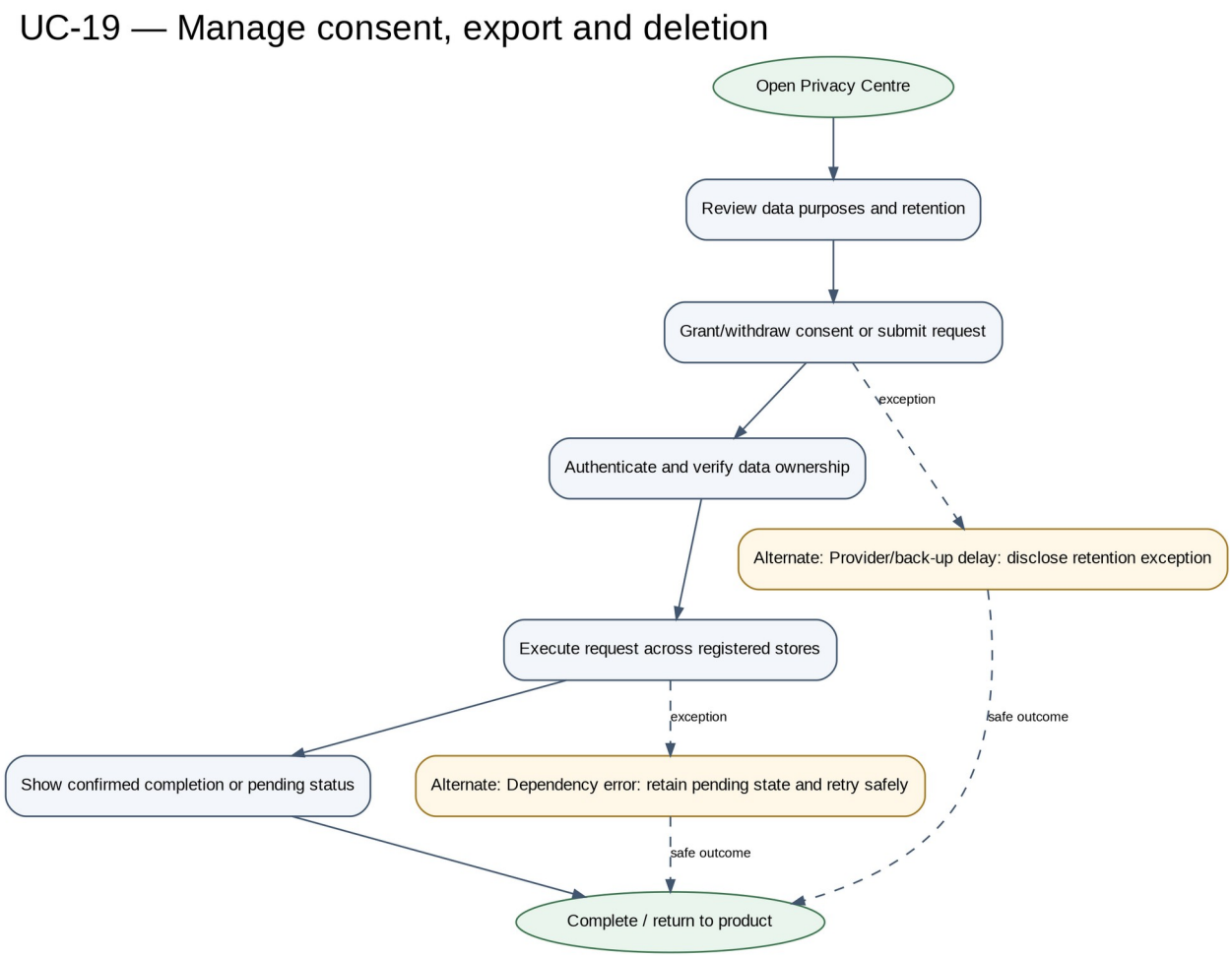
External provider/backups cannot be synchronously purged; the system communicates retention exceptions and expiry.

A dependency is unavailable; request stays pending with an auditable retry state.

A user lacks account access; protected data is not disclosed and recovery guidance is provided.

Postconditions

Consent and request state are accurate; a request is not reported completed until the defined completion criteria are met.



UC-20 — Measure Network Boost recommendations

Goal: The user requests a supported Network Boost assessment.
Primary actor: End user
Preconditions: The application is available; configuration and permissions required for the selected path are satisfied. Purpose-specific consent is checked before optional processing.
Trigger: User starts optimization.
Related requirements: REQ-NETOPT-001, REQ-FUNC-005, REQ-FUNC-006, REQ-FUNC-007, REQ-PERF-001, REQ-REL-001

Main success flow

The system identifies the relevant baseline scenario and measures supported indicators such as latency, loss, jitter, throughput or reconnection time.

It records test conditions, source, sample window and measurement availability.

The engine identifies a supported candidate action, such as trying an alternative endpoint or a validated preference.

The app explains expected trade-offs and asks for approval.

After approval, the app applies only changes supported by the current platform/tunnel integration.

The system repeats measurements using comparable conditions and reports observed differences with limitations.

Alternate and failure flows

Baseline is missing/incomplete; no percentage claim is displayed.

The proposed action is unsupported; the user receives an explanation without a false success message.

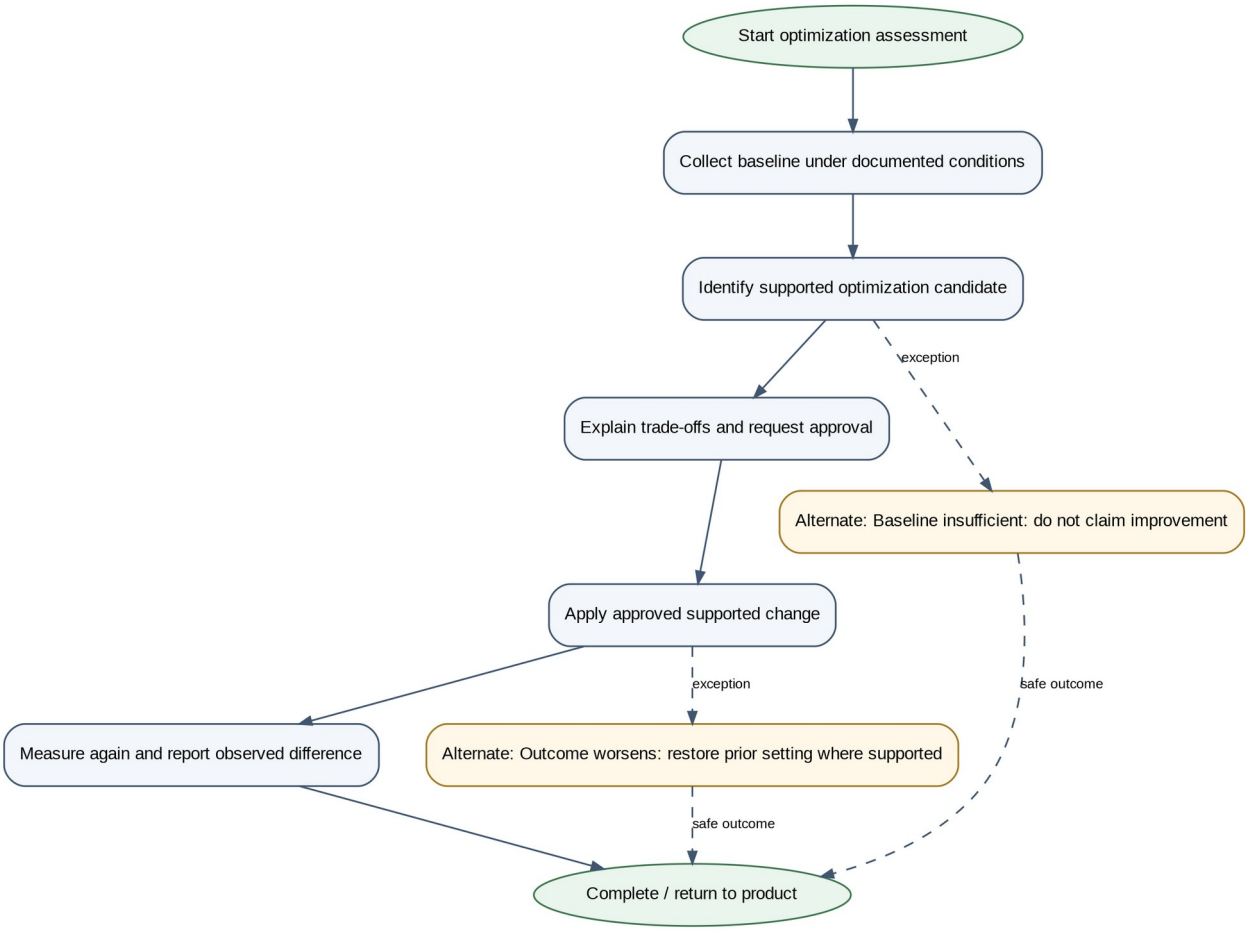
Post-action result worsens; restore previous settings where supported and describe the outcome.

The endpoint changes only after explicit approval unless a separately approved automation requirement applies.

Postconditions

Comparison evidence is saved according to data policy; improvements are reported only where measured.

UC-20 — Measure Network Boost recommendations



UC-21 — Maintain the authorized endpoint catalogue

Goal: An administrator maintains authorized endpoint catalogue records.
Primary actor: Authorized operations administrator
Preconditions: The application is available; configuration and permissions required for the selected path are satisfied. Purpose-specific consent is checked before optional processing.
Trigger: Admin opens endpoint operations.
Related requirements: REQ-AUTH-003, REQ-API-001, REQ-API-002, REQ-OPS-001, REQ-SEC-001, REQ-OBS-001, REQ-CM-001

Main success flow

The identity service authenticates the administrator with the configured strong authentication policy.

The server authorizes the requested action against a role/permission matrix.

The administrator creates/updates/disables endpoint metadata and source/configuration-version references.

The service validates schema, region, ownership, duplicate identifiers and required health fields.

The change is recorded in an audit event and goes through any defined approval/publish step.

Published metadata is versioned and visible to client catalogue queries.

Alternate and failure flows

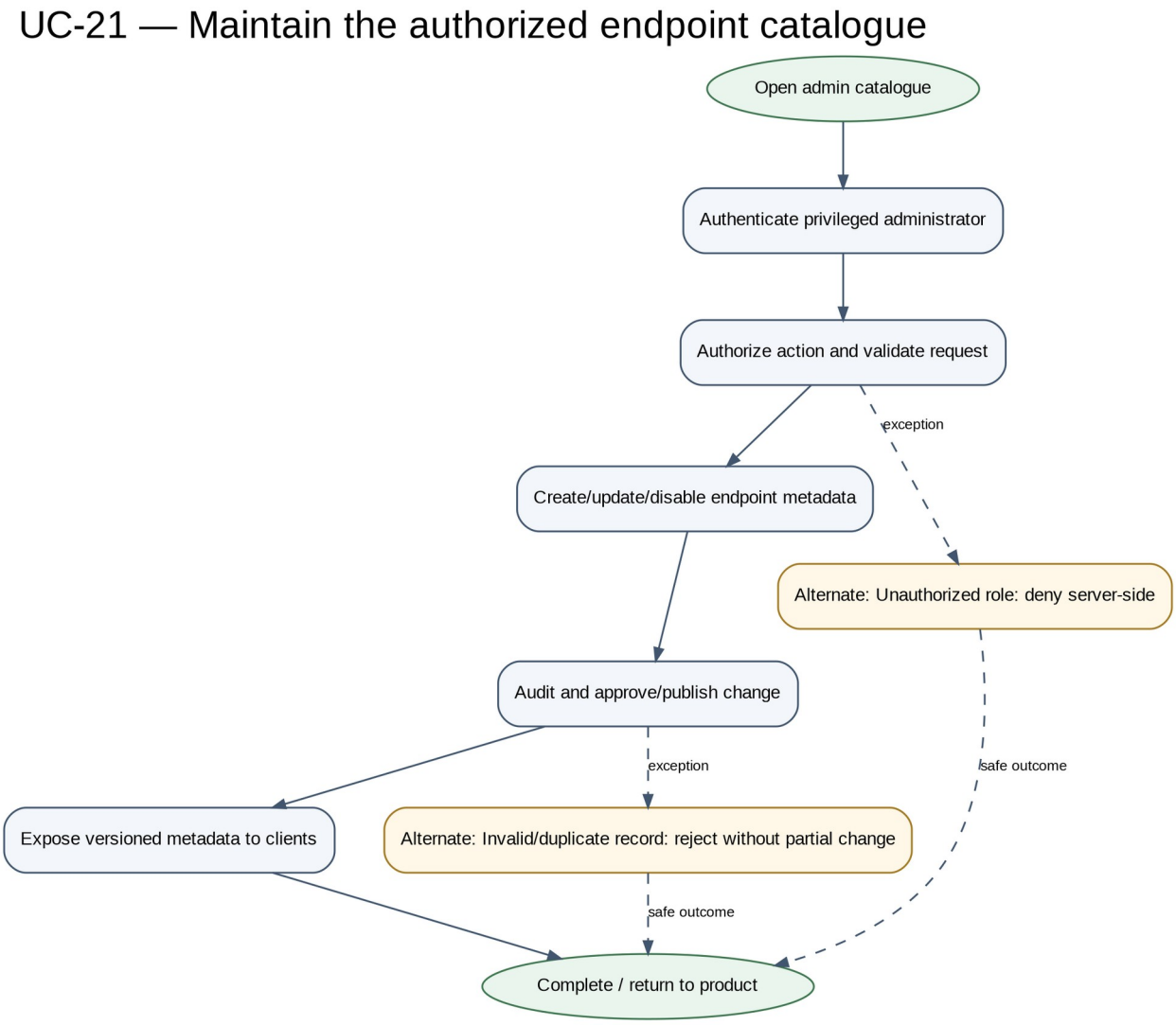
Unauthorized role/direct API request is denied regardless of UI routing.

Invalid or duplicate metadata is rejected without partial mutation.

An endpoint is disabled/withdrawn; clients receive the correct non-available state.

Postconditions

Catalogue mutation is authorized, traceable and consistent; no user traffic or private key is stored in catalogue records.



UC-22 — Monitor service health, capacity and cost

Goal: The service team monitors service health, resource capacity and cost.
Primary actor: Operations administrator / monitoring scheduler
Preconditions: The application is available; configuration and permissions required for the selected path are satisfied. Purpose-specific consent is checked before optional processing.
Trigger: Scheduled monitoring, alert threshold or operator query.
Related requirements: REQ-OPS-002, REQ-OPS-003, REQ-OBS-001, REQ-AVAIL-002, REQ-DATA-001

Main success flow

The monitoring system collects approved service/endpoint health, API error/latency, resource and traffic/billing metrics.

It tags metrics with source, observation time and availability.

Policy evaluates thresholds and checks budget/transfer ceilings.

An alert is issued to authorized operators with a safe summary and relevant correlation ID.

An operator investigates or executes bounded approved mitigation.

The incident and outcome are recorded without sensitive traffic payloads.

Alternate and failure flows

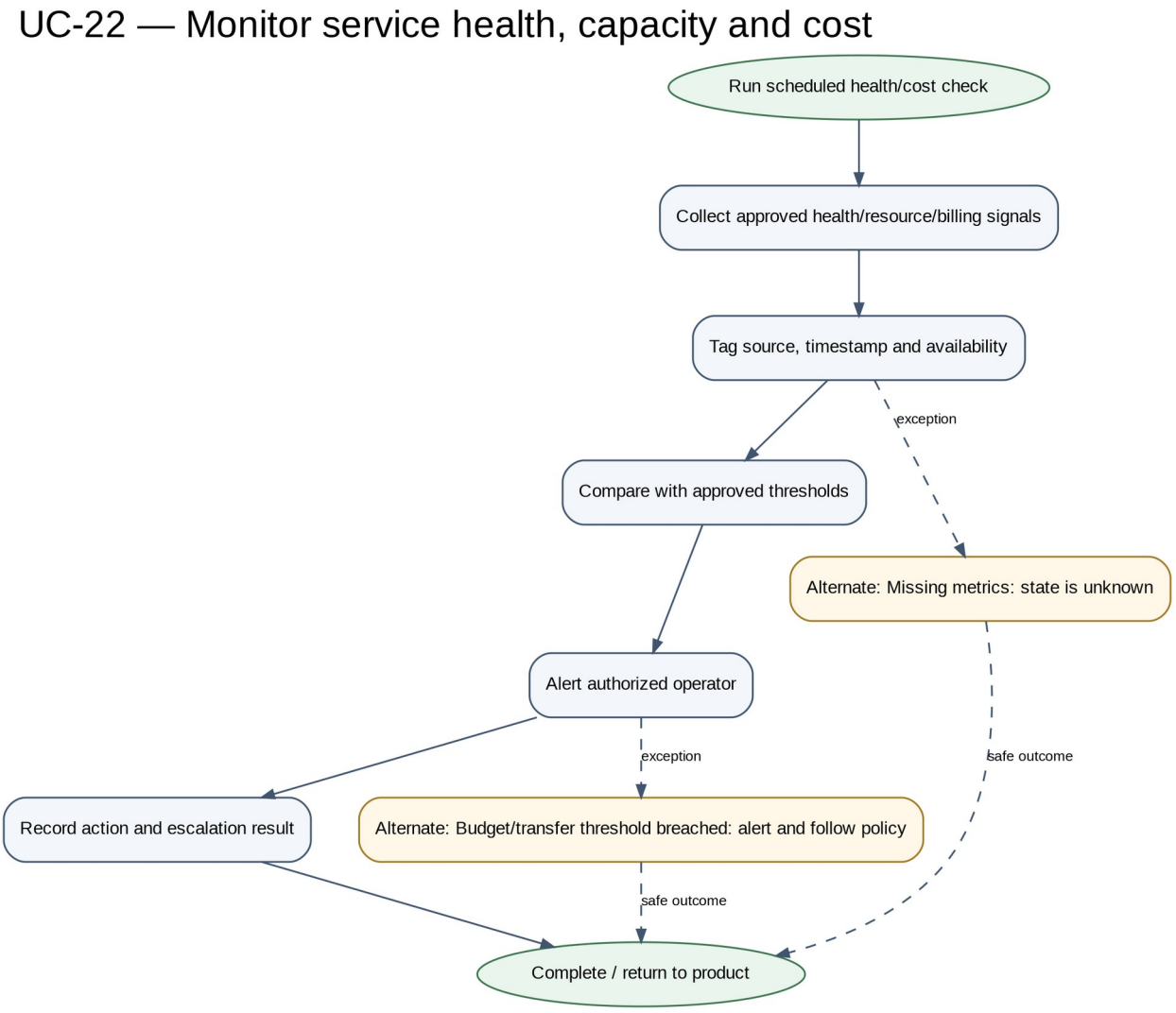
Metrics are missing; state is unknown rather than healthy.

Threshold breaches trigger the configured alert/escalation; any automatic restriction follows approved operational policy.

Monitoring integration is unavailable; local/provider alerts and an operational fallback are documented.

Postconditions

Health/cost state and action history are visible to authorized operations staff; user payload content is not recorded.



UC-23 — Evaluate, promote and roll back an AI model

Goal: The operator evaluates a candidate AI model and safely promotes or rolls it back.
Primary actor: AI/model operator
Preconditions: The application is available; configuration and permissions required for the selected path are satisfied. Purpose-specific consent is checked before optional processing.
Trigger: A candidate model or rule-set version is submitted.
Related requirements: REQ-ML-001, REQ-ML-002, REQ-AI-002, REQ-AI-003, REQ-OPS-002, REQ-CM-001

Main success flow

The pipeline validates model identity, intended task, data lineage, licensing/consent and evaluation split metadata.

The candidate is compared with the approved baseline on a versioned, held-out evaluation suite.

Quality, latency, robustness, privacy, security and justified bias/fairness checks are reviewed.

The release gate records approval, model hash/version, metrics, limits and rollback target.

A limited rollout monitors agreed metrics and error/insufficient-data rates.

If a trigger is met, traffic is returned to the last approved version or the feature is disabled.

Alternate and failure flows

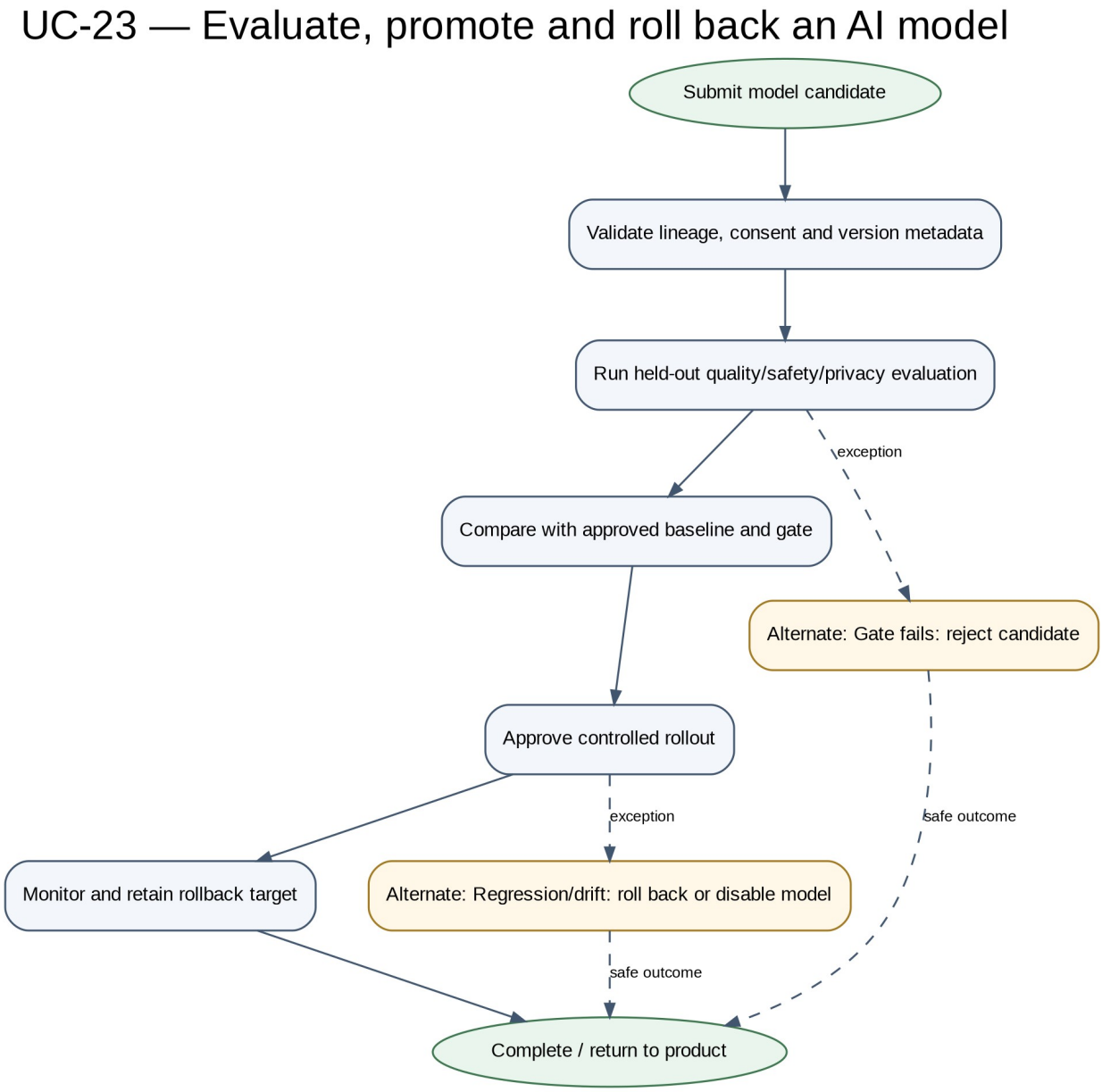
Candidate fails a threshold or data-governance check; it is rejected with a recorded rationale.

Production drift or safety incident triggers rollback and incident review.

No trained model is approved; the explainable rule baseline remains active.

Postconditions

Only an approved version is served; previous approved version remains recoverable and no claim exceeds measured evidence.



UC-24 — Request support using redacted diagnostics

Goal: The user shares selected diagnostics to request support.
Primary actor: End user / support agent
Preconditions: The application is available; configuration and permissions required for the selected path are satisfied. Purpose-specific consent is checked before optional processing.
Trigger: User opens help/support after an issue.
Related requirements: REQ-AI-001, REQ-AI-005, REQ-SEC-001, REQ-SEC-003, REQ-OBS-001, REQ-PRIV-002, REQ-DATA-001

Main success flow

The app explains the purpose and exact fields proposed for sharing.

It redacts secrets, raw profiles, browsing content, unnecessary identifiers and sensitive payloads.

The user reviews the payload and explicitly consents to send it.

The support service receives a minimal package with version/platform/error correlation metadata.

The user receives a case reference/status and can withdraw or request deletion where applicable.

Alternate and failure flows

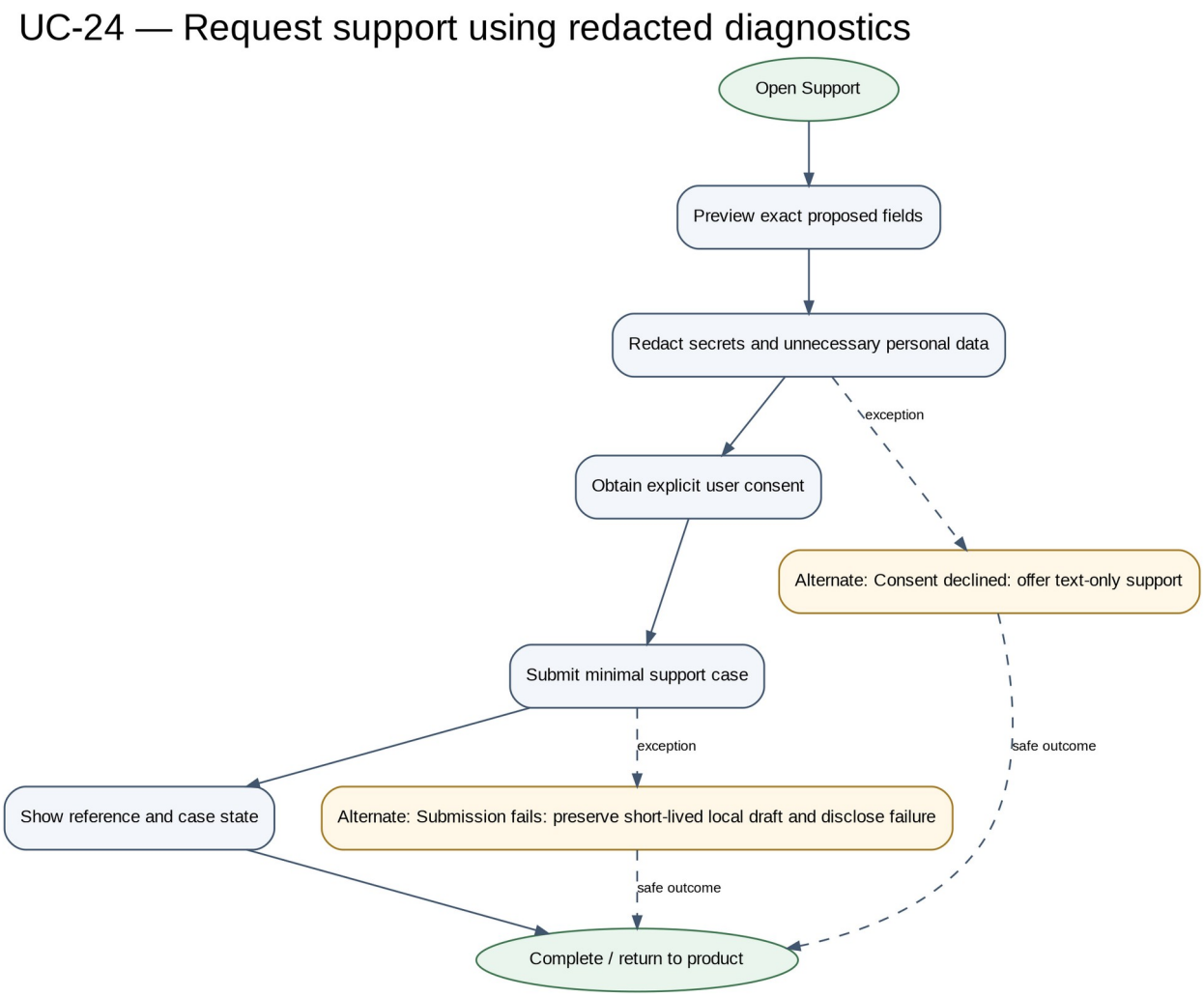
Consent is declined; a text-only support request or local help remains available.

Redaction cannot guarantee safe content; submission is blocked and manual review is requested.

Service failure is shown accurately and an unsubmitted draft expires per policy.

Postconditions

Only the reviewed, consented data is sent; support access is controlled and auditable.



6. Quality attributes and service objectives

6.1 Performance and resource use

Performance objectives shall be established per supported platform and workload before release acceptance. They shall include, as applicable, connection-state update latency, recommendation response latency, AI response latency, diagnostic completion time, API latency, resource use and background polling. Reports shall state sample count, workload, device/OS, network, percentile or maximum, errors/timeouts and measurement limitations. No fixed improvement such as “10% faster” or a battery/data saving percentage may be advertised without controlled, reproducible measurements.

6.2 Security and privacy

The product shall apply least privilege, server-side authorization, secure session handling, protected local secret storage, encrypted transport, input validation, rate limiting, dependency/license review, secret rotation, redacted logs and secure configuration management. Private keys, full tunnel profiles, access tokens, browsing content and packet payloads shall not enter application APIs, cloud documents, AI prompts, support exports, screenshots or ordinary logs. Diagnostic collection shall be purpose-limited and minimized. Security controls shall be reviewed against the deployed threat model before public launch.

6.3 Reliability and recovery

Connection state must reflect the authoritative platform adapter. Remote APIs, AI providers, catalogue endpoints and notification services are optional dependencies for the features that consume them; outages must be isolated with bounded timeouts/retries and truthful user feedback. Recovery behavior, data retention, backup restoration and endpoint/node replacement shall be described in operational runbooks and exercised before release.

6.4 Availability, capacity and operations

Availability objectives and recovery time/objectives shall be set after deployment topology and service criticality are approved. Health probes shall distinguish healthy, unhealthy and unknown. Production operations shall monitor API/tunnel service health, traffic and budget ceilings and provide an incident path. A server count alone does not establish user capacity or availability; capacity and failover are subject to load and resilience tests.

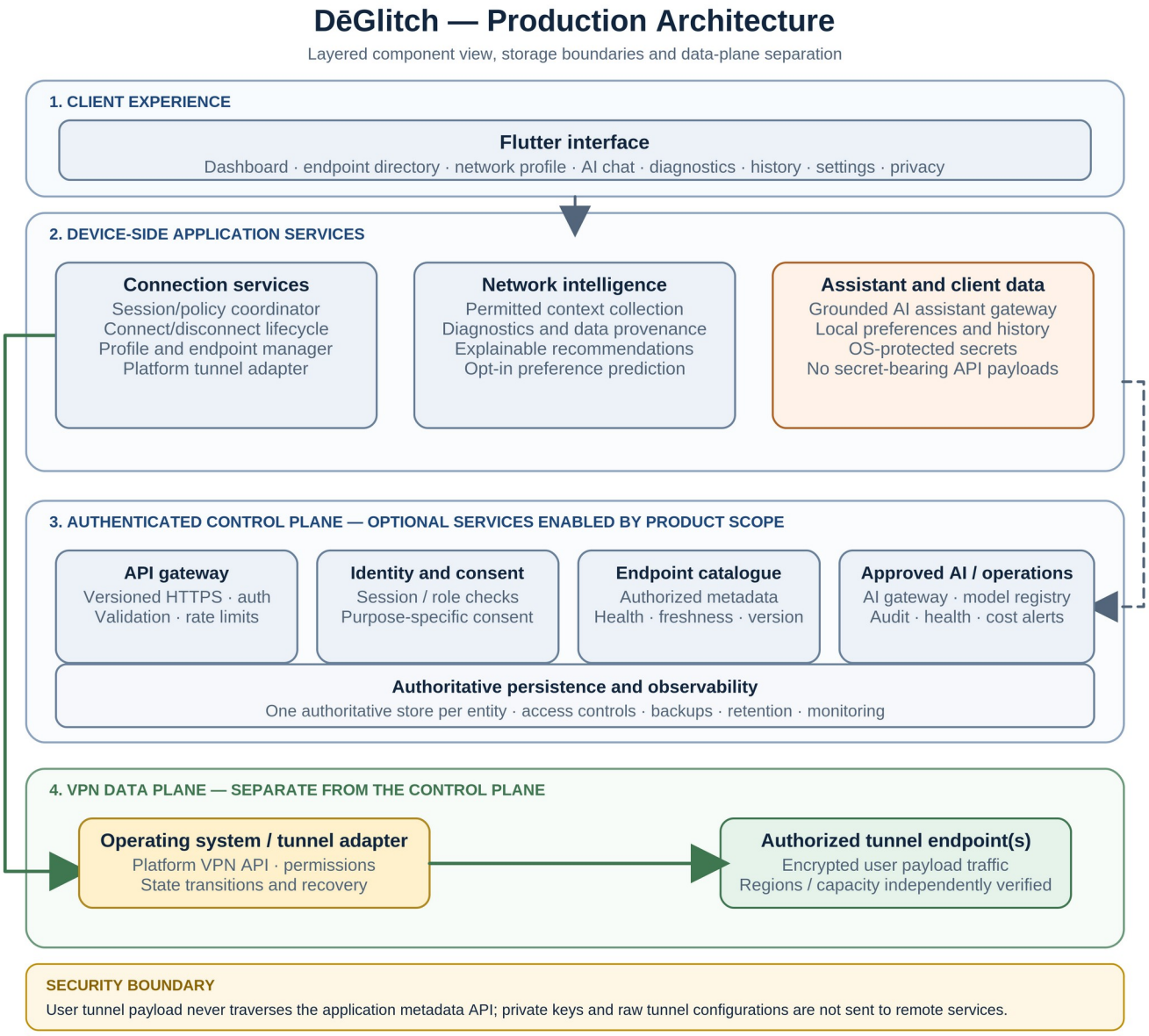
6.5 Portability, accessibility and localization

The release matrix shall identify verified OS/device combinations and feature-specific limitations. Critical flows shall be tested for screen-reader semantics, text scaling, contrast, keyboard navigation where applicable and reduced motion. English, Chinese, Spanish, French, Russian and Krio are intended product locales; only reviewed locales may be claimed as production-supported for a specific release.

7. System architecture and external interfaces

7.1 Architecture overview

The proposed production architecture separates the device/client domain, the control plane and the VPN data plane. The device owns the connection lifecycle and local secrets. The control plane manages authenticated identity/consent, endpoint catalogue metadata, approved recommendation/support interfaces and operational metadata. The VPN data plane transports user traffic through the configured authorized tunnel endpoint; it is never routed through the application metadata API.



Layered production architecture showing the client experience, device-side services, optional authenticated control plane, persistence, and separate VPN data plane.

7.2 Logical components and responsibilities

Component	Responsibility	Primary related requirements
Flutter presentation layer	Dashboard, endpoint directory, network profile, chat, diagnostics, settings, consent and truthful loading/error/empty states.	REQ-FUNC-001..010, REQ-NPROF-001/002, REQ-AI-001..005, REQ-UX-001, REQ-I18N-001
Session and policy coordinator	Orchestrates use cases, verifies user intent and consent, enforces safe UI-to-service boundaries.	REQ-AUTH-001..003, REQ-AI-003, REQ-PRIV-002
Connection coordinator	Serializes requests and maps adapter state to user-visible state transitions.	REQ-FUNC-001..003, REQ-REL-001
Local profile manager	Validates profile metadata, references OS-protected profile secrets and maps catalogue entries to configured profiles.	REQ-INT-003, REQ-FUNC-010, REQ-SEC-001/002
Platform network/tunnel adapter	Encapsulates each OS API, permission flow, connect/disconnect and lifecycle feedback.	REQ-INT-001/002, REQ-COMP-002, REQ-PORT-001
Context and diagnostics service	Collects permitted indicators, probe measurements and provenance fields.	REQ-FUNC-005/009, REQ-NPROF-001, REQ-SEC-003, REQ-OBS-001
Recommendation service	Uses versioned, explainable rules first; returns reasons or insufficient-data status.	REQ-FUNC-006/007, REQ-PREF-001..003, REQ-PERF-001, REQ-ML-001/002
AI assistant gateway	Applies consent, context allowlisting, source grounding, safety checks, provider timeouts and output formatting.	REQ-AI-001..005, REQ-PERF-003, REQ-AVAIL-002
Local data boundary	Stores settings, non-secret profile metadata and optional local history with documented lifecycle.	REQ-INT-003, REQ-FUNC-010, REQ-DATA-001
API gateway and control-plane services	Authenticated/versioned APIs for endpoint metadata, identity support, data requests, notifications and operations.	REQ-INT-004, REQ-API-001/002, REQ-AUTH-003, REQ-OPS-001..003
Persistence layer	PostgreSQL for relational catalogue/operations where selected; optional Firestore for approved document-style data; schemas and authority are defined per store.	REQ-API-001, REQ-DATA-001
Monitoring and release pipeline	Health/cost alerts, audit records, model registry, rollout gates and rollback.	REQ-OPS-002/003, REQ-ML-002, REQ-CM-001

7.3 Runtime and trust boundaries

Device trust boundary: local UI and domain services consume OS-provided network/tunnel state. Sensitive profile material is referenced or stored through platform-protected facilities; application-level databases contain no raw private keys.

Control-plane trust boundary: all remote requests use authenticated and authorized interfaces. The server validates ownership/roles independently of client navigation. Endpoint metadata and approved operational fields only are exposed to clients.

AI-provider boundary: a separate data minimization and consent gate filters context before any remote inference. The assistant does not receive raw tunnel profiles, private keys, complete browsing history or packet payloads.

VPN data-plane boundary: encrypted tunnel payload traffic follows the authorized tunnel adapter to the configured endpoint. It does not pass through the control-plane API, AI service or general-purpose support pipeline.

Operations boundary: privileged endpoint/model changes require authorization, audit evidence and release/rollback rules. Monitoring captures safe metadata, not browsing content.

7.4 API contract principles

The public or internal control-plane API, if selected for deployment, shall define versioned routes, authentication, authorization, request/response schemas, pagination/limits, idempotency for mutation where necessary, error codes, timeouts, rate limits, observability and backward compatibility. Proposed logical operations include:

Logical operation	Purpose	Required controls
GET /v1/endpoints	Filter eligible catalogue metadata.	Auth/publication policy, pagination, source/freshness and explicit unavailable state.
GET /v1/endpoints/{id}	Retrieve a single endpoint description.	Object-level authorization where applicable; no secret profile payload.

Logical operation	Purpose	Required controls
GET /v1/health/summary	Return approved health/status metadata.	Role/scope restriction, observation time and unknown values for missing data.
POST /v1/recommendations	Optional remote recommendation if selected.	Minimized allowlisted context, schema version, consent gate and rate limiting.
POST /v1/assistant/messages	Optional remote AI assistance.	Minimized prompt context, secret filtering, timeout, moderation/safety and retention policy.
GET/POST /v1/privacy/requests	Create or inspect data export/deletion requests.	Authenticated owner check, idempotency and auditable state.
POST /v1/support/cases	Submit consented redacted support content.	User preview/consent, redaction and size limits.
POST/PATCH /v1/admin/endpoints	Authorized catalogue management.	Strong authentication, least privilege, validation and audit.

The route names are a logical contract proposal and shall be versioned and approved during interface design before production integration. They are not a claim that an API has already been deployed.

7.5 Firebase and relational storage decision

Firebase can support a Flutter product through Authentication, Cloud Firestore, Cloud Functions, App Check and Cloud Messaging when those services meet the selected release requirements. Cloud Firestore is document-oriented NoSQL, not a relational database; an ERD used as the conceptual model must map to collections/references, deliberate denormalization, security rules, indexes and trusted server-side integrity checks. Firebase does not provide the VPN edge servers themselves.

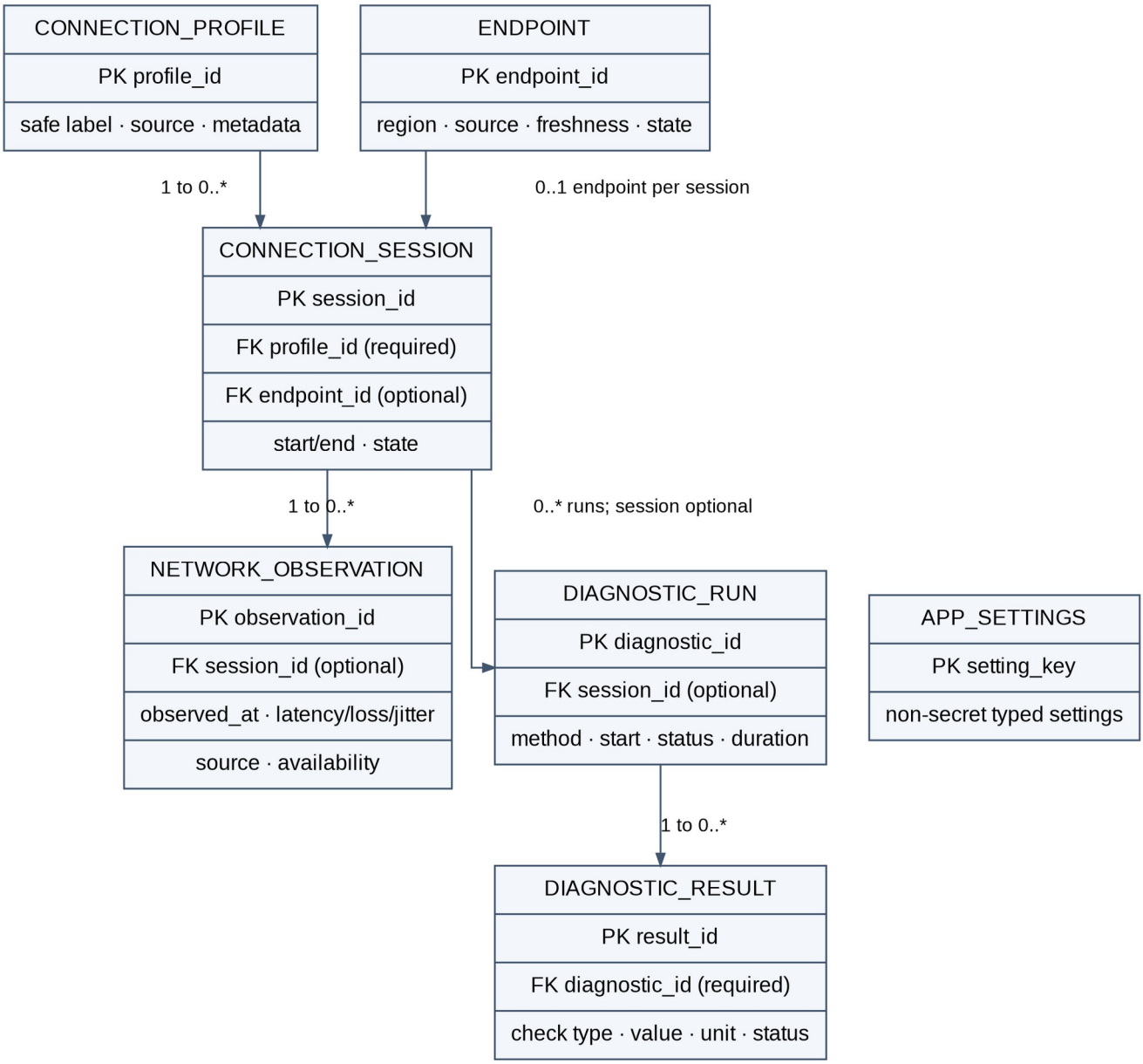
For production catalogue and operational data requiring foreign keys, uniqueness, joins and transactionally consistent relationships, the v1.2 design recommends PostgreSQL as the authoritative relational store behind a versioned API. A Firebase-only MVP may be selected if its data model, authorization rules, query/index pattern, backup, cost and data lifecycle are designed and tested. Do not duplicate authoritative fields across stores without defining sync direction, conflict ownership and reconciliation. The project shall record one authoritative store per field/entity before implementation.

8. Data architecture, ERD and AI contracts

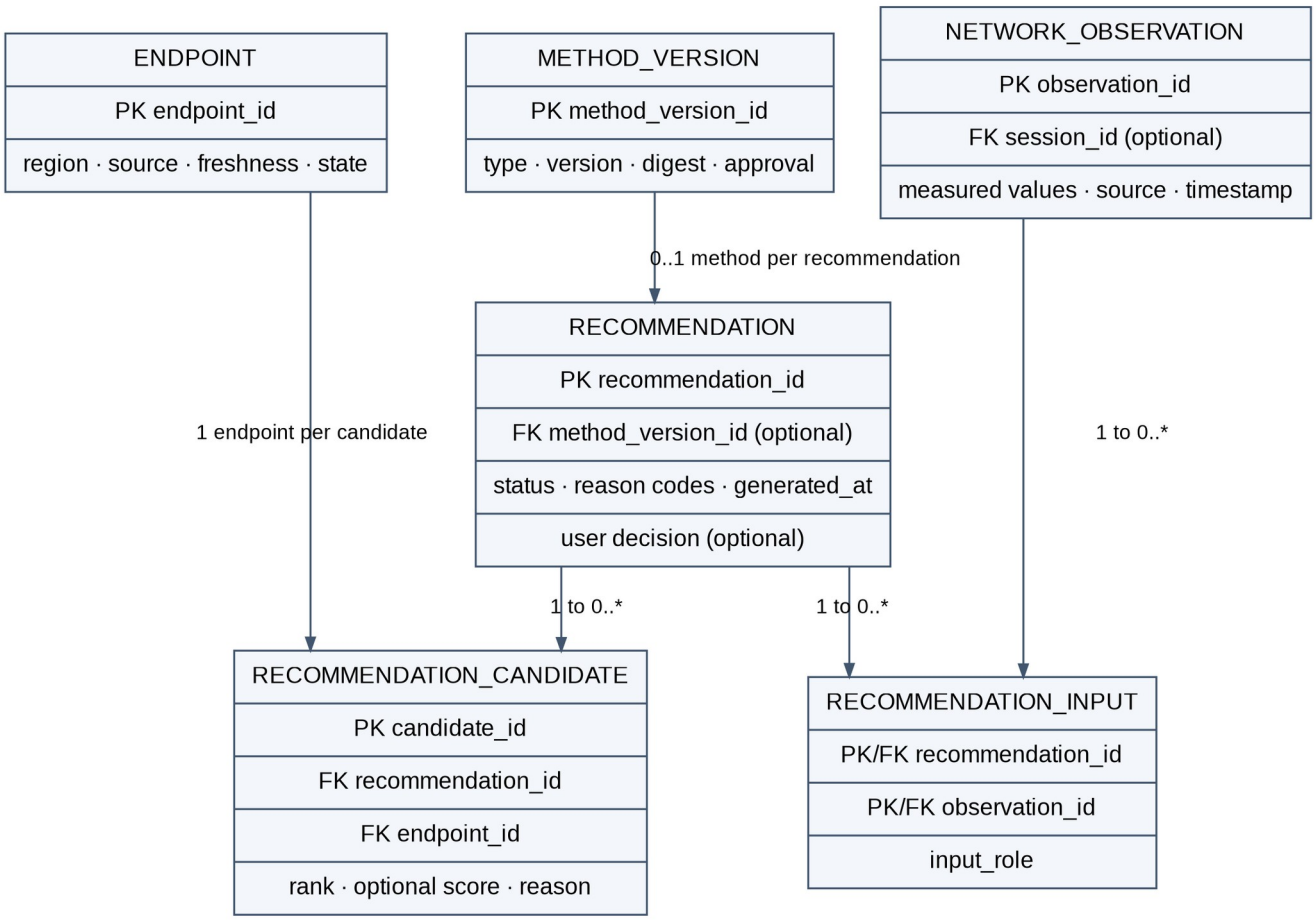
8.1 Core connectivity and recommendation ERD

The core logical data model retains the 11 entities from v1.2 and their relationship constraints. This model is relational in concept; implementation in Firestore requires explicit collection/document mapping rather than assuming SQL foreign keys.

Core connectivity and diagnostics ERD (view 1 of 2)



Recommendation lineage and measurement ERD (view 2 of 2)



Entity	Key fields and integrity rule
APP_SETTINGS	Primary key setting_key; stores non-secret typed application settings.
CONNECTION_PROFILE	Primary key profile_id; stores safe labels/source/metadata, never private keys or raw profile text.
ENDPOINT	Primary key endpoint_id; stores region, label, authorized source, freshness and availability state.
CONNECTION_SESSION	Primary key session_id; mandatory profile reference, nullable endpoint reference where a local profile cannot be mapped to catalogue metadata; start/end/state.
NETWORK_OBSERVATION	Primary key observation_id; optional session reference; observed-at timestamp, measurement values, source and availability.
DIAGNOSTIC_RUN	Primary key diagnostic_id; optional session reference; method, start time, status and duration.
DIAGNOSTIC_RESULT	Primary key result_id; mandatory diagnostic reference; check type, value, unit and status.
METHOD_VERSION	Primary key method_version_id; method type, version, configuration digest and approval metadata for a ruleset or approved model.
RECOMMENDATION	Primary key recommendation_id; nullable method version when not tracked; status, reason codes, generation time and optional user decision.
RECOMMENDATION_CANDIDATE	Primary key candidate_id; mandatory recommendation and endpoint references; rank, optional supported score and reason code; unique recommendation/endpoint pair.
RECOMMENDATION_INPUT	Composite primary/foreign key (recommendation_id, observation_id); records which measurements supported a recommendation and their role.

Cardinality and integrity rules

One connection profile may be referenced by zero or many sessions; each retained session must point to exactly one profile.

An endpoint may be referenced by zero or many sessions; a session's endpoint association is optional when no reliable catalogue mapping exists.

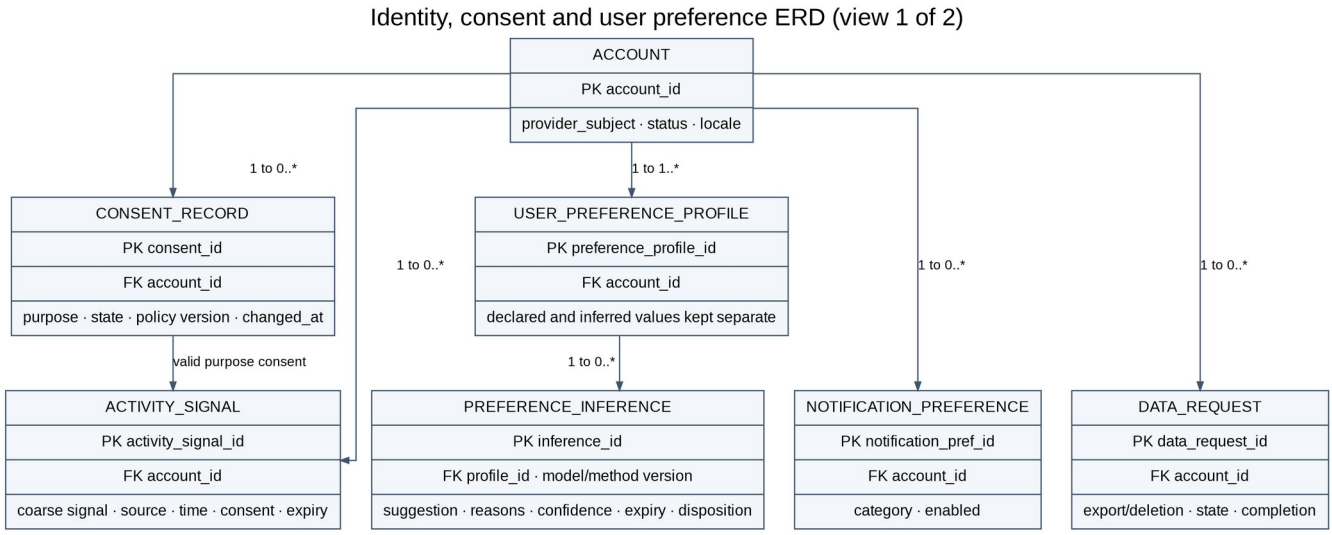
A session may have zero or many observations and zero or many diagnostic runs. Observations and diagnostics may also exist without a session.

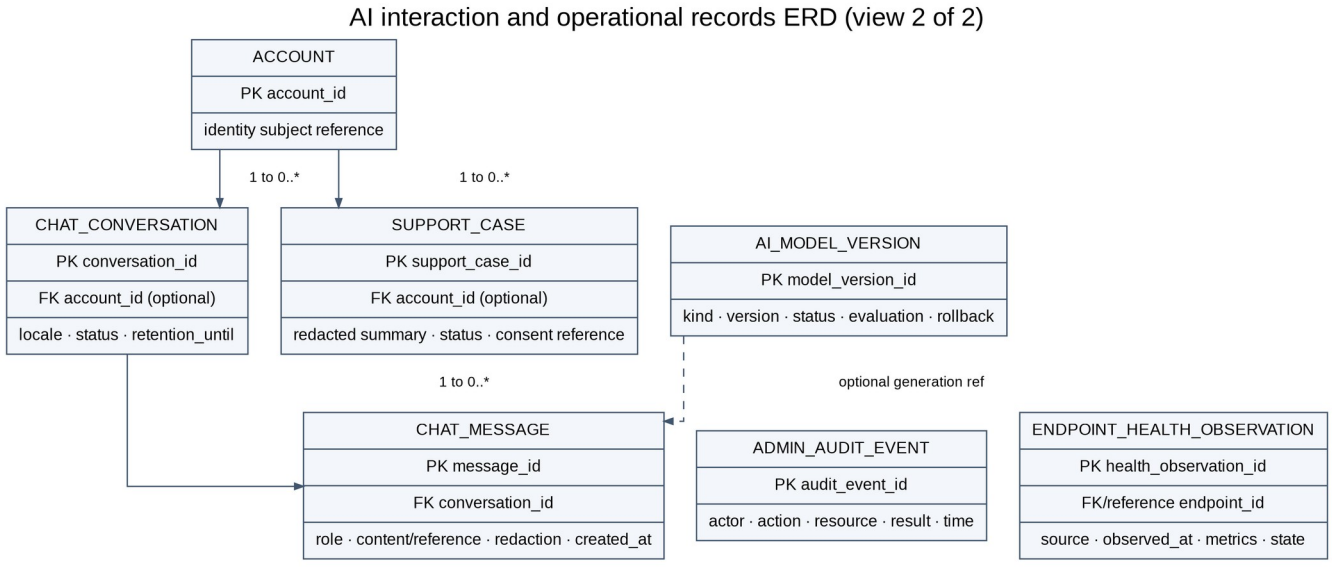
Each diagnostic result belongs to one diagnostic run; a failed run may contain zero results and a summarized failure state.

A method version can be used by many recommendations; a recommendation can have zero or many candidate endpoints, including zero candidates for an insufficient-data/no-candidate result.

Recommendation-input pairs are unique. IDs are immutable; timestamps use UTC; unavailable metrics are null with explicit source/availability state rather than invented values.

8.2 Identity, consent, activity and AI data model





The personalization model extends the core ERD with the following logical entities. Actual persistence may be relational, document-oriented or split by data class; each entity shall have one authoritative store and an approved retention policy.

Entity	Purpose and constraints
ACCOUNT	Identity-provider subject reference, account state, locale and timestamps; does not store raw identity-provider passwords.
CONSENT_RECORD	Purpose-specific state, policy version and change timestamp; withdrawal is enforced at the point of processing.
USER_PREFERENCE_PROFILE	User-declared preferences and separately marked inferred suggestions; stores version and update time. Declared values are never silently overwritten.
ACTIVITY_SIGNAL	Minimized, coarse activity category/value, source, observation time, consent reference and expiry; not browsing URLs/content.
PREFERENCE_INFERENCE	Candidate result, reason codes, method version, calibrated confidence if available, expiry and user disposition.

Entity	Purpose and constraints
CHAT_CONVERSATION / CHAT_MESSAGE	User conversation metadata and messages under configured access, retention, export and deletion controls. Raw profile secrets and diagnostic payloads are excluded.
AI_MODEL_VERSION	Model/ruleset identity, version, status, evaluation reference, approval and rollback target.
NOTIFICATION_PREFERENCE	User-selected notification category and enabled state.
DATA_REQUEST	Export/deletion request type, status, timestamps and completion evidence.
SUPPORT_CASE	Redacted support summary and case state; attached diagnostic fields require explicit consent.
ADMIN_AUDIT_EVENT	Actor reference, action, target resource, outcome and safe metadata; no secrets.
ENDPOINT_HEALTH_OBSERVATION	Endpoint status, source/time and available performance measurements with explicit unknown values.

8.3 Recommendation input contract

The contract is versioned and method-agnostic. The rule-based implementation is the initial comparator; a trained model is used only after its dataset governance, evaluation and release gate are approved.

Field	Type / rule	Purpose
schema_version	Required string	Contract version and compatibility.
observed_at	Required UTC timestamp	Time the signal was measured.
source	Required enum/reference	OS adapter, authorized endpoint telemetry or explicitly labelled test fixture.
latency_ms	Number or null	Round-trip latency from a documented method.
packet_loss_pct	Number or null	Loss over a documented measurement window.
jitter_ms	Number or null	Variation, only if measurement method is available.
throughput_mbps	Number or null	Actual measured throughput only; never inferred solely from network generation.
endpoint_state	available, unavailable, unknown	Eligibility context with freshness.
user_preferences	Allowlisted enum/object	User-declared preferences and accepted inferred preferences where consent permits.
device_constraints	Optional allowlisted object	Only approved resource and platform constraints necessary for ranking.

8.4 Recommendation output contract

Field	Contract
status	ranked, insufficient_data or no_available_candidate.
ranked_endpoint_ids	Ordered eligible endpoint IDs; empty when none qualify.
method	rule_based for the default implementation, or a named model type after approval.
method_version	Stable ruleset/model version for reproducibility.
reason_codes	Supported explanations such as lower observed latency/loss; no unsupported reason code.
generated_at	UTC output timestamp.
accepted_by_user	Optional user decision; acceptance does not itself imply automatic connection.

Missing measurements remain null or explicitly unavailable. The contract shall exclude private keys, raw profile configuration, browser history/content, complete URLs, packet payloads and unrelated personal details. The UI shall display the source and observation age when freshness affects interpretation.

8.5 AI chat contract and guardrails

AI chat is intended to explain DēGlitch features, summarize the user's displayed network profile, explain recommendation factors, interpret diagnostics, guide setup and suggest troubleshooting steps. It shall not claim to have examined telemetry unless the current request passed those values into the context. A network-profile summary must distinguish measurement from inference and cannot diagnose an ISP, device fault or remote server cause without evidence.

The assistant request envelope should carry a schema version, locale, request purpose, allowlisted context references, consent/policy state, approved knowledge-source versions and a correlation ID. The response should carry answer text, supported source references, reason/limitation markers, action intent if any, generation time and model/ruleset version where applicable. A consequential action intent is only a proposal until the user confirms it through a native application control.

8.6 Predictive preferences from user activities

Preference prediction shall be a consent-based personalization function rather than covert activity surveillance. It may use only documented coarse categories such as user-selected activity profile, explicit mode choices, opted-in interactions with recommendations, or locally computed aggregates approved through the privacy review. Browsing destinations, packet payloads, message contents, exact location and unrelated device data are excluded by default. Automatic activity detection is not assumed by this specification.

Every inference shall be linked to consent purpose/version, allowed input categories, method version, generation time, expiry, reason codes and the user's accept/edit/reject response. The system shall not treat uncalibrated scores as probabilities. No adequate signals means no recommendation. The user must be able to inspect, correct, clear and disable inferred preferences. The inference must not silently change network settings, connection state or declared preferences.

9. Data governance, privacy and security

9.1 Data-class boundaries

Data class	Default location / flow	Required controls
Tunnel profile, private key, access tokens	Device / OS-protected secret storage	Never sent to API, Firebase, chat, support export, logs, repo or screenshots.
Profile label and non-secret metadata	Local store; optional approved metadata sync only if needed	Validate fields; keep secret references separate; user can update/remove.
Connection state/counters	Read from platform adapter and display locally	Mark source, timestamp and unknown state; avoid collection beyond purpose.
Network observations/diagnostics	On-demand local collection; remote only if purpose-approved	Minimal measurement fields; no browsing or packet payload.
Activity signals	Local processing by default; remote processing only after specific opt-in	Coarse allowlisted values, explicit purpose, expiry/retention and withdrawal.
Preferences/inference	Local store by default; account sync only if approved	Distinguish declared from inferred, version and source, expiry and user control.
Chat conversation	Local or approved provider/service according to user choice and release policy	Prompt minimization, access isolation, retention/deletion, no profile secrets.
Endpoint catalogue/health	Authorized metadata control plane	Provenance, cache age, admin authorization and audit.
Operational logs/audit	Restricted operations store	Safe identifiers, redaction, retention and least privilege.
Export/support package	User-reviewed and consented channel	Redaction, exact field preview, transport protection and deletion policy.

9.2 Data lifecycle

Before a data store or provider integration is enabled, its owner shall document collection purpose, basis/consent, fields, source, destination, access roles, retention period, deletion/expiry behavior, backup propagation, export behavior and operational audit trail. Local connection history remains on-device by default. Account sync or remote access to

history/chat/activity requires an explicit requirement, consent and deletion design. Backup retention exceptions shall be communicated accurately rather than promising immediate physical erasure where the service cannot guarantee it.

9.3 Threat and abuse controls

The deployed threat model shall address account takeover, unauthorized direct API access, privilege escalation, endpoint catalogue poisoning, malicious/malformed profile files, secret leakage, prompt injection, AI hallucination, consent bypass, activity-profile inference abuse, data deletion gaps, provider compromise, rate abuse, denial of service, cost exhaustion and endpoint/server compromise. Controls include server-side authorization, schema validation, allowlisting, output/source checks, bounded retries, secret management, MFA for privileged operators, dependency scanning, patching, audit trails, alerting, backup/restore drills and incident response. Controls are release acceptance items, not aspirational statements.

9.4 Privacy impact and AI governance

Before enabling remote AI, activity inference, or stored conversation history, the product owner shall approve a purpose and data inventory, minimization, consent UX, retention, provider terms, access boundaries, failure behavior, evaluation set and a user-facing explanation of limitations. AI outputs shall not replace platform-authoritative state. Sensitive inferences shall not be made from unrelated activity data. Model versions shall be evaluated before rollout and monitored for degradation; the last approved version or rule-based baseline remains the fallback.

10. Verification and acceptance evidence

10.1 Verification methods and release evidence

Verification uses test, inspection, analysis, measurement, resilience testing and demonstration as appropriate. Every record shall contain evidence ID, related requirement IDs, use case/test, SRS version, source commit/build, platform/device/OS, configuration reference without secrets, preconditions, input data, expected and observed results, outcome, date, tester and evidence location. Implemented, tested and verified for release are separate lifecycle states.

10.2 Test catalogue

The verification catalogue retains the original test identifiers and adds coverage for account/session flows, consent/data rights, profile explanation, AI chat, preference prediction, endpoint search, localization, operations and model release. Every permanent requirement maps to a verification case or to an explicitly justified inspection/analysis item in Requirement_Traceability_Matrix.md.

Test ID	Scenario	Requirement coverage	Test procedure
TC-01	Connection-state mapping	REQ-FUNC-001, REQ-INT-001, REQ-INT-002	Adapter reports each supported state.
TC-02	Valid connect request	REQ-FUNC-002, REQ-SEC-002	Connect with a valid authorized profile.
TC-03	OS VPN permission denial	REQ-FUNC-003, REQ-COMP-002	Deny OS VPN permission.
TC-04	Missing/malformed profile	REQ-FUNC-003, REQ-SEC-002	Connect with missing or malformed profile.
TC-05	Tunnel fails or stops	REQ-REL-001	Force adapter/tunnel failure.
TC-06	Valid profile import	REQ-INT-003, REQ-SEC-001	Import valid profile.
TC-07	Invalid profile import	REQ-FUNC-010, REQ-SEC-002	Import invalid profile.
TC-08	Secret leakage inspection	REQ-SEC-001, REQ-OBS-001	Inspect logs/repository after profile operations.
TC-09	Eligible endpoint listing	REQ-FUNC-004	Endpoint source returns eligible entries.
TC-10	Endpoint-source outage	REQ-INT-004, REQ-AVAIL-001	Make endpoint source unavailable.
TC-11	Endpoint without profile mapping	REQ-SEC-002	Select endpoint with no authorized profile mapping.
TC-12	Reproducible recommendation	REQ-FUNC-005, REQ-FUNC-006	Run reviewed measurement fixture scenarios.
TC-13	Recommendation with missing context	REQ-FUNC-006	Remove required context inputs.

Test ID	Scenario	Requirement coverage	Test procedure
TC-14	Human control over recommendation	REQ-FUNC-007	Display recommendation while automation is disabled.
TC-15	Model not configured	REQ-ML-001	No approved trained model is configured.
TC-16	On-demand diagnostics	REQ-FUNC-009	Run supported diagnostics.
TC-17	Diagnostic privacy inspection	REQ-SEC-003, REQ-OBS-001	Inspect diagnostic payloads and logs.
TC-18	Diagnostic dependency failure	REQ-AVAIL-001	Fail an optional diagnostic service.
TC-19	Lite Mode behavior	REQ-FUNC-008	Toggle Lite Mode on and off.
TC-20	Lite Mode resource comparison	REQ-PERF-002	Measure matched Lite Mode-on/off runs.
TC-21	History disabled	REQ-FUNC-010, REQ-INT-003	Disable connection history.
TC-22	View and clear local history	REQ-FUNC-010	View then clear history.
TC-23	Network loss while connected	REQ-REL-001, REQ-FUNC-001	Inject network loss.
TC-24	Interruption log review	REQ-OBS-001	Inspect interruption evidence.
TC-25	Optional service unavailable	REQ-AVAIL-001	Disable optional service.
TC-26	Safe remote error	REQ-SEC-001	Cause remote service error.
TC-27	End-to-end platform connection	REQ-POC-001	Exercise authorized connection lifecycle on a named platform.
TC-28	Clean build/install reproduction	REQ-INST-001, REQ-BUILD-001	Build and install from documented prerequisites.
TC-29	Dependency/license/permission review	REQ-COMP-001, REQ-COMP-002	Inspect release dependencies and platform permissions.
TC-30	Supported platform matrix	REQ-PORT-001	Review claimed platform support.
TC-31	Controlled requirement change	REQ-CM-001, REQ-MAINT-001	Change a requirement through change control.
TC-32	Recommendation latency measurement	REQ-PERF-001	Measure recommendation response time.
TC-33	Sign-up/sign-in with valid and invalid provider responses	REQ-AUTH-001	Execute the sign-up/sign-in with valid and invalid provider responses scenario with controlled test inputs; capture actual result.
TC-34	Expired/revoked session, recovery and sign-out	REQ-AUTH-002	Execute the expired/revoked session, recovery and sign-out scenario with controlled test inputs; capture actual result.
TC-35	Direct API call by wrong role/owner	REQ-AUTH-003, REQ-API-002	Execute the direct api call by wrong role/owner scenario with controlled test inputs; capture actual result.
TC-36	Onboarding permission decline	REQ-ONB-001	Execute the onboarding permission decline scenario with controlled test inputs; capture actual result.
TC-37	Network profile inputs missing or stale	REQ-NPROF-001	Execute the network profile inputs missing or stale scenario with controlled test inputs; capture actual result.

Test ID	Scenario	Requirement coverage	Test procedure
TC-38	Explain measured versus inferred profile factor	REQ-NPROF-002, REQ-AI-002	Execute the explain measured versus inferred profile factor scenario with controlled test inputs; capture actual result.
TC-39	Change/reset declared preferences	REQ-PREF-001	Execute the change/reset declared preferences scenario with controlled test inputs; capture actual result.
TC-40	Preference inference without consent or enough data	REQ-PREF-002	Execute the preference inference without consent or enough data scenario with controlled test inputs; capture actual result.
TC-41	Reject suggestion and withdraw consent	REQ-PREF-003, REQ-PRIV-002	Execute the reject suggestion and withdraw consent scenario with controlled test inputs; capture actual result.
TC-42	AI answers supported product/network question	REQ-AI-001	Execute the ai answers supported product/network question scenario with controlled test inputs; capture actual result.
TC-43	AI asked to fabricate unavailable latency or claim guaranteed speed	REQ-AI-002	Execute the ai asked to fabricate unavailable latency or claim guaranteed speed scenario with controlled test inputs; capture actual result.
TC-44	Chat proposes connect/change setting without confirmation	REQ-AI-003	Execute the chat proposes connect/change setting without confirmation scenario with controlled test inputs; capture actual result.
TC-45	AI locale mismatch/fallback	REQ-AI-004, REQ-I18N-001	Execute the ai locale mismatch/fallback scenario with controlled test inputs; capture actual result.
TC-46	Chat history access, retention and deletion	REQ-AI-005, REQ-DATA-001	Execute the chat history access, retention and deletion scenario with controlled test inputs; capture actual result.
TC-47	Search/filter/favorite endpoint and unavailable list	REQ-ENDPT-001	Execute the search/filter/favorite endpoint and unavailable list scenario with controlled test inputs; capture actual result.
TC-48	Live, cached, stale and fixture endpoint-health fields	REQ-ENDPT-002	Execute the live, cached, stale and fixture endpoint-health fields scenario with controlled test inputs; capture actual result.
TC-49	Baseline/post-measurement Network Boost	REQ-NETOPT-001	Execute the baseline/post-measurement network boost scenario with controlled test inputs; capture actual result.
TC-50	Notification category disabled / OS permission denied	REQ-NOTIF-001	Execute the notification category disabled / os permission denied scenario with controlled test inputs; capture actual result.
TC-51	Locale matrix, text expansion and fallback	REQ-I18N-001	Execute the locale matrix, text expansion and fallback scenario with controlled test inputs; capture actual result.

Test ID	Scenario	Requirement coverage	Test procedure
TC-52	Screen reader, text scaling, keyboard and reduced motion	REQ-UX-001	Execute the screen reader, text scaling, keyboard and reduced motion scenario with controlled test inputs; capture actual result.
TC-53	Export and deletion across all registered stores	REQ-PRIV-001, REQ-DATA-001	Execute the export and deletion across all registered stores scenario with controlled test inputs; capture actual result.
TC-54	Consent purpose withdrawn during feature use	REQ-PRIV-002	Execute the consent purpose withdrawn during feature use scenario with controlled test inputs; capture actual result.
TC-55	API schema/version/auth/error contract tests	REQ-API-001	Execute the api schema/version/auth/error contract tests scenario with controlled test inputs; capture actual result.
TC-56	Malformed payload, unauthorized object ID, rate abuse	REQ-API-002	Execute the malformed payload, unauthorized object id, rate abuse scenario with controlled test inputs; capture actual result.
TC-57	Endpoint admin permission, mutation and audit	REQ-OPS-001	Execute the endpoint admin permission, mutation and audit scenario with controlled test inputs; capture actual result.
TC-58	Health alarm and missing metrics	REQ-OPS-002	Execute the health alarm and missing metrics scenario with controlled test inputs; capture actual result.
TC-59	Billing/transfer threshold alarms	REQ-OPS-003	Execute the billing/transfer threshold alarms scenario with controlled test inputs; capture actual result.
TC-60	Candidate model fails quality/safety gate; rollback	REQ-ML-002	Execute the candidate model fails quality/safety gate; rollback scenario with controlled test inputs; capture actual result.
TC-61	AI latency/timeouts against approved objective	REQ-PERF-003	Execute the ai latency/timeouts against approved objective scenario with controlled test inputs; capture actual result.
TC-62	Remote dependency timeouts and recovery	REQ-AVAIL-002	Execute the remote dependency timeouts and recovery scenario with controlled test inputs; capture actual result.
TC-63	Lifecycle audit for local, cloud, log, backup and AI data	REQ-DATA-001	Execute the lifecycle audit for local, cloud, log, backup and ai data scenario with controlled test inputs; capture actual result.
TC-64	Duplicate or delayed notification event	REQ-NOTIF-002	Replay a duplicate event ID and deliver a connection event after its validity window; capture delivery and in-app state.

Execution status: These additional scenarios are defined for implementation verification. Their outcomes shall be recorded from actual build execution; no test is marked passed merely because a requirement or screen is documented.

10.3 Evidence record template

Evidence ID: EV-YYYY-NNN
Requirement ID(s): REQ-...
Use case / test case: UC-... / TC-...
SRS version / source commit / build:
Platform / device / OS:
Configuration reference (non-secret):
Preconditions and input data:
Expected result:
Observed result:
Outcome: Pass | Fail | Blocked | Not run
Evidence link:
Privacy / secret-leakage check:
Tester / date:
Defect or corrective action:

11. Production delivery, service operations and release management

11.1 Delivery lifecycle

Stage	Principal outputs	Exit criteria
Requirements baseline	SRS, use cases, traceability, data inventory and decision log	Each permanent requirement is testable or has a recorded rationale for inspection/analysis; unresolved decisions have owners.
Architecture and detailed design	Context/component/deployment views, API contracts, ERDs, trust boundaries, interface/state models	Every critical requirement is allocated; module dependencies and trust boundaries are unambiguous.
Implementation and integration	Reproducible builds, platform adapters, services, APIs, configuration and migration scripts	Critical success and failure paths work on selected target environments with secrets protected.
Verification and hardening	Functional, security, privacy, accessibility, resilience, performance and model-evaluation evidence	Release gates pass; high/critical defects are resolved or explicitly blocked from release.
Operational readiness	Monitoring, alerts, backup/restore, incident response, abuse handling, runbooks and support procedures	On-call owner, escalation path, recovery exercise, budget/traffic alert and rollback strategy exist.
Controlled release	Release notes, supported-platform matrix, data/AI disclosures, versioned configuration and rollback reference	Only verified features/platforms are enabled; product claims match recorded evidence.

11.2 Release gates

Production enablement requires: approved scope and threat model; per-platform connection verification; server-side authorization tests; privacy/data retention review; dependency/license review; secure build/release procedure; endpoint capacity and resilience tests; AI evaluation and rollback where AI is enabled; operational monitoring and cost alerts; and tested recovery/deletion flows. Features failing a gate remain disabled or clearly unavailable rather than being presented as working.

11.3 Operational responsibilities

Define accountable owners for client release, identity/access, endpoint catalogue, network operations, data governance, AI/model lifecycle, incident handling and cost management. Each production service shall have health checks, alert routes, documented maintenance windows where applicable, incident severity policy, credential rotation, patching and recovery runbooks. Store configuration and infrastructure definitions in version control without secrets; use separate per-environment credentials and least privilege.

11.4 Capacity and production claims

Capacity shall be derived from measurement of concurrent tunnels, throughput, CPU per packet, packet loss, connection setup/recovery, egress/transfer limits and actual endpoint placement. Neither a ten-country list nor server-count assumption proves global availability, speed gains, anonymity or security. Claims are limited to measured and reviewed outcomes under stated conditions.

12. Infrastructure and cost baseline

All estimates below are carried forward from v1.2 and are planning scenarios, not binding offers. Price, availability, taxation, setup, traffic, DDoS and support terms require confirmation with current written vendor quotes before purchase. Historical spend is reported by the product owner and is not independently audited here.

12.1 Historical expenditure already reported

Item	Amount (USD)	Treatment
Windows development workstation	$2,000	Reported historical spend; not a new request.
Mac development/platform-testing computer	$4,000	Reported historical spend.
Setup, software/tools and utilities	$1,500	Reported historical spend; retain receipts/licenses where available.
Total historical spend	**$7,500**	Owner-reported amount.

12.2 Scenario A — Ten leased dedicated physical servers in ten countries

Planning country set: United States, Canada, United Kingdom, Germany, France, Poland, Italy, India, Singapore and Australia. These are planning locations, not a confirmed commercial footprint. The quoted ranges are not matched hardware offers for all regions.

Cost item	Monthly estimate	Notes
United States edge node	$100–$250	Bare-metal planning range; confirm SKU/location/traffic.
Canada edge node	$100–$250	Bare-metal planning range.
United Kingdom edge node	$100–$250	Bare-metal planning range.
Germany edge node	$100–$220	Bare-metal planning range.
France edge node	$100–$220	Bare-metal planning range.
Poland edge node	$100–$220	Bare-metal planning range.
Italy edge node	$120–$260	Bare-metal planning range.
India edge node	$120–$300	Confirm region/provider policy and bandwidth.
Singapore edge node	$150–$350	Regional pricing/traffic require quote.
Australia edge node	$150–$350	Regional pricing/traffic require quote.
Ten-node subtotal	**$1,140–$2,670**	Planning only.
Two API/control-plane VMs	$126.00	Assumption: two × $63.00/month.
Daily backups for API VMs	$37.80	Assumption: 30% backup allowance on two $63 VMs.
Managed PostgreSQL high-availability pair	$260.00	Assumption: primary + matching standby at $130/month each.
API load balancer	$12.00	HTTPS/API only; not tunnel traffic relay.
Monitoring, encrypted configuration backup, security/abuse reserve and domain	$191.25	Aggregate planning allowance.
Control-plane subtotal	**$627.05**	Separate from edge nodes.
Total before contingency	**$1,767.05–$3,297.05**	Excludes taxes, setup, extraordinary traffic and scope growth.
30% operating contingency	$530.12–$989.12	Buffer, not observed expenditure.
Planning monthly envelope	**$2,297–$4,286**	Rounded estimate.

Cost item	Monthly estimate	Notes
Planning annual run rate	**$27,566–$51,434**	Twelve times the monthly envelope.
Year-two +10% price stress scenario	$30,323–$56,577/year	Illustrative stress assumption, not a supplier price prediction.
One-time setup allowance	$0–$3,500	Modeled $0–$350/node; quote required.
One-time readiness allowance	$4,500–$12,500	Infrastructure hardening/automation, independent security assessment, and privacy/acceptable-use review.
First-year new-cash estimate	**$32,066–$67,434**	Annual run-rate envelope plus setup and readiness allowance; excludes unknown taxes/overage.
Cumulative incl. historical expenditure	**$39,566–$74,934**	Adds the reported $7,500 historical spend.

The v1.2 baseline recommends piloting two or three countries before expanding. Obtain quotes that specify physical data-centre location, exact server SKU, setup cost, bandwidth/transfer terms, egress price/TB, port speed, DDoS scope, IP charges, acceptable-use policy, provider support and exit terms. Monitor budget/transfer limits at 50%, 75% and 90%, or at thresholds approved by operations.

12.3 Scenario B — Purchase ten physical servers and colocate them

v1.2 carried a broad model for purchasing ten rack servers and placing one in each target country. Example equipment figures were ten Dell R260 units at $5,399 each ($53,990) and ten Dell R760 units at $23,199 each ($231,990). Logistics were modeled at 10–25% ($5,399–$57,998), site deployment at $5,000–$20,000, colocation/power/port at $1,500–$4,000 per month, transit/egress/DDoS at $500–$3,000+ per month, control plane around $7,525/year and spare hardware at $5,399–$23,199. The broad first-year planning envelope was approximately **$110,000–$442,000**. This is a high-uncertainty conceptual comparison rather than a vendor quote; shipping, duties, local hands, replacement logistics, network commitments and taxes could materially change it.

12.4 Cloud VM comparison

The prior baseline used example DigitalOcean CPU-Optimized virtual-machine figures of $84/month (4 vCPU/8 GiB) and $168/month (8 vCPU/16 GiB). Ten such VMs would be $840/month or $1,680/month for compute only, before load balancing, database, backups, egress, monitoring, security and contingency. VMs are not bare-metal servers and must not be represented as satisfying a requirement for one physical server in each of ten countries. Confirm current provider pricing and regional availability before use.

12.5 Procurement and cost controls

The ten-node plan is an architectural scenario, not a capacity statement. Pilot tests shall measure concurrent sessions, throughput, CPU saturation, connection/recovery behavior, packet loss, transfer usage and endpoint health before choosing production sizes. No public rollout should begin without approved traffic forecasts, acceptable-use/abuse handling, security assessment, backups, cost alerts, incident ownership and written provider terms. Firebase, if selected, is a separate service-cost line and does not include VPN edge nodes.

13. Risks, dependencies and decision register

ID	Risk / decision	Impact	Required control / evidence
D-01	Select first verified OS/tunnel implementation	OS lifecycle and permission differences affect connection correctness.	Record target adapter, device matrix and authorized endpoint; verify connect/disconnect/failure.
D-02	Decide identity provider and supported account capabilities	Identity, recovery, deletion, cost and privacy behavior depend on provider.	Approve auth/session contract, server-side role matrix and threat model.
D-03	Decide which APIs and data stores are necessary	Adds security boundaries and operational costs.	Select authoritative store for each entity and versioned API contract; document no-backend local flows.

ID	Risk / decision	Impact	Required control / evidence
D-04	Decide endpoint catalogue ownership and fleet rollout	No verified endpoints means no real catalogue/live health.	Obtain authorized sources, confirm physical locations and written provider terms.
D-05	Approve local history/chat retention and export/deletion behavior	Unclear lifecycle creates privacy risk.	Approve data inventory, purpose, retention and tested deletion paths.
D-06	Approve AI provider and allowed prompt context	Vendor processing and inference can transmit sensitive user data.	Conduct privacy/security review; minimize fields; consent and provider contract.
D-07	Approve activity-signal policy for predictive preferences	Inference may be surprising or over-collective without explicit boundaries.	Use opt-in only, coarse permitted signals, transparent reasons, expiry and user reset.
D-08	Approve AI evaluation gate and fallback	Unsupported accuracy claims or unsafe model output could harm trust.	Keep rule-based baseline; version/evaluate models; monitor and roll back.
D-09	Establish performance and availability objectives	No measurable criteria means no objective pass/fail.	Approve workload, test method, sample count/statistic and threshold before measurement.
D-10	Confirm production region, traffic policy and capacity	Availability, lawful use and costs vary by country/provider.	Written quotes, capacity tests, abuse process, resilience and cost limits.
D-11	Establish incident and support responsibilities	Security/availability incidents otherwise lack accountable handling.	Named duty owners, severity matrix, runbooks, contact/escalation path and exercises.
D-12	Define export/deletion across providers and backups	Deletion may be incomplete if caches/backups/third-party stores are omitted.	Store inventory, retention exceptions, completion checks and auditable request status.

14. References

ISO/IEC/IEEE 29148:2018, *Systems and software engineering — Life cycle processes — Requirements engineering*. https://www.iso.org/standard/72089.html

ISO/IEC/IEEE 42010:2022, *Software, systems and enterprise — Architecture description*. https://www.iso.org/standard/74393.html

ovikrai, *SRS-Template-IEEE*. https://github.com/ovikrai/SRS-Template-IEEE

GitHub Docs, documentation for Markdown, Issues and repository version control. https://docs.github.com/

DēGlitch Software Requirements Specification v1.2 — Expanded Features and Cost Baseline, 26 October 2026.

DigitalOcean pricing references recorded in v1.2: Droplets, CPU-Optimized Droplets, Load Balancers, Managed Databases/PostgreSQL. https://www.digitalocean.com/pricing/droplets ; https://www.digitalocean.com/products/droplets/cpu-optimized ; https://www.digitalocean.com/pricing/load-balancers ; https://docs.digitalocean.com/products/databases/postgresql/details/pricing/

Hetzner, price adjustment dated 15 June 2026. https://docs.hetzner.com/general/infrastructure-and-availability/price-adjustment/

Cloudflare DDoS Protection and DNS FAQ. https://developers.cloudflare.com/ddos-protection/ ; https://developers.cloudflare.com/dns/faq/

Firebase documentation/pricing and security references as listed in the v1.2 baseline; verify current prices, quotas, and terms before procurement.

Vendor quotes, product/privacy policies, dependency licenses, platform API documentation and deployment-specific security references shall be recorded at implementation/release time.

Appendix A. Change record

Change ID:
Requested by / date:
Affected requirements and use cases:
Reason / user need:
Proposed change:

Impact on architecture, interfaces, data, security/privacy, tests and cost:
Compatibility / migration / rollback:
Decision and rationale:
Owner / decision date:
Version and evidence links:

Appendix B. Review finding record

Location: section / requirement ID / use case / figure / table
Type: completeness | ambiguity | consistency | verifiability | traceability | security/privacy | operations | other
Severity: critical | major | minor | editorial
Finding:
Impact:
Recommended correction:
Disposition and rationale:
Owner / target version:

Appendix C. Status definitions

Planned: included in scope but implementation has not been evidenced.

Implemented: code/configuration exists; acceptance is not yet implied.

Tested: a recorded test has executed and has an outcome.

Verified for release: all defined acceptance criteria for the stated release configuration have passed, evidence is available, and known limitations are dispositioned.

Conditional: applies only after the related feature, provider, platform or processing purpose is explicitly enabled under the required design and governance controls.

TBD: a decision or threshold must be set before the affected requirement can be formally verified.

A specification describes required product behavior. It does not, by itself, demonstrate that production infrastructure, an identity provider, an AI service, a custom trained model or any particular platform integration has been deployed. Every production claim must be backed by release-specific evidence.